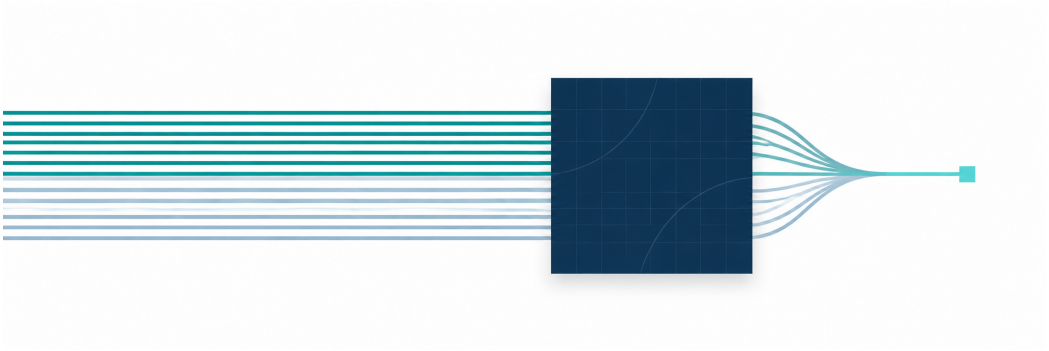
香山昆明湖 V2

自定义向量点积指令

三阶段技术文档合订本

技术报告



工程组合： 0f ＋ firstdot V2 ＋原 C8

团队名称： 待团队确认

成员： 待团队确认

学校： 待团队确认

材料日期： 2026 年 11 月 2 日

文档修订： 2026 年 11 月 5 日

目录

第一阶段：环境验证与向量执行过程 1

1.1 验证环境与协同仿真框架 1

1.1.1 构建链路与运行产物 1

1.2 环境部署与运行步骤 3

1.2.1 WSL 资源配置 3

1.2.2 基础工具安装 4

1.2.3 JDK 安装与校验 4

1.2.4 Verilator 安装与环境核查 4

1.2.5 程序构建环境 5

1.2.6 集成构建与运行入口 5

1.2.7 构建资源与并行恢复 8

1.2.8 程序镜像、装载与启动 9

1.3 启动差分故障的定位与修复 10

1.4 hello 与 vadd 运行结果 12

1.4.1 10 月 25 日英文输出与运行验证 13

1.4.2 定宽加法与字节结果 13

1.5 基于波形的向量执行过程 14

1.5.1 执行、写回与提交 15

1.5.2 指令、事件与波形联合定位 17

1.5.3 10 月 25 日含时钟的波形调试记录 18

1.6 阶段结论与复现资料 18

1.6.1 阶段结论与材料入口 18

1.6.2 源码版本与依赖恢复 18

1.6.3 完整工具版本 19

1.6.4 跨环境复现条件 20

1.6.5 10 月 25 日验证的版本与产物 21

1.6.6 技术资料索引 21

1.7 原始操作日志 **21**
1.7.1 10 月 25 日生产模型原始输出 21
1.7.2 10 月 25 日 FST 模型原始输出 22
1.7.3 程序构建原始日志 22
1.7.4 历史 hello 与 vadd 运行摘要 23
1.7.5 NEMU 构建原始日志 24
1.7.6 EMU 构建原始日志 26
1.7.7 工具与编译链版本原始输出 28
1.7.8 固定子模块版本清单 29
1.7.9 原始波形事件记录 32
1.7.10 向量运算单元事件记录 33

第二阶段：向量点积设计与功能验证 **34**

2.1 指令语义与微架构设计 **34**
2.1.1 点积定义、位宽与结果布局 34
2.1.2 指令编码与合法配置 35
2.1.3 架构参考伪代码 39
2.1.4 P/R 两槽数据通路 39

2.2 RTL 功能单元扩展与父级接入 **42**
2.2.1 译码、微操作与配置依赖 42
2.2.2 逐拍状态转移与握手 43
2.2.3 VFEX4 派发与共享资源 48
2.2.4 时钟门控、唤醒与写回 50

2.3 基础功能验证 **51**
2.3.1 正数点积测试 53
2.3.2 负数点积测试 53
2.3.3 边界点积测试 53
2.3.4 完整定向向量与精确预期 54
2.3.5 数据布局、别名与异常测试 55
2.3.6 随机输入与完整结果读回 56

2.4 协议与系统验证 **57**

2.4.1 反压期间的结果与身份保持 57
2.4.2 ROB 取消与复位优先级 58
2.4.3 DIV 切换、取消清理与关钟 60
2.4.4 写回身份与全核结果检查 62
2.4.5 控制场景的组合与回归覆盖 63

2.5 设计演进、参数扩展与取舍 63
2.5.1 吞吐与实现成本 63
2.5.2 本阶段小结与接口衔接 64

2.6 复现资料与接口索引 64
2.6.1 汇编封装与编码冲突记录 64
2.6.2 源码接入位置与职责 64
2.6.3 参考模型的构建与运行身份 66
2.6.4 事件记录与检查不变量 66
2.6.5 原 O2 分阶段判据与负控 67
2.6.6 RVV 对照的功能等价条件 68
2.6.7 待确认规格与变更记录 68
2.6.8 技术资料索引 69

第三阶段：协同仿真与性能分析 70

3.1 NEMU 扩展与协同仿真 70
3.1.1 独立参考模型与架构行为 70
3.1.2 指令译码与异常处理 72
3.1.3 提交边界上的差分比较 74
3.1.4 高位注错与正常参考对照 75

3.2 RVV 对照与周期测量方法 77
3.2.1 等价 RVV 指令序列 77
3.2.2 计数窗口与实际执行边界 77
3.2.3 工作负载、输入与重复测量 80
3.2.4 加速比与统计方法 81

3.3 点积与矩阵向量计算性能 82

3.3.1 四类点积微基准 82
3.3.2 单次调用与独立批量 84
3.3.3 依赖链与流式循环 85
3.3.4 每次点积的平均成本 85
3.3.5 GEMV 调用与配置复用收益 86
3.3.6 硬件与软件收益的对应关系 86

3.4 普通 C 接口与软件包接入 87
3.4.1 firstdot 与 C8 的调用组织 87
3.4.2 输入输出布局与调用约定 88
3.4.3 编译期选择与 RVV 回退 89
3.4.4 对象、装载镜像与调用表核对 89
3.4.5 接入检查器的正负控制 90
3.4.6 接入结果与功能记录复用 92

3.5 面积、时序与实现成本 93
3.5.1 评估条件与主要结果 93
3.5.2 寄存状态与综合面积 93
3.5.3 时序约束与路径分类 94
3.5.4 父级关键路径与改进方向 95
3.5.5 取消关钟与成本结果 96
3.5.6 门控修复的工程取舍 96

3.6 验证汇总与设计结论 96
3.6.1 功能、复现与性能检查汇总 96
3.6.2 优化候选的实测取舍 99
3.6.3 阶段结论 100

3.7 工程复现资料 100
3.7.1 固定版本与主要运行产物 100
3.7.2 两槽结构对照的样本记录 101
3.7.3 硬件对照与工具身份 101
3.7.4 离线复算与模型数据 102
3.7.5 取消关钟修复的对照输入 102
3.7.6 修复前后逐窗周期核对 103

3.7.7 综合面积与完整接口 104
3.7.8 五档时序详细结果 104
3.7.9 复算入口与实验归档 105
3.7.10 C 接口与编译开关 106
3.7.11 对象构建与接入命令 106
3.7.12 链接对象与检查入口 107
3.7.13 宿主运行记录与并发审计 107
3.7.14 工程使用与回退资料 108
3.7.15 技术资料索引 108

第一阶段：环境验证与向量执行过程

工程组合：0f ＋ firstdot V2 ＋原 C8　材料日期：2026 年 11 月 2 日

本阶段记录香山仿真环境部署、hello/vadd 运行与向量执行调试。2026 年 10 月 9 日记录展示环境建立、启动差分修复和原波形过程；10 月 25 日的部署复核使用既有硬件构建产物，运行配置个人标识的英文 hello 程序，并完成含时钟信号的 FST 调试链。两组实验分别按各自源码、输入镜像与事件坐标说明。

1.1　验证环境与协同仿真框架

本阶段建立香山 RTL 仿真器、NEMU 参考模型与裸机程序协同工作的运行入口。hello 程序用于检查启动流程和串口输出，vadd 程序用于观察向量配置、数据加载、按元素运算、写回与提交。通过共同启动镜像和差分比较，将工具链生成的产物与实际运行结果连接起来，为后续指令扩展提供可复用的环境。

1.1.1　构建链路与运行产物

协同仿真由硬件执行端、软件参考端和目标程序共同组成。香山 EMU 负责模拟硬件的执行过程，NEMU reference 负责按指令语义推进参考状态，裸机程序提供可重复的输入并检查软件可见结果。共同 flash 连接复位后的启动过程与程序入口。这几类产物在一次运行中同时参与工作，部署过程因此围绕它们之间的接口组织。

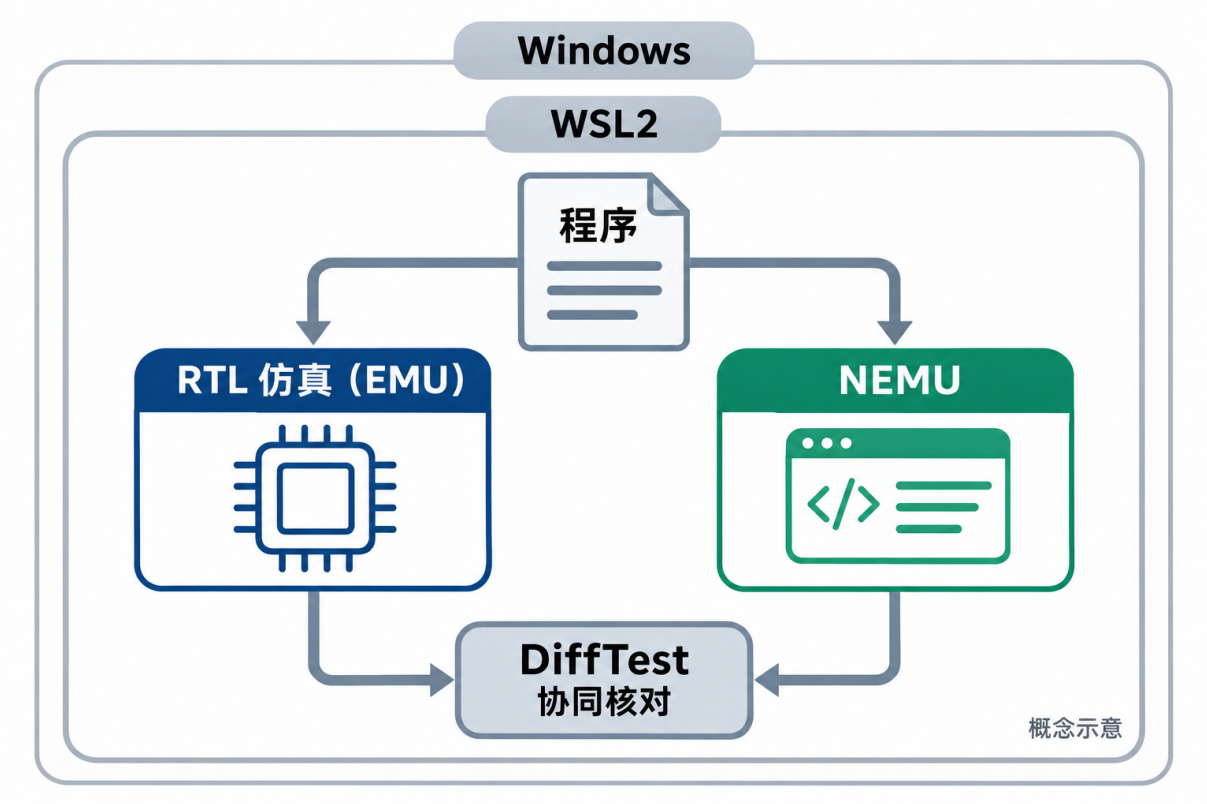
硬件侧首先通过 Java 与 Mill 解析 Scala、Chisel 工程及其依赖，再由 firtool 处理硬件中间表示，供 Verilator 生成宿主侧仿真代码。宿主 C++ 编译器将这些代码构建为 EMU。每一步的产物都会成为下一步的输入：Scala 工程配置影响生成的硬件，硬件生成结果影响仿真代码，宿主工具链则决定最终可执行文件的构建方式。运行环境的版本表对应这条生成链，便于沿产物关系定位问题。

目标程序采用另一条工具链。RISC-V GCC 编译程序与所需运行支持，链接器确定代码和数据布局，binutils 提供二进制转换及反汇编。NEMU reference 在宿主侧构建为可加载的参考库，由 EMU 运行参数指定。这样，硬件仿真器、目标程序和参考模型具有各自清楚的构建入口，并在差分运行时汇合。

实际组织时，构建日志与运行日志分别保存。构建记录说明工具如何生成产物；运行记录说明加载器选择了哪些文件、程序如何启动以及最后如何结束。两者通过产物路径、文件摘要和运行参数对应。对一个具体问题，先找到相关产物，再沿依赖链向上查看生成步骤，能够较快区分源码、工具和装载配置三个层面的原因。

表 1-1 构建链路与运行产物

产物	生成入口	运行中的作用	主要定位信息
香山 EMU	硬件生成与宿主编译	执行 DUT 仿真	硬件版本、配置、构建参数、文件摘要
NEMU reference	参考模型构建	提供架构状态参考	源码版本、配置、加载路径、文件摘要
程序 ELF/BIN	RISC-V 编译与链接	执行 hello 及 vadd 测试	入口、装载段、反汇编、镜像摘要
共同 flash	启动汇编与二进制转换	建立一致的启动输入	文件长度、内容摘要、显式加载参数



概念示意：共同输入与软硬件状态核对。技术依据见图 1-1。

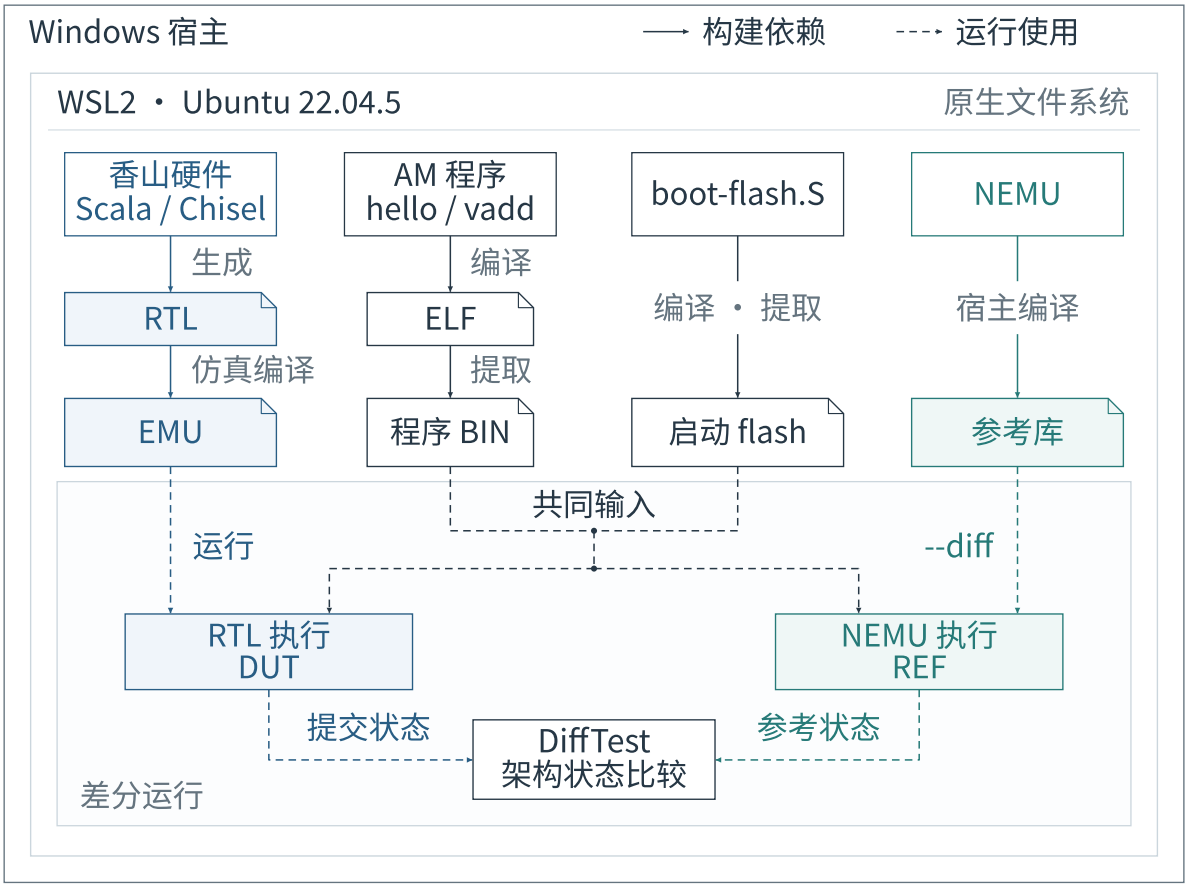


图 1-1　开发环境与软硬件协同验证框架

Windows 宿主中的 WSL2 环境承载构建与差分运行。实线为构建依赖，虚线为运行使用；程序 BIN 与启动 flash 是共同输入，EMU 和 NEMU 参考库分别提供 DUT 与 REF 功能。框图不表示独立操作系统进程或内部逐周期锁步。阶段一历史环境，未混入最终 0f 身份。

1.2　环境部署与运行步骤

1.2.1　WSL 资源配置

本次为 WSL 分配 20 GB 内存和 16 GB 交换空间。Windows 用户目录%UserProfile%下的.wslconfig 内容如下：

```
[wsl2]
memory=20GB
swap=16GB
```

保存正在进行的 WSL 工作后，在 PowerShell 执行 wsl --shutdown，重新进入 Ubuntu，再用 free -h 核对生效结果。原机器的交换文件路径为 D:\WSL\wsl-swap.vhdx；新环境按本机磁盘布局选择路径即可。

内存规划需要同时考虑 JVM、硬件生成工具和并行 C++ 编译进程。交换空间可为短时内存压力提供缓冲，但频繁换页会增加构建时间，因此它与可用物理内存具有不同作用。内存较小的机器应相应降低 JVM 堆与编译并行度，使多个进程的合计需求处于

可用范围内。构建日志同时记录并行参数和结束状态，便于区分资源配置问题与源码编译问题。

1.2.2 基础工具安装

完成配套源码检出后，在团队仓库根目录加载路径，并安装基础工具：

```
source team/scripts/activate.sh
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target base
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target llvm
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target mill
sudo apt-get install -y cmake ninja-build gtkwave
```

入口来自固定 xs-env 的 setup-tools.sh。软件源提供的补丁版本可能随时间变化，安装步骤完成后仍需通过环境检查确认实际版本。这样可以把“安装入口相同”与“得到的工具版本相同”分别记录。

1.2.3 JDK 安装与校验

原安装入口使用 Oracle 21/latest 地址。为了恢复本阶段的 21.0.12，团队固定 GraalVM 安装脚本使用对应 Linux x64 归档及官方 SHA-256，安装到用户目录：

```
export JAVA_HOME="$HOME/.local/share/graalvm-jdk-21.0.12"
bash team/scripts/install-graalvm.sh
source team/scripts/activate.sh
```

归档校验用于确认下载字节，版本检查用于确认实际启动的 Java。二者配合，可以发现下载路径正确但环境变量仍指向另一套 JDK 的情况。新终端继续导出相同 JAVA_HOME。原环境中已有 /opt/graalvm-jdk-21 的安装，也可以在确认版本后沿用；脚本遇到版本不同的已有安装会停止，以保留当前配置。

安装方法参考 Oracle GraalVM 安装说明。固定下载项为 21.0.12 Linux x64，校验来源为官方 SHA-256。工具更新时同步记录新版本及相应验证结果，便于将新产物与本阶段历史产物区分。

1.2.4 Verilator 安装与环境核查

Verilator 从固定提交构建，以下流程显式使用两个编译任务。构建并行度应与本机可用资源相配合；阶段一 EMU 编译记录中的四任务参数与这里安装工具本身的参数分别记录。

```
sudo apt-get install -y help2man perl autoconf flex bison libgoogle-perftools-dev \
  libjemalloc-dev numactl perl-doc zlib1g-dev liblz4-dev libfl2 libfl-dev
mkdir -p "$HOME/riscv-tools"
git clone https://github.com/verilator/verilator.git "$HOME/riscv-tools/verilator"
```

```
cd "$HOME/riscv-tools/verilator"
git checkout ea338be98e1e838d3518809ce8899f85a009963c
unset VERILATOR_ROOT
autoconf
./configure CC=/usr/local/llvm/bin/clang CXX=/usr/local/llvm/bin/clang++ LINK=/usr/local/llvm/bin/clang++
make -j2
sudo make install
```

编译前清除旧的 VERILATOR_ROOT，并显式选择已安装的 Clang，以便构建输入与记录一致。安装后回到团队仓库根目录检查环境：

```
source team/scripts/activate.sh
bash team/scripts/check-environment.sh
```

环境检查核对配套源码提交、香山递归子模块和关键工具版本。香山提交可以包含本队新增文档，但应保留指定硬件基线祖先。检查结果用于确认构建条件；硬件行为则由对应版本的仿真记录说明。这样的分工使环境恢复与功能验证各自具有清楚的输出，也便于定位失败发生在哪一层。

1.2.5 程序构建环境

原记录的工作目录为 /home/riscv/riscv-lab/xs-env/XiangShan。复用其他机器或其他检出时，在实际团队仓库根目录加载环境即可，并保留本次路径和提交记录。

activate.sh 将当前团队仓库设置为 NOOP_HOME，按照 TEAM_XS_ENV_ROOT 指向配套 NEMU 和 AM，并设置 LINUX_GNU_TOOLCHAIN=1。这些变量把硬件仓库、参考模型和程序构建入口连接起来，运行前可据此核对各组件来自哪套环境。

AM 构建向量工作负载时仍需显式指定 MARCH=rv64gcv。这使编译器采用包含向量扩展的目标 ISA，与 hello-vadd 所用的向量装载、加法和存储指令一致。程序构建完成后，将 ELF、运行镜像、反汇编和实际运行命令一起归档，便于从源程序追踪到日志中的提交 PC。

1.2.6 集成构建与运行入口

原实验使用 Windows 下的 WSL2 Ubuntu 22.04.5，宿主处理器为 Ryzen 7 7840H，可用物理内存约 27.7GiB。WSL 配置 20GB 内存与 16GB swap，硬件生成、编译和仿真在 WSL 原生文件系统中执行。下表列出阶段一的主要版本与运行设置；完整子模块锁和工具输出列于文后。

表 1-2 集成构建与运行入口

组件	固定版本或设置
xs-env	b0b506481e839a5df8a580e76f5aa246dc60fe7c
原香山基线	16ae9ddcda54fc9a2fddffad73174cf793ac7814
原 NEMU 基线	a48a11f208ff81ad765591d83271d1b339fb3861
nexus-am	086a238062fdfb3281b745a537b361983d0154b5
JDK / Mill / firtool	21.0.12 / 0.12.3 / 1.62.1
Verilator	5.052，固定 ea338be 提交
RISC-V GCC / binutils	11.4.0 / 2.38
DUT	MinimalConfig、单核、VLEN128
原波形构建	EMU_THREADS=1、EMU_TRACE=fst

部署按照依赖关系推进：配置 WSL 资源，检出配套源码和递归依赖，核对工具版本，分别构建 NEMU reference 与香山 EMU，编译 hello/vadd 及启动镜像，最后执行差分运行。命令、退出状态、版本和文件摘要随各阶段产物一起保存。

复现入口由以下脚本组成。工作负载中的团队标识由成员确认后填写；构建与运行输出应使用独立目录，以便对应具体实验。

```
source team/scripts/activate.sh
bash team/scripts/check-environment.sh
bash team/scripts/stage1.sh nemu
bash team/scripts/stage1.sh emu
TEAM_ID= 正式团队标识 bash team/scripts/stage1.sh workload
bash team/scripts/stage1.sh run
```

部署中遇到的首个差分问题位于启动 PC 0x10000004。检查发现两端默认 flash 分别包含 3 条和 6 条启动指令。随后使用同一 boot-flash.S 生成 12 字节镜像，并在启动命令中显式指定共同 flash、128MB RAM 和固定 reference。完整定位过程见后文“启动差分故障的定位与修复”。

概念示意：部署、运行与证据归档。技术依据见图 1-2。

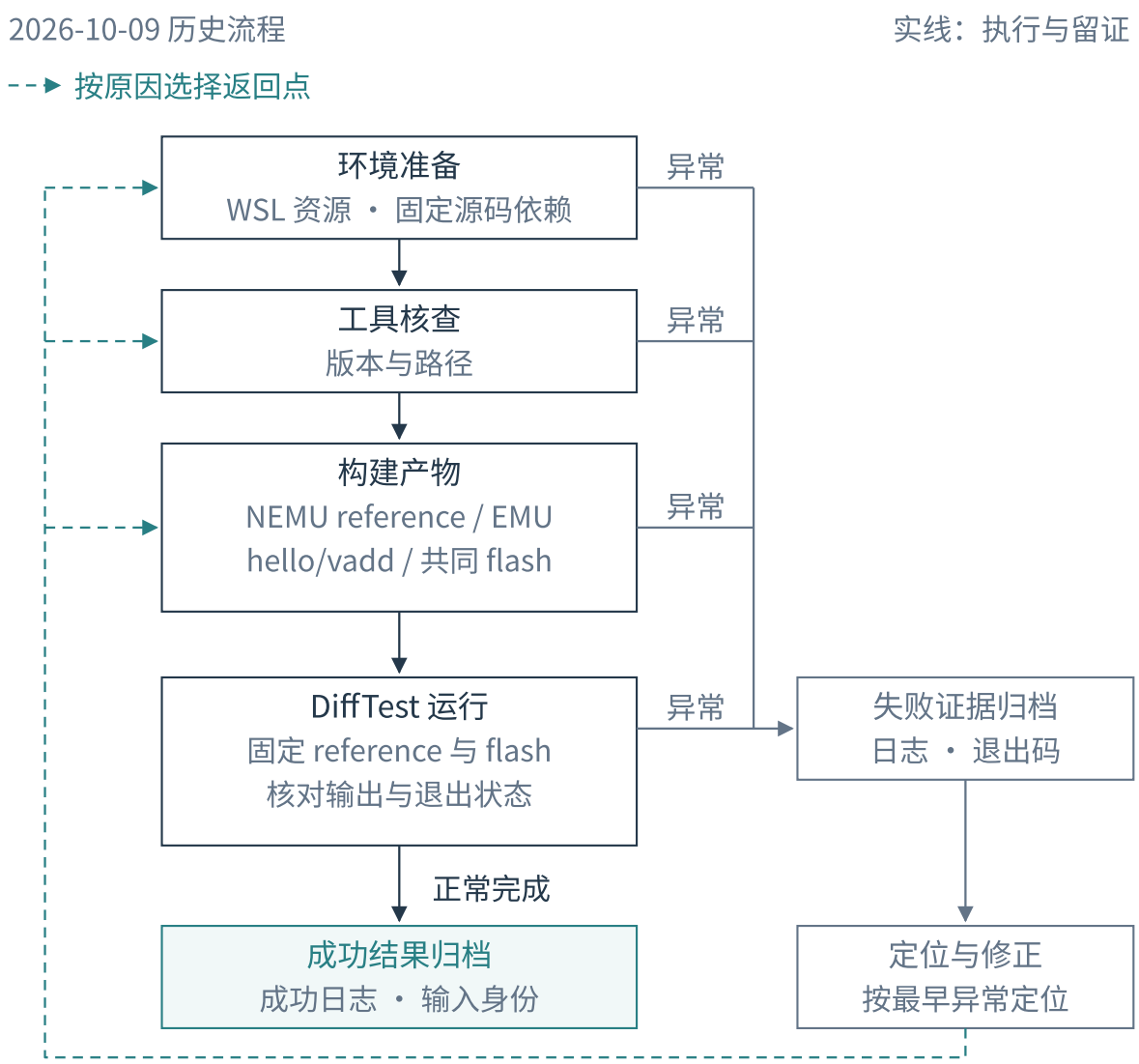


图 1-2 环境部署与验证执行流程

环境准备、工具核查、产物构建、DiffTest 运行和成功归档构成主流程；失败证据另行归档。定位修正后按故障原因返回准备、工具核查或构建环节，虚线为处理路径，不表示每一类历史故障均实际发生，也不表示脚本自动诊断或修复。2026-10-09 阶段一流程：香山 16ae9dd、NEMU a48a11f。

1.2.7 构建资源与并行恢复

香山 EMU 的构建涉及多个资源特征不同的阶段。Java 进程需要堆空间解析工程和生成硬件，firtool 处理硬件表示，Verilator 生成仿真代码，C++ 阶段则启动多个编译进程并产生对象文件。宿主文件缓存与操作系统也占用内存。资源配置要为这些部分保留共同工作的空间，单独增大某一进程的内存上限并不等于增加了整条构建链的可用资源。

原实验主机可用物理内存约 27.7GiB，WSL 配置 20GB 内存和 16GB swap。这里的 20GB 是 WSL 内存上限，swap 用于承接部分内存压力；二者在访问成本和作用上不同。持续发生换页时，构建墙钟时间可能明显增加。因此，资源记录同时关注内存配置、并行度、进程状态和日志进展，而不是只记录一项内存容量。

2026 年 10 月 9 日的构建记录中，C++ 编译在 20:42:15 中断并恢复，并行度调整为

4，已完成对象保留；EMU 随后在 20:53:32 完成构建。这个过程体现了分阶段调节并发的方式。复现脚本中的较低默认并发适合控制初次构建的内存压力，原记录中的-j4 则对应当时的实际执行参数。两种配置都通过各自命令和时间记录定位。

并行资源也应与仿真配置分开描述。阶段一 DUT 采用单核，原波形构建使用单仿真线程；宿主可以同时运行多少独立任务，属于另一项调度条件。构建并发影响宿主资源和等待时间，程序的 EMU 周期统计则来自仿真内部。将构建耗时、宿主执行时间和 DUT 周期分别记账，便于后续评价环境效率与指令性能。

对象文件和构建缓存有助于恢复长作业，但它们对应具体源码、工具和编译参数。沿用已完成对象时，继续保存原命令与依赖身份；调整会影响生成内容的参数时，按照构建系统的依赖关系更新相应产物。报告保留中断和恢复记录，使最终 EMU 的形成过程具有连续的解释，而非只有最后一条成功提示。

- 2026 年 10 月 9 日 20:42:15，中断并恢复 C++ 构建，将并行度调为 4，保留已完成对象。
- 2026 年 10 月 9 日 20:53:32，EMU 构建结束，状态文件记录 exit_code=0。
- 2026 年 10 月 9 日 20:55:26，workload 与共同 flash 构建结束，记录 WORKLOAD_BUILD_PASS。
- 原正常运行于 21:00:28 结束，退出状态为 0。

1.2.8 程序镜像、装载与启动

ELF、BIN 和 flash 在运行中承担不同作用。ELF 包含符号、代码与数据段信息，可用于查看入口地址和反汇编；BIN 提供加载器使用的程序字节；flash 提供复位后的启动代码。启动镜像与目标程序是相互配合的两个输入，分析程序尚未到达 C 入口的差异时，flash 及装载配置通常是重要线索。

反汇编连接源程序和硬件观察点。编译器选择指令，链接器安排代码与数据，最终地址由这组构建条件共同决定。通过目标指令的 PC 查找同次构建的机器字，再与提交记录对应，可以确定波形中观察的是哪一次操作。即使 C 代码保持原样，链接参数或依赖库改变后，PC 表也可能随之变化。

显式运行参数把构建产物与一次执行绑定。阶段一在命令中指定共同 flash、128MB RAM 与固定 reference，使两端启动输入可以直接检查。共同文件的含义是字节实物一致；两端如何将它装入各自平台地址空间，仍由各自装载实现决定。描述共同启动条件时，文件内容、装载参数和地址映射分别列出，因果关系更清楚。

hello 程序提供了启动链路中的一个软件观察点。串口输出表明程序已进入对应的软件路径，随后 vadd 用例设置向量状态、执行运算并打印比较结果。原 hello 中的 IPv4 来自 WSL 宿主，用于识别当时环境；团队字符串为原记录中的 riscv-vdot-team。正式身份信息与历史输出分别保存，便于在更新展示材料时继续检索原运行。

装载问题与运算问题也具有不同的日志特征。启动日志给出 flash、程序和 reference 的选择，软件输出给出已到达的程序阶段，差分信息给出首次观察到的架构差异。按执行顺序阅读这些信息，可以先确定故障发生在启动之前、启动过程中还是进入测试程序之后，再选择相应源码和数据继续分析。

1.3 启动差分故障的定位与修复

原失败的首次差异位于 PC 0x10000004，涉及 t0 寄存器，记录的 instrCnt 为 2。这个位置处于启动阶段，早于 hello/vadd 测试主体。定位首先围绕两端的初始执行序列展开：检查启动输入，查看该位置的机器字，并与启动文件的反汇编对应。

检查发现，两端默认 flash 分别包含 3 条和 6 条启动指令。默认输入差异使两端在到达同一程序入口前就经历了不同的执行过程。修复采用一份 boot-flash.S 生成共同的 12 字节 boot-flash.bin，再通过显式 flash 参数将同一实物交给 DUT 与 NEMU。三条启动指令的文件偏移、机器字与反汇编保留在图数据附件中。

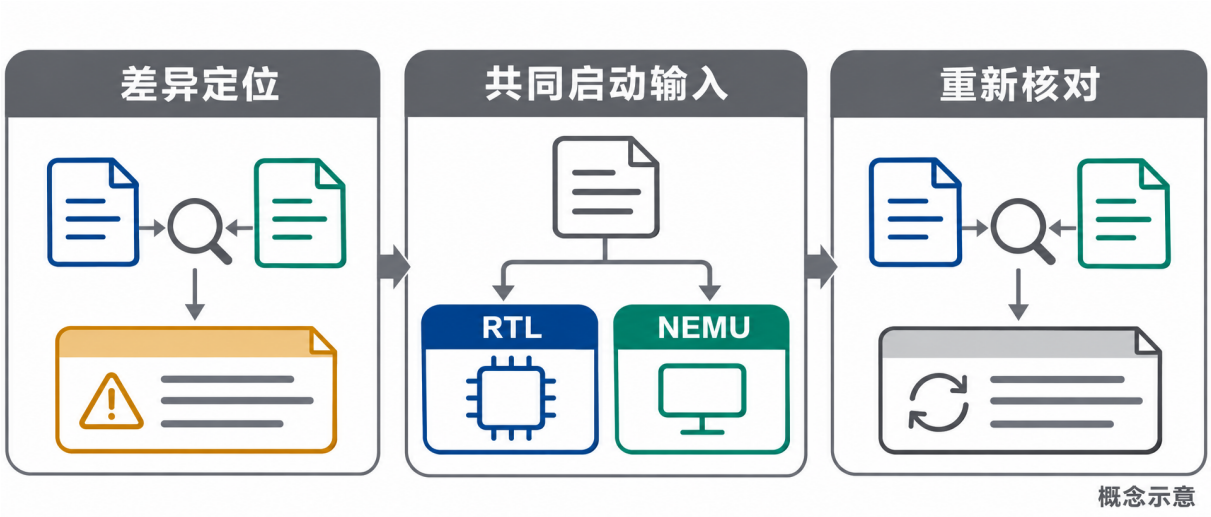
文件偏移定位 flash 中的指令，启动 PC 定位处理器执行地址，两者通过装载映射关联。同一份 12 字节 boot-flash.bin 分别交给 DUT 与 NEMU 装载。首次差异 PC 用于定位原故障，后续终止或 ABORT 事件的地址按对应事件记录。

恢复运行继续启用 DiffTest，执行三类 vadd 用例，记录 24 个元素通过、GOOD TRAP 和 exit 0。失败记录与恢复记录分别保存，前者解释问题如何被发现，后者说明统一启动输入之后的结果。报告中的修复过程由“首次差异—默认输入检查—共同文件—正常运行”组成，每一步都可以回到原日志或构建实物。

表 1-3 启动差分故障的定位与修复

排查步骤	使用材料	定位结果
查找首次差异	原失败日志、PC 与 t0 状态	差异发生于启动阶段
比较启动输入	默认 flash 与反汇编	两端默认指令序列不同
统一启动文件	同一 boot-flash.bin 及显式参数	两端加载同一 12 字节实物
查看恢复结果	软件摘要、GOOD TRAP 与退出状态	hello/vadd 完成正常差分运行

这一案例也给后续故障分析提供了合适的起点：先定位最早出现差异的阶段，再分析对应输入和执行状态。若差异仍在启动入口，应优先检查 flash 与装载；若已经进入向量程序，则转向指令、寄存器和结果记录。分层定位可以缩小检查范围，并使每次参数调整与观察结果直接对应。



概念示意：统一启动输入与差分恢复。技术依据见图 1-3。

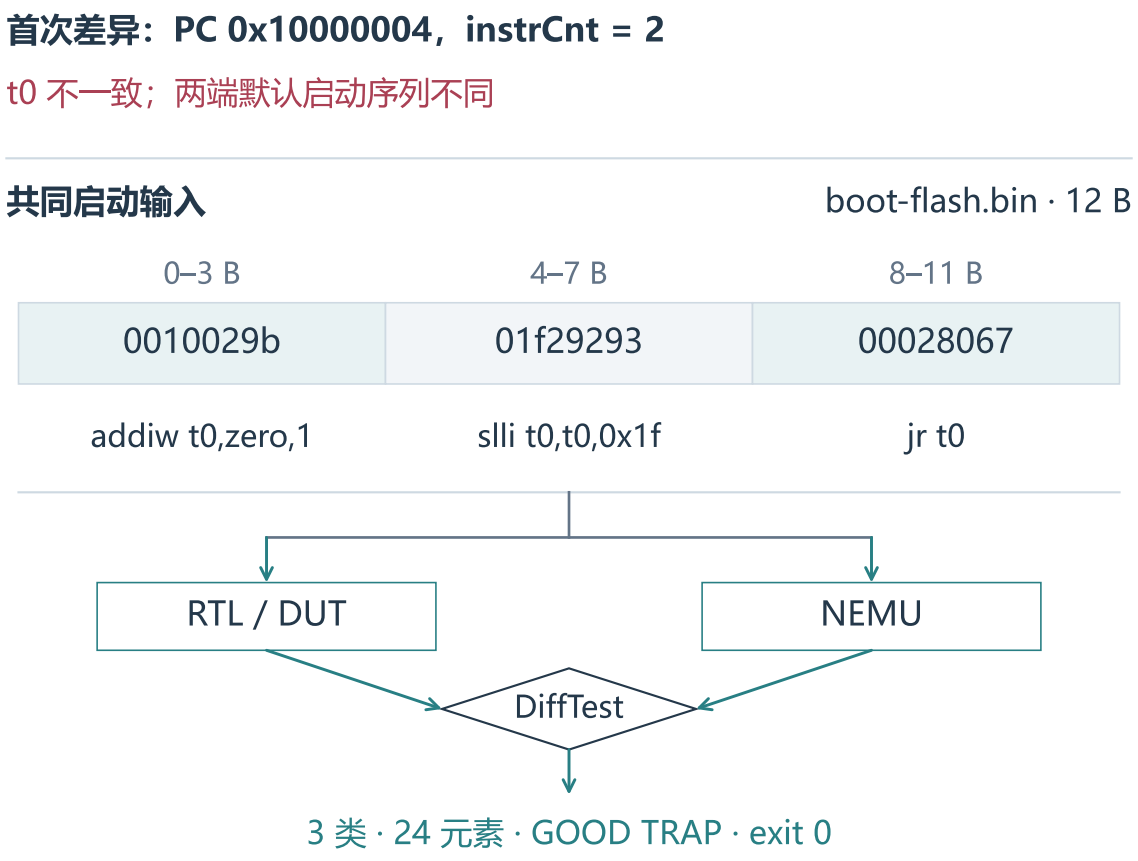


图 1-3　启动输入统一与差分恢复

首次差异位于启动 PC 0x10000004，instrCnt 为 2。共同 boot-flash.bin 的 12 字节由三条机器指令组成，图中 0–3、4–7、8–11 为文件字节偏移。显式向 RTL 与 NEMU 提供同一启动文件后，三类共 24 个元素通过检查，以 GOOD TRAP 和 exit 0 结束；共享文件不等于两端物理映射地址相同。

1.4 hello 与 vadd 运行结果

hello 输出包含团队标识和 WSL 宿主 IPv4。2026 年 10 月 9 日记录使用的团队字符串为 riscv-vdot-team，本文按原日志保留。正式团队名称等身份信息集中列于封面。

vadd 程序采用 SEW8、LMUL1 和 VL8，依次执行两次向量加载、一次 vadd.vv 及一次向量存储，再将八个结果字节与标量参考逐项比较。三个用例分别检查基本加法、负数补码和定宽回绕。

表 1-4 hello 与 vadd 运行结果

用例	输出字节
正数	9 9 9 9 9 9 9 9
负数补码	254 252 250 248 246 244 242 240
边界回绕	128 127 0 0 0 0 254 254

正常运行记录包含 HELLO_VADD_PASS cases=3 lanes=24、GOOD TRAP 和真实退出状态 0，运行期间启用 DiffTest。EMU 统计为 16557 条指令、42962 周期，统计对象是包含启动、UART 与结果校验的完整程序。各用例的数值解释见本节“定宽加法与字节结果”，事件关系见“基于波形的向量执行过程”。

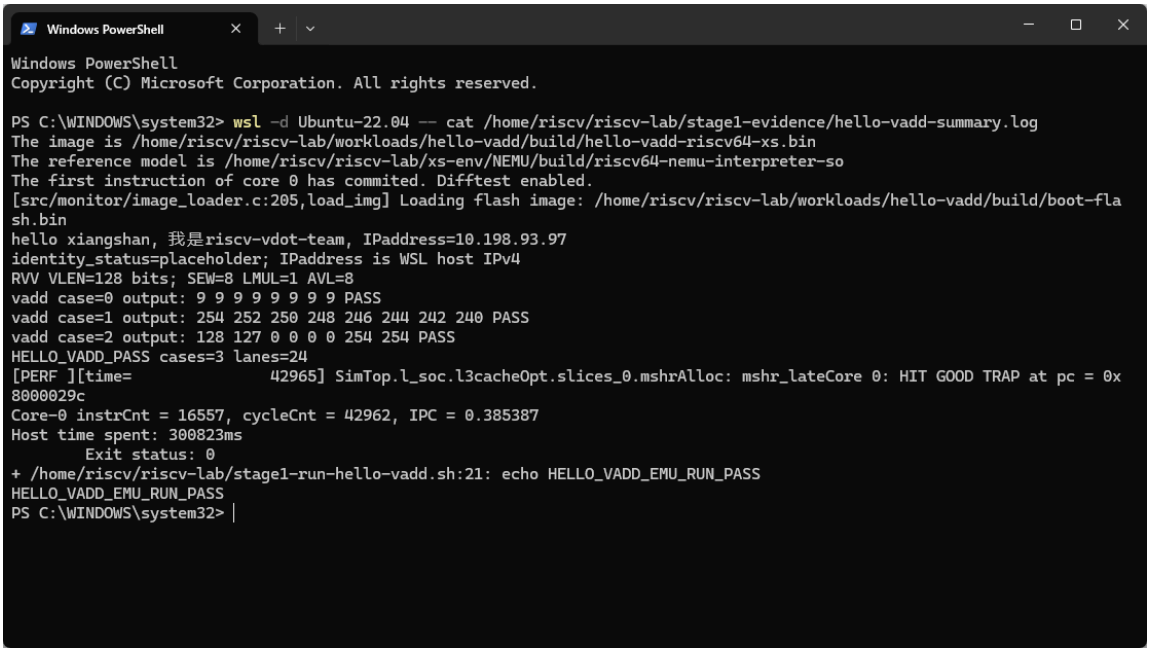


图 1-4 hello 与 vadd 正常结束输出

运行日志记录三类 vadd 输出、DiffTest 比较和 GOOD TRAP 终态。

1.4.1 10 月 25 日英文输出与运行验证

2026 年 10 月 25 日（UTC+8），补证实验使用个人标识 LiZufan001 构建 hello/vadd 程序，身份状态为 configured。原始串口输出如下，其中 IPv4 为构建宿主地址：

```
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
```

同一 BIN 分别在已封存生产 EMU 和独立 FST 模型上启用普通 NEMU DiffTest，两次运行均通过 3 组、24 个元素检查，记录HELLO_VADD_PASS cases=3 lanes=24、GOOD TRAP 及退出状态 0。每次统计均为 16621 条指令、43249 周期，覆盖启动、UART 和结果校验的整程序。原始输出与运行回执保存了具体输入、命令和结束状态。

1.4.2 定宽加法与字节结果

阶段一 vadd 采用 SEW8、LMUL1 和 VL8，参与用例比较的是八个 8 位元素。程序通过两次向量加载取得输入，执行按元素加法，再将结果存回内存供标量比较。三个用例使用相同的执行入口，分别选择便于观察基本运算、补码和回绕行为的数据，从而让环境验证具有不同的数值特征。

RISC-V V1.0 规范定义同宽向量整数加法，并说明未特别指定的整数运算在溢出时回绕。八位加法因此按模 256 保存结果。若把一个结果字节记为 r，它的无符号表示位于 0 到 255 之间；按八位补码解释时，最高位为 1 的字节对应负值。例如原负数用例打印的 254 是十六进制 FE，对应补码整数 −2。打印形式由格式选择决定，存储的八位比特保持相同。比较时以相同宽度和截断规则计算标量参考，就能直接核对程序结果。

正数用例的八个结果字节均为 9，低 64 位显示为 0x0909090909090909。负数用例按元素顺序打印 254、252、250、248、246、244、242、240，对应低 64 位 0xf0f2f4f6f8fafcfe。观察十六进制向量值时，右端是最低字节，因而与按元素序号递增打印的字节列表方向不同。把两个显示形式按字节位置对应，可以避免把书写顺序差异当成数据差异。

边界用例输出 128、127、0、0、0、0、254、254，低 64 位为 0xfefe000000007f80。这里同时出现最高位变化、零和接近字节上限的值，适合检查定宽结果的表示。八个字节在向量低 64 位中的组合关系可以写成各结果字节乘以相应的 256 次幂后求和；因此逐元素列表与整体十六进制值是同一结果的两种表达。

VLEN128 说明向量寄存器的物理宽度，VL8 说明该用例的活动元素数。本文结果表按实际八个元素解释，整体寄存器宽度与有效运算范围分别标注。随后 vdot 使用低八字节参与点积并形成自定义结果布局，属于另一套指令语义，其低 32 位结果和高位规则在阶段二说明。

软件输出还保留了用例数量与比较数量。HELLO_VADD_PASS 中的 cases=3 对应三种输入组织，lanes=24 对应三次各八个活动元素。读者可以将该摘要与逐字节结果及 GOOD TRAP、exit 0 一起阅读，得到从数值比较到运行结束的完整解释。

1.5 基于波形的向量执行过程

取指和译码之后，向量指令经重命名、调度与操作数准备进入执行单元。两个 vle8.v 提供输入，vadd.vv 完成按元素加法，结果进入写回路径并在 ROB 提交。差分系统在架构状态比较点与 NEMU 对应状态核对，内部观察器则记录输入、输出和提交事件。

原波形提取的 vadd 提交 PC 为 0x80000138，机器码为 0x021101d7，vecwen=1、wdest=3。下表按这组指令身份关联三个用例的事件坐标与低 64 位结果。

表 1-5 基于波形的向量执行过程

用例	输入事件坐标	输出事件坐标	提交事件坐标	运算输出低 64 位
正数	19625	19626	19631	0x0909090909090909
负数	25460	25461	25466	0xf0f2f4f6f8fafcfe
边界	33009	33010	33015	0xfefe000000007f80

三个观察点展示了执行结果产生到架构提交的流水关系。表中数字是原 CSV 事件坐标，图中间距用于组织阅读；现有局部截图未显示时钟，因而本节按事件位置解释记录，不把坐标差换算为物理时间或固定指令时延。需要进一步查看握手时，可恢复对应完整波形，联合定位时钟、valid、ready 和结果信号。

原归档记载的完整 FST 路径为/home/riscv/riscv-lab/stage1-evidence/hello-vadd.fst，文件大小 220295598 字节。该记录属于 2026 年 10 月 9 日基线，恢复时以归档索引和文件摘要定位。最终 0f 环境的 hello/vadd 验证记录由 R5 验收单独绑定。

概念示意：向量操作的执行与状态观察。技术依据见图 1-5。

图 1-5 vadd 输入、输出与提交事件

三类 vadd 用例通过 PC 0x80000138、指令 0x021101D7 和目的 v3 关联。数字为 CSV 事件坐标。

1.5.1 执行、写回与提交

程序中的一条向量指令会经过多个硬件阶段。取指和译码确定指令及操作类型，重命名与调度安排数据依赖，执行单元计算结果，写回路径传递结果与身份，ROB 按架构顺序提交。执行结果出现与架构提交是不同观察点，二者之间还存在父级选择及在途状态的处理。

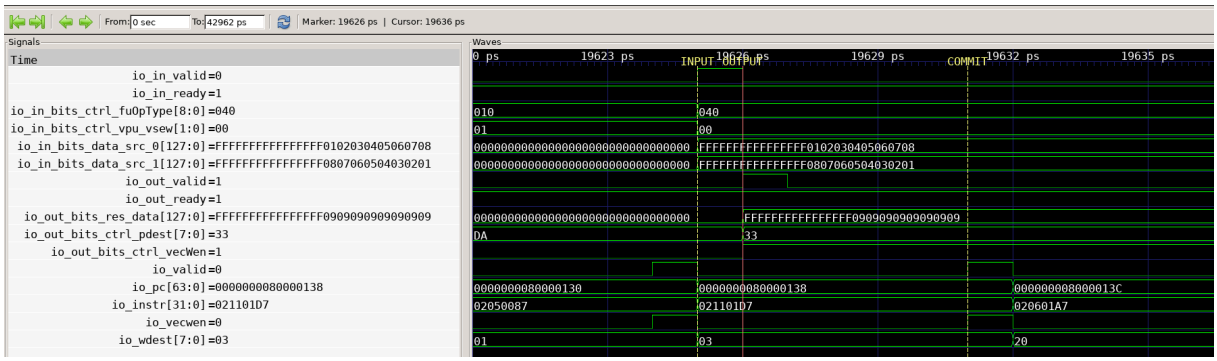
阶段一按 PC 0x80000138、机器字 0x021101d7 及目的 v3 关联 vadd 记录。提交侧 vecwen=1 说明该记录带有向量写入，wdest=3 给出对应目的。PC 和机器字共同提供静态指令位置，目的信息帮助确认被观察的结果。它们与用例顺序、事件坐标和结果值结合后，才能把三类软件输入准确连接到各自的硬件事件。

正数用例的输入、输出和提交事件分别位于 19625、19626 和 19631，负数用例为 25460、25461 和 25466，边界用例为 33009、33010 和 33015。这些数字沿用原 CSV 坐标，表示所列记录在波形中的位置。图中采用三列布局突出阶段关系，列间距离没有按原坐标比例绘制。

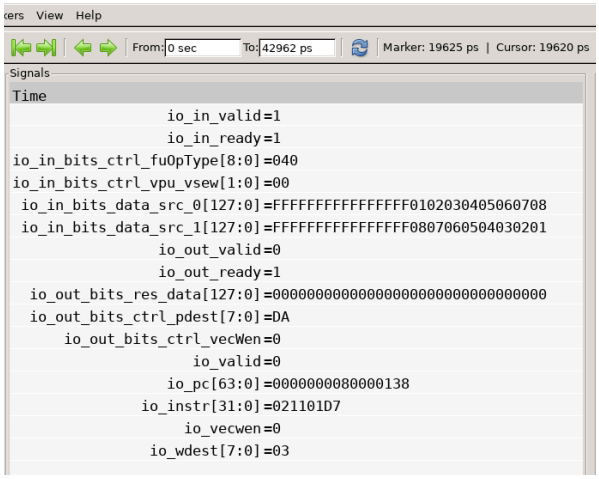
内部信号的含义还取决于采样位置。输入记录可能表示执行端可见的操作数，输出记录表示结果观察点，握手则需要结合 valid 和 ready 判断一次事务是否实际转移。阶段一图表按现有输入、输出与提交字段命名；需要讨论握手行为时，应在原波形对应位置同时查看双方条件。这样既能利用已有记录解释流水关系，也能清楚说明各字段承担的作用。

现有局部波形截图未显示时钟。截图中的 ps 是波形显示单位，事件坐标是用于定位的数值；EMU 周期统计和宿主运行时间则来自不同的计量位置。本阶段通过 PC、字段与坐标连接记录，没有为这些局部窗口建立物理频率或固定时延结论。

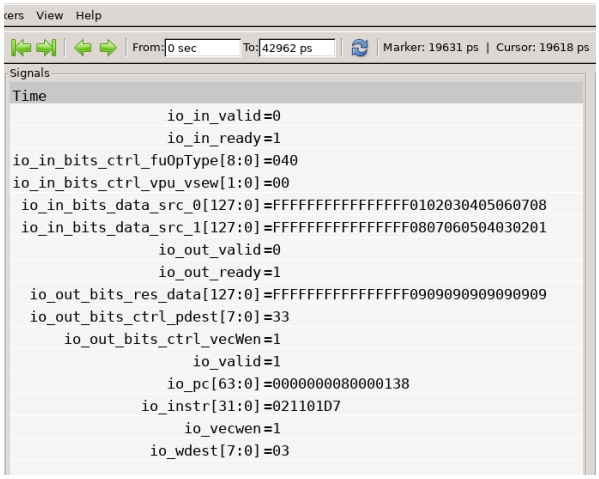
乱序执行中，程序依赖、执行资源及下游状态会影响执行和提交的间隔。报告用三个具体窗口说明该次运行的过程，后续性能分析则采用专门的 DUT 计数窗口和相同工作量对照。把事件定位与性能计量分开，读者就能分别回答“结果在哪里出现”和“规定工作用了多少周期”。



(a) 输出位置：19626 ps



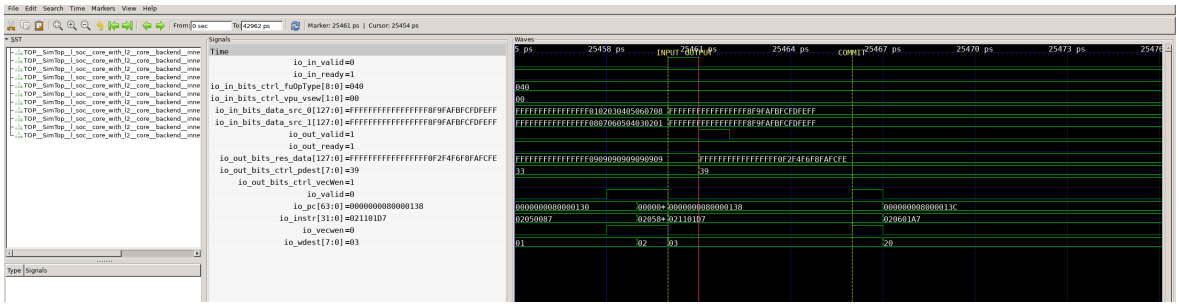
(b) 输入位置：19625 ps



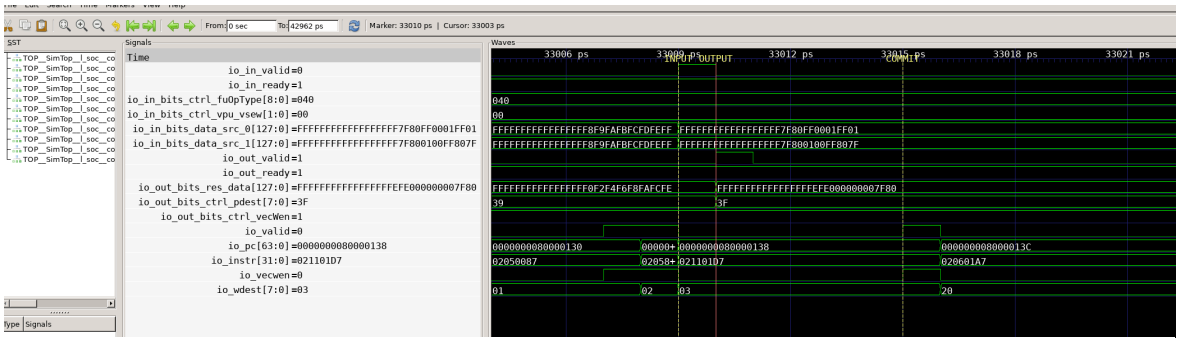
(c) 提交位置：19631 ps

图 1-6 vadd 正数用例的执行与提交波形

(a) 输出位置及相关事件；(b) 输入游标处信号；(c) 提交游标处信号。19625、19626 和 19631 为 selected VCD 中的事件坐标，截图单位为 ps，时钟信号未显示。



(a) 负数样例：输出位置 25461 ps



(b) 边界样例：输出位置 33010 ps

图 1-7 vadd 负数与边界用例结果波形

负数与边界样例的输出低 64 位分别为 F0F2F4F6F8FAFCFE 和 FEFE000000007F80，游标显示各自输入、输出与提交位置。

1.5.2 指令、事件与波形联合定位

波形、CSV 与反汇编提供互补的观察方式。反汇编给出静态地址与机器字，事件 CSV 按字段记录被提取的变化，FST 保留更完整的信号历史。局部截图适合快速呈现某一窗口，完整文件则适合围绕具体疑点展开查看。报告按这些用途组织图、表与附件。

一种直接的定位顺序是先从提交记录取得 PC，再在对应反汇编中找到指令，随后用事件坐标打开执行窗口。对 vadd，还可以核对机器字、目的 v3 和结果低 64 位；如果三者与软件用例一致，就可以进一步沿信号层级查看操作数和控制条件。这条路径把静态程序、软件输出和内部波形连接到同一条被观察指令。

阅读结果值时需要注意数据展示方向。日志可以按元素顺序输出十进制字节，波形窗口也可以把一组字节合并显示为十六进制。通过元素宽度和低位位置统一表示后，正数、负数和边界三个用例均可以按同样步骤阅读。数据解释与时间定位在这里是两项独立工作：先确认数值对应，再查看它出现在哪个观察点。

信号索引记录了提交与执行端的信号层级、位宽和标识，帮助读者解释 CSV 中的字段。检查某个字段时，可结合索引确定原信号；恢复对应完整波形后，再查看该坐标附近的上下文。报告通过精选窗口与附件索引兼顾主要过程的阅读和具体疑点的深入分析。

1.5.3 10 月 25 日含时钟的波形调试记录

10 月 25 日的独立 FST 模型使用同一组既有硬件生成物构建，保留真实时钟信号。最终提取覆盖两个 VialuFix、八个提交槽和 127 个信号，检查 33249 个完整上升沿，关联三次 FU 输入、三次 FU 输出与三次 DPIC 提交观测，共 9 条事件。

采样取每个局部上升沿前的稳定低相值，FST 原始时间与周期计数分别保存。DPIC 记录经过 Rob.scala 中 delay=3 的观察路径，其坐标表示提交接口的观测时刻。选通信号 FST 可在 GTKWave 中查看时钟、操作数、握手与事务标识。本节原波形图仍对应 10 月 9 日；10 月 25 日记录由独立波形与事件索引定位。

该次构建的向量 ALU 接口见固定源码中的VIAluFix 包装层，用于对照操作数、控制与结果的连接。

1.6 阶段结论与复现资料

1.6.1 阶段结论与材料入口

阶段一完成了配套环境部署、启动差异定位、共同 flash 配置和三类向量加法用例的差分运行。软件输出、EMU 退出状态、事件 CSV 和原波形为后续分析提供了互相对应的记录；自定义 vdot 的语义、父级控制与性能验证在后续阶段展开。

阶段一证据目录为 team/evidence/stage1-20261009，包含 hello-vadd-summary.log、工具版本、子模块锁、boot-flash 反汇编、vadd-commit-events.csv 及 vector-fix-alu-events.csv。程序源码位于 team/workloads/hello-vadd，本文源目录中的 evidence 保存用于报告展示的对应材料。

阶段一原记录 • 环境记录

1.6.2 源码版本与依赖恢复

本阶段使用的配套源码由以下完整提交号标识。xs-env 提供环境入口，香山及其子模块决定硬件生成输入；NEMU、nexus-am 和 DRAMsim3 分别参与参考执行、程序构建和存储系统仿真。对这些组件同时记录版本，可以避免仅固定顶层仓库、依赖却随下载时间变化的问题。

表 1-6 源码版本与依赖恢复

组件	已验证提交
xs-env	b0b506481e839a5df8a580e76f5aa246dc60fe7c
香山硬件基线	16ae9ddcda54fc9a2fddffad73174cf793ac7814
NEMU	a48a11f208ff81ad765591d83271d1b339fb3861

表 1-6 源码版本与依赖恢复（续）

组件	已验证提交
nexus-am	086a238062fdfb3281b745a537b361983d0154b5
DRAMsim3	1d2a9bf6da8e050d975ef2669c42faa93943f489
DiffTest	62e4540c03c03aa947bdb67f4b6bf66f1a29c5d0
yunsuan	cadd3c2f43096e253d61296b33ac697be8354e29

在团队仓库执行 bash team/scripts/prepare-dependencies.sh。该入口检出固定 xs-env，初始化配套 NEMU、nexus-am、DRAMsim3 及其递归子模块，并锁定 NEMU 的额外依赖。团队仓库自身的香山子模块通过 git clone --recurse-submodules 获取。脚本在发现已有依赖版本不一致或已跟踪文件有修改时停止，保留现场供核对，避免把不同输入混入一次恢复过程。

NEMU 额外依赖的提交记录见锁定脚本。其中 SimPoint 按 NEMU 已记录的子模块版本递归初始化，其余下载仓库也以锁定提交为准。初始化完成后，应核对实际检出的提交，而不是只确认下载命令返回成功；完整源码锁与后续构建日志共同说明产物的来源。

1.6.3 完整工具版本

工具链包括运行构建系统的 JDK、硬件生成与转换工具、宿主编译器、目标程序编译器，以及波形阅读工具。各类工具处理的输入不同，记录版本时也应保留完整输出；例如目标 RISC-V GCC 决定程序指令，而宿主 Clang 或 GCC 处理的是仿真器的 C++ 代码。

表 1-7 完整工具版本

工具	版本
Oracle GraalVM JDK / javac	21.0.12
LLVM / Clang	19.1.7
Mill 项目版本	0.12.3
Scala / Chisel	2.13.15 / 6.6.0，由项目解析
firtool	1.62.1，由本次项目依赖解析
Verilator	5.052，提交 ea338be98e1e838d3518809ce8899f85a009963c
RISC-V GCC	11.4.0，riscv64-linux-gnu- 前缀
GNU binutils	2.38

表 1-7 完整工具版本（续）

工具	版本
宿主 GCC / G++	11.4.0
Make / CMake / Ninja	4.3 / 3.22.1 / 1.10.1
Python / GTKWave	3.10.12 / 3.3.104

完整输出见 tool-versions.txt。Ubuntu 软件包清单用于说明原环境的软件来源，新环境优先安装所需工具并核对关键版本。Mill 读取仓库.mill-version 选择项目版本，启动器的安装来源与项目实际运行版本应分别识别；Scala、Chisel 和 firtool 则按本项目解析结果核对。

1.6.4 跨环境复现条件

环境部署涉及 Windows 上的工作树管理和 WSL 中的硬件构建。选择执行位置时，首先考虑文件所在的文件系统：Git 的索引检查需要访问大量文件，Scala、Chisel 和 C++ 构建也会产生大量中间产物。把工具放在相应文件系统的原生环境执行，可以减少跨环境文件访问，同时使路径、权限和工具行为更容易对应。

表 1-8 跨环境复现条件

工作对象	执行位置	配套记录
Windows NTFS 工作树中的 Git 检查、提交与推送	Windows 原生 Git，使用 Windows 路径	分支、完整提交号、工作区差异及索引状态
Linux 或 WSL 原生目录中的 Git 操作	Linux 原生 Git	实际仓库根目录与提交身份
Chisel/Scala、硬件生成、RISC-V 编译、EMU/NEMU 和波形分析	Linux 或 WSL 原生文件系统	工具版本、构建参数、输入文件及输出目录
Windows 与 Linux 之间的源码、程序和证据交换	固定快照或独立检出，在接收端核验	完整提交号、未提交差异、依赖版本及关键文件 SHA-256

开始构建前，记录仓库根目录、提交号和实际输入；输出单独放置，避免与正在分析的旧记录混合。源码中存在未提交修改时，提交号应与差异快照共同描述输入。跨环境传输后，再核对用于运行的源码和二进制摘要，这样报告中的程序、运行日志和波形才能对应到同一次实验。

这一安排也受到后续工程实践的支持。2026-10-24 的 R4 调用入口发布检查中，一

次由 WSL 访问 Windows 工作树的检查在约 300 秒后超时。核对确认未产生提交、没有残留进程或索引锁后，Windows 原生 Git 完成了剩余检查与提交。过程见原始发布检查和Issue 记录。这是文件系统与工具选择的实际案例；两次执行条件不同，因此记录的是处理过程，没有据此计算 Git 操作的加速比例。

1.6.5 10 月 25 日验证的版本与产物

T04 运行绑定 T17 硬件生成来源 34bae6d4acceac939f958199dae53c14bc1f6183，两次运行共同 BIN 的 SHA-256 为 d81437b44473cef2b0c5777330e13fd076567d4fc1a4296be35d971e51a4dcad。生产 EMU、FST 模型、普通 NEMU、flash 及波形产物各自保留独立摘要。

本报告证据目录 evidence/stage1-current-20261025/保存以下七份原件，其固定公开版本为 2690c737：

- production-run/emulator-output.log：英文 hello、身份状态、用例结果和整程序统计。
- production-run/run.json：实际命令、输入前后摘要、日志摘要与退出状态。
- wave/wave.json：采样定义、信号清单、事件及选通信号产物摘要。
- acceptance.json：软件输入等价、原输入核验、最终波形与工具检查汇总。
- RESULT.md：两次运行、FST 构建与完整产物的复现索引。
- trace-run/emulator-output.log：独立 FST 模型的原始串口与终态。
- trace-run/run.json：FST 采集命令、运行输入与退出状态。

1.6.6 技术资料索引

环境部署详细方法。

1.7 原始操作日志

1.7.1 10 月 25 日生产模型原始输出

以下列出该次构建或运行的记录。

```
emu compiled at Oct 24 2026, 09:22:45
Using simulated 32768B flash
use /home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin as flash bin
Using simulated 128MB RAM
The image is
/home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/hello-vadd-riscv64-xs.bin
```

```
The reference model is
/home/lizufan/riscv-lab/t17-clean-20261024-0854/support-01/nemu/reference/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image:
/home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin

[src/monitor/image_loader.c:250,load_img] Read 12 bytes from file
/home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin to 0x0xf0000000
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
Core 0: HIT GOOD TRAP at pc = 0x800002a4
Core-0 instrCnt = 16621, cycleCnt = 43249, IPC = 0.384309
Seed=0 Guest cycle spent: 43253 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 56742ms
```

1.7.2 10 月 25 日 FST 模型原始输出

以下列出该次构建或运行的记录。

```
emu compiled at Oct 25 2026, 01:16:11
Using simulated 32768B flash
use /home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin as flash bin
Using simulated 128MB RAM
The image is
/home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/hello-vadd-riscv64-xs.bin
The reference model is
/home/lizufan/riscv-lab/t17-clean-20261024-0854/support-01/nemu/reference/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image:
/home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin

[src/monitor/image_loader.c:250,load_img] Read 12 bytes from file
/home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin to 0x0xf0000000
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
Core 0: HIT GOOD TRAP at pc = 0x800002a4
Core-0 instrCnt = 16621, cycleCnt = 43249, IPC = 0.384309
Seed=0 Guest cycle spent: 43253 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 145975ms
```

1.7.3 程序构建原始日志

以下列出该次构建或运行的记录。

```
+ /home/riscv/riscv-lab/stage1-build-workload.sh:11: date -Is
```

```
2026-10-09T20:55:26+08:00
+ /home/riscv/riscv-lab/stage1-build-workload.sh:12: cd /home/riscv/riscv-lab/workloads/hello-vadd
+ /home/riscv/riscv-lab/stage1-build-workload.sh:13: python3 -
+ /home/riscv/riscv-lab/stage1-build-workload.sh:20: make ARCH=riscv64-xs MARCH=rv64gcv -j1
# Building hello-vadd [riscv64-xs] with AM_HOME {/home/riscv/riscv-lab/xs-env/nexus-am}
+ CC main.c
# Building lib-am [riscv64-xs]
+ AS src/nemu/common/mainargs.S
+ AR -> build/am-riscv64-xs.a
# Building lib-klib [riscv64-xs]
# Creating binary image [riscv64-xs]
+ LD -> build/hello-vadd-riscv64-xs.elf
+ OBJCOPY -> build/hello-vadd-riscv64-xs.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:21: riscv64-linux-gnu-gcc -c -march=rv64gc -mabi=lp64d
boot-flash.S -o build/boot-flash.o
+ /home/riscv/riscv-lab/stage1-build-workload.sh:22: riscv64-linux-gnu-objcopy -O binary -j .text
build/boot-flash.o build/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:23: riscv64-linux-gnu-objdump -d build/boot-flash.o
+ /home/riscv/riscv-lab/stage1-build-workload.sh:24: cp build/boot-flash.bin
/home/riscv/riscv-lab/stage1-evidence/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:25: sha256sum build/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:26: cp build/hello-vadd-riscv64-xs.txt
/home/riscv/riscv-lab/stage1-evidence/hello-vadd-disassembly.txt
+ /home/riscv/riscv-lab/stage1-build-workload.sh:27: sha256sum build/hello-vadd-riscv64-xs.elf
build/hello-vadd-riscv64-xs.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:28: echo WORKLOAD_BUILD_PASS
WORKLOAD_BUILD_PASS
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: rc=0
++ /home/riscv/riscv-lab/stage1-build-workload.sh:1: date -Is
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: printf 'exit_code=%s end=%s\n' 0
2026-10-09T20:55:26+08:00
```

1.7.4 历史 hello 与 vadd 运行摘要

以下列出该次构建或运行的记录。

```
The image is /home/riscv/riscv-lab/workloads/hello-vadd/build/hello-vadd-riscv64-xs.bin
The reference model is /home/riscv/riscv-lab/xs-env/NEMU/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image:
/home/riscv/riscv-lab/workloads/hello-vadd/build/boot-flash.bin
hello xiangshan, 我是 riscv-vdot-team, IPaddress=10.198.93.97
identity_status=placeholder; IPaddress is WSL host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
[PERF ][time=              42965] SimTop.l_soc.l3cacheOpt.slices_0.mshrAlloc: mshr_lateCore 0: HIT GOOD
TRAP at pc = 0x8000029c
Core-0 instrCnt = 16557, cycleCnt = 42962, IPC = 0.385387
Host time spent: 300823ms
      Exit status: 0
+ /home/riscv/riscv-lab/stage1-run-hello-vadd.sh:21: echo HELLO_VADD_EMU_RUN_PASS
HELLO_VADD_EMU_RUN_PASS
```

1.7.5 NEMU 构建原始日志

以下列出该次构建或运行的记录。

```
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:9: date -Is
2026-10-09T20:24:27+08:00
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:10: cd /home/riscv/riscv-lab/xs-env/NEMU
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:11: git rev-parse HEAD
a48a11f208ff81ad765591d83271d1b339fb3861
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:12: make riscv64-xs-ref_defconfig
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/nanopb'...
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/LibCheckpoint'...
Submodule 'resource/nanopb' (https://github.com/nanopb/nanopb) registered for path 'resource/nanopb'
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/LibCheckpoint/resource/nanopb'...
Submodule path 'resource/nanopb': checked out '671672b4d7994a9b07a307ae654885c7202ae886'
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/gcpt_restore'...
/home/riscv/riscv-lab/xs-env/NEMU/scripts/config.mk:20: Warning: .config does not exists!
/home/riscv/riscv-lab/xs-env/NEMU/scripts/config.mk:21: To build the project, first run 'make menuconfig'.
+ CC confdata.c
+ CC expr.c
+ CC preprocess.c
+ CC symbol.c
+ CC util.c
+ YACC build/parser.tab.h
+ LEX build/lexer.lex.c
+ CC build/lexer.lex.c
+ CC build/parser.tab.c
+ CC conf.c
+ g++ /home/riscv/riscv-lab/xs-env/NEMU/tools/kconfig/build/conf
+ CC fixdep.c
+ g++ /home/riscv/riscv-lab/xs-env/NEMU/tools/fixdep/build/fixdep
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:13: /usr/bin/time -v make -j1
Makefile:40: You have set CONFIG_FLASH_IMG_PATH option, but the specified path '"" does not exist!
Cloning into 'resource/softfloat/repo'...
+ CC src/nemu-main.c
+ CC src/device/device.c
+ CC src/device/alarm.c
+ CC src/device/intr.c
+ CC src/device/flash.c
+ CC src/cpu/tcache.c
+ CC src/cpu/cpu-exec.c
+ CC src/cpu/instr_stat.c
+ CC src/cpu/difftest/ame/amuctrl.c
+ CC src/cpu/difftest/ame/msync.c
+ CC src/cpu/difftest/dut.c
+ CC src/cpu/difftest/ref.c
+ CC src/monitor/ui.c
+ CC src/monitor/watchpoint.c
+ CC src/monitor/monitor.c
+ CC src/monitor/fdt_rng_seed.c
+ CC src/monitor/aligncheck.c
+ CC src/monitor/image_loader.c
+ CC src/utils/state.c
+ CC src/utils/iqueue.c
+ CC src/utils/br_log.c
+ CC src/utils/expr.c
+ CC src/utils/log.c
+ CC src/utils/filename.c
+ CC src/utils/dynamic-config.c
```

+ CC src/utils/timer.c
+ CC src/memory/host-tlb.c
+ CC src/memory/vaddr.c
+ CC src/memory/paddr.c
+ CC src/isa/riscv64/system/mmu.c
+ CC src/isa/riscv64/system/priv.c
+ CC src/isa/riscv64/system/trigger.c
+ CC src/isa/riscv64/system/intr.c
+ CC src/isa/riscv64/system/timer.c
+ CC src/isa/riscv64/system/aia.c
+ CC src/isa/riscv64/reg.c
+ CC src/isa/riscv64/ame/raw_fp4_memory.c
+ CC src/isa/riscv64/clint.c
+ CC src/isa/riscv64/difftest/dut.c
+ CC src/isa/riscv64/difftest/ref.c
+ CC src/isa/riscv64/init.c
+ CC src/isa/riscv64/logo.c
+ CC src/isa/riscv64/instr/priv/system.c
+ CC src/isa/riscv64/instr/fp.c
+ CC src/isa/riscv64/instr/rvk/aes_common.c
+ CC src/isa/riscv64/instr/rvk/crypto_impl.c
+ CC src/isa/riscv64/instr/ame/mcompute_impl.c
+ CC src/isa/riscv64/instr/ame/mldst_transpose_fast.c
+ CC src/isa/riscv64/instr/ame/mreg.c
+ CC src/isa/riscv64/instr/ame/mldst_impl.c
+ CC src/isa/riscv64/instr/ame/mldst_fast.c
+ CC src/isa/riscv64/instr/decode.c
+ CC src/isa/riscv64/instr/rva/amo.c
+ CC src/isa/riscv64/instr/rvv/vldst_impl.c
+ CC src/isa/riscv64/instr/rvv/vfexp2.c
+ CC src/isa/riscv64/instr/rvv/vcompute_impl.c
+ CC src/isa/riscv64/instr/rvv/vreg_impl.c
+ CC src/isa/riscv64/instr/rvv/vcommon.c
+ CC src/isa/riscv64/instr/rvcbo/cbo_impl.c
+ CC src/isa/riscv64/instr/mp.c
+ CC src/isa/riscv64/instr/rvb/rvintrin.c
+ CC src/isa/riscv64/instr/rvt/sfu.c
+ CC src/isa/riscv64/instr/rvt/matmul_subfloat.c
+ CC src/isa/riscv64/instr/rvt/element_wise.c
+ CC src/isa/riscv64/instr/rvt/convert.c
+ CC src/isa/riscv64/instr/rvt/conv2d.c
+ CC src/isa/riscv64/instr/rvt/type.c
+ CC src/isa/riscv64/instr/rvt/tensor_init.c
+ CC src/isa/riscv64/instr/rvt/tensor_copy.c
+ CC src/isa/riscv64/instr/rvt/pooling.c
+ CC src/isa/riscv64/instr/rvt/register.c
+ CC src/isa/riscv64/instr/rvt/matmul.c
+ CC src/isa/riscv64/instr/vp.c
+ CC src/engine/interpreter/hostcall.c
+ CC src/engine/interpreter/fp.c
+ CC src/engine/interpreter/init.c
+ CC src/device/io/mmio.c
+ CC src/device/io/port-io.c
+ CC src/device/io/map.c
+ CC src/profiling/profiling_control.c
+ CC src/checkpoint/cpt_env.c
+ CXX src/checkpoint/simpoint.cpp
+ CXX src/checkpoint/path_manager.cpp
+ CXX src/checkpoint/serializer.cpp

```
+ CXX src/checkpoint/semantic_point.cpp
+ CXX src/base/output.cpp
+ CXX src/iostream3/zfstream.cpp
+ CXX src/cpu/difftest/ame/msync_queue_wrapper.cpp
+ CXX src/cpu/difftest/ame/amu_ctrl_queue_wrapper.cpp
+ CXX src/memory/store_queue_wrapper.cpp
+ CXX src/memory/elfloader.cpp
SPECIALIZE_TYPE=RISCV SOFTFLOAT_OPTS="-DSOFTFLOAT_ROUND_ODD -DINLINE_LEVEL=5
-DSOFTFLOAT_FAST_DIV32TO16 -DSOFTFLOAT_FAST_DIV64TO32 -fPIC" make -s -C
/home/riscv/riscv-lab/xs-env/NEMU/resource/softfloat/repo/build/Linux-x86_64-GCC all
mkdir -p resource/softfloat/build
ln -sf /home/riscv/riscv-lab/xs-env/NEMU/resource/softfloat/repo/build/Linux-x86_64-GCC/softfloat.a
resource/softfloat/build/softfloat.a
+ g++ /home/riscv/riscv-lab/xs-env/NEMU/build/riscv64-nemu-interpreter-so
        Command being timed: "make -j1"
        User time (seconds): 22.97
        System time (seconds): 7.18
        Percent of CPU this job got: 101%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:29.76
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 445324
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 1324
        Minor (reclaiming a frame) page faults: 1488436
        Voluntary context switches: 7740
        Involuntary context switches: 491
        Swaps: 0
        File system inputs: 227800
        File system outputs: 96112
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:14: cp .config /home/riscv/riscv-lab/stage1-evidence/nemu.config
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:15: file build/riscv64-nemu-interpreter-so
build/riscv64-nemu-interpreter-so: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically
linked, BuildID[sha1]=4841e67452e47681eecaff72b0e3526c02cec25f, not stripped
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:16: sha256sum build/riscv64-nemu-interpreter-so
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:17: date -Is
2026-10-09T20:25:12+08:00
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:18: echo NEMU_BUILD_PASS
NEMU_BUILD_PASS
```

1.7.6 EMU 构建原始日志

以下列出该次构建或运行的记录。

```
+ /home/riscv/riscv-lab/stage1-build-emu.sh:12: date -Is
2026-10-09T20:42:28+08:00
+ /home/riscv/riscv-lab/stage1-build-emu.sh:13: cd /home/riscv/riscv-lab/xs-env/XiangShan
+ /home/riscv/riscv-lab/stage1-build-emu.sh:14: git rev-parse HEAD
16ae9ddcda54fc9a2fddffad73174cf793ac7814
+ /home/riscv/riscv-lab/stage1-build-emu.sh:15: free -h
```

```
               total        used        free      shared  buff/cache   available
Mem:            19Gi       623Mi        16Gi       2.0Mi       2.3Gi        18Gi
Swap:           16Gi        73Mi        15Gi
+ /home/riscv/riscv-lab/stage1-build-emu.sh:16: /usr/bin/time -v make emu CONFIG=MinimalConfig
NUM_CORES=1 EMU_THREADS=1 EMU_TRACE=fst JVM_XMX=12G -j4
make -C ./difftest emu SIM_TOP=SimTop DESIGN_DIR=/home/riscv/riscv-lab/xs-env/XiangShan
NUM_CORES=1 RTL_SUFFIX=sv
make[1]: Entering directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
Fri, 09 Oct 2026 20:42:28 +0800
Building emu...
make[2]: Entering directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
time -avp -o /home/riscv/riscv-lab/xs-env/XiangShan/build/time.log make -s VM_PARALLEL_BUILDS=1
OPT_SLOW="-O0" \
                              OPT_FAST=-O3 \
                              PGO_CFLAGS= \
                              PGO_LDFLAGS= \
                              -C /home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile -f
                              /home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile/VSimTop.mk
make[3]: Entering directory '/home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile'
make[3]: Leaving directory '/home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile'
make[2]: Leaving directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
make[1]: Leaving directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
        Command being timed: "make emu CONFIG=MinimalConfig NUM_CORES=1 EMU_THREADS=1
        EMU_TRACE=fst JVM_XMX=12G -j4"
        User time (seconds): 2451.21
        System time (seconds): 187.35
        Percent of CPU this job got: 397%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 11:03.88
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 1301648
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 479
        Minor (reclaiming a frame) page faults: 50883606
        Voluntary context switches: 5000
        Involuntary context switches: 4568
        Swaps: 0
        File system inputs: 8832
        File system outputs: 3129248
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
+ /home/riscv/riscv-lab/stage1-build-emu.sh:17: ./build/emu --help
+ /home/riscv/riscv-lab/stage1-build-emu.sh:18: sha256sum build/emu
+ /home/riscv/riscv-lab/stage1-build-emu.sh:19: cp build/time.log
/home/riscv/riscv-lab/stage1-evidence/emu-time.log
+ /home/riscv/riscv-lab/stage1-build-emu.sh:20: date -Is
2026-10-09T20:53:32+08:00
+ /home/riscv/riscv-lab/stage1-build-emu.sh:21: echo XIANGSHAN_EMU_BUILD_PASS
XIANGSHAN_EMU_BUILD_PASS
+ /home/riscv/riscv-lab/stage1-build-emu.sh:1: rc=0
++ /home/riscv/riscv-lab/stage1-build-emu.sh:1: date -Is
+ /home/riscv/riscv-lab/stage1-build-emu.sh:1: printf 'exit_code=%s end=%s\n' 0 2026-10-09T20:53:32+08:00
```

1.7.7 工具与编译链版本原始输出

以下列出该次构建或运行的记录。

```
java 21.0.12 2026-07-21 LTS
Java(TM) SE Runtime Environment Oracle GraalVM 21.0.12+7.1 (build 21.0.12+7-LTS-jvmci-23.1-b96)
Java HotSpot(TM) 64-Bit Server VM Oracle GraalVM 21.0.12+7.1 (build 21.0.12+7-LTS-jvmci-23.1-b96,
mixed mode, sharing)
javac 21.0.12
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

g++ (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Ubuntu clang version 19.1.7 (++20250114103320+cd708029e0b2-1~exp1~20250114103432.75)
Target: x86_64-pc-linux-gnu
Thread model: posix
InstalledDir: /usr/lib/llvm-19/bin
Verilator 5.052 2026-09-05 rev v5.052
LLVM (http://llvm.org/):
  LLVM version 18.0.0git
  Optimized build.
CIRCT firtool-1.62.1
riscv64-linux-gnu-gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

GNU assembler (GNU Binutils for Ubuntu) 2.38
Copyright (C) 2022 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or later.
This program has absolutely no warranty.
This assembler was configured for a target of `riscv64-linux-gnu'.
Mill Build Tool version 0.12.3
Java version: 21.0.12, vendor: Oracle Corporation, runtime: /opt/graalvm-jdk-21
Default locale: en, platform encoding: UTF-8
OS name: "Linux", version: 6.6.87.2-microsoft-standard-WSL2, arch: amd64

[Scala/Chisel dependency versions]
val defaultScalaVersion = "2.13.15"
  "chisel"        -> ivy"org.chipsalliance::chisel:6.6.0",
  "chisel-plugin" -> ivy"org.chipsalliance:::chisel-plugin:6.6.0",
  "chiseltest"    -> ivy"edu.berkeley.cs::chiseltest:6.0.0"
  override def scalaVersion = defaultScalaVersion
  def scalaVersion: T[String] = T(defaultScalaVersion)
    def scalaVersion: T[String] = T(defaultScalaVersion)
    def scalaReflectIvy = ivy"org.scala-lang:scala-reflect:${defaultScalaVersion}"
    def scalaVersion: T[String] = T(defaultScalaVersion)
    def scalaVersion: T[String] = T(defaultScalaVersion)
  override def scalaVersion: T[String] = T(defaultScalaVersion)
  override def ivyDeps = super.ivyDeps() ++ Agg(ivy"org.scala-lang:scala-reflect:${defaultScalaVersion}")
  def scalaReflectIvy = ivy"org.scala-lang:scala-reflect:${defaultScalaVersion}"
```

1.7.8 固定子模块版本清单

以下列出该次构建或运行的记录。

1d2a9bf6da8e050d975ef2669c42faa93943f489 DRAMsim3 (heads/cosim-kmh)
68739c5a23cc48c9a13d28793834e4fee95d33ee DRAMsim3/ext/SuperLU_MT_3.1 (68739c5)
a48a11f208ff81ad765591d83271d1b339fb3861 NEMU (a48a11f)
e51a936d7fddfa03c81692039f184ab6c437e99e NEMU/resource/simpoint/simpoint_repo (heads/main)
bba8f3a4e3a15ef135baed37566be3e8a0c7d384 NutShell (bba8f3a)
29e80f8525735d1cb77f30fd2a3ea5eff3e6528c NutShell/difftest (29e80f8)
16ae9ddcda54fc9a2fddffad73174cf793ac7814 XiangShan (16ae9dd)
82079f4081bbc1f0221c790fbf8a29b1f23f48b8 XiangShan/coupledL2 (82079f4)
8faa17c9f7115782da1bd1665bd95587fba525ec XiangShan/coupledL2/HuanCun (8faa17c)
de50694f6ffb48780a4b4665907442b3e4dd0d6d XiangShan/coupledL2/HuanCun/Utility (de50694)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/coupledL2/HuanCun/Utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/HuanCun/Utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/HuanCun/Utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/HuanCun/Utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/HuanCun/Utility/rocket-chip/torture/env (ce70afb)
90ae9a9314d444ade097fd5a471916869559b4fd XiangShan/coupledL2/HuanCun/rocket-chip (90ae9a9)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/HuanCun/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/HuanCun/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/HuanCun/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/HuanCun/rocket-chip/torture/env (ce70afb)
90ae9a9314d444ade097fd5a471916869559b4fd XiangShan/coupledL2/rocket-chip (90ae9a9)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/coupledL2/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/coupledL2/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/rocket-chip/torture/env (ce70afb)
992ce71166405de47c93102100a897726bd93bac XiangShan/coupledL2/utility (992ce71)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/coupledL2/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/coupledL2/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/coupledL2/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/utility/rocket-chip/torture/env (ce70afb)
62e4540c03c03aa947bdb67f4b6bf66f1a29c5d0 XiangShan/difftest (62e4540)
e1bd4695ca7beb36a5ce7357e9527ad9e95b9ec1 XiangShan/fudian (heads/pipeline)

b64af41c3276f97f0e181920400ee056b9c88037 XiangShan/fudian/berkeley-softfloat-3 (b64af41)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/fudian/berkeley-testfloat-3 (06b2007)
90aaf5935206ff322e461c3d021436c20dd0ac85 XiangShan/huancun (90aaf59)
de50694f6ffb48780a4b4665907442b3e4dd0d6d XiangShan/huancun/Utility (de50694)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/huancun/Utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/huancun/Utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/huancun/Utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/huancun/Utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/huancun/Utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/huancun/Utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/huancun/Utility/rocket-chip/torture/env (ce70afb)
e64b744b1490992bd038aaf9ca628f0e29fa0e59 XiangShan/huancun/rocket-chip (e64b744)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/huancun/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/huancun/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/huancun/rocket-chip/hardfloat/berkeley-softfloat-3
(5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/huancun/rocket-chip/hardfloat/berkeley-testfloat-3
(06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/huancun/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/huancun/rocket-chip/torture/env (ce70afb)
614ceb4c5d39d32e6ae786693a35da429ea32c12 XiangShan/openLLC (614ceb4)
d3b04fef92092f9b27023bea57cc2ca99a8ffea0 XiangShan/openLLC/coupledL2 (d3b04fe)
90aaf5935206ff322e461c3d021436c20dd0ac85 XiangShan/openLLC/coupledL2/HuanCun (90aaf59)
de50694f6ffb48780a4b4665907442b3e4dd0d6d XiangShan/openLLC/coupledL2/HuanCun/Utility (de50694)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip
(175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28
XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f
XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5
XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d
XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/torture/env (ce70afb)
e64b744b1490992bd038aaf9ca628f0e29fa0e59 XiangShan/openLLC/coupledL2/HuanCun/rocket-chip
(e64b744)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/cde
(52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f
XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/torture
(99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d
XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/torture/env (ce70afb)
bb4baf85c5bd4b55ffdcda12a75648fef212ab69 XiangShan/openLLC/coupledL2/rocket-chip (bb4baf8)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/coupledL2/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/coupledL2/rocket-chip/hardfloat
(d93aa57)

5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/coupledL2/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/coupledL2/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/coupledL2/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/coupledL2/rocket-chip/torture/env
(ce70afb)
342e0ad98bf24bf550f4d44099bd64875145d07c XiangShan/openLLC/coupledL2/utility (342e0ad)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/openLLC/coupledL2/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/coupledL2/utility/rocket-chip/cde
(52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/coupledL2/utility/rocket-chip/hardfloat
(d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/coupledL2/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/coupledL2/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/coupledL2/utility/rocket-chip/torture
(99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/coupledL2/utility/rocket-chip/torture/env
(ce70afb)
546a621ed82302db1bd1edc900b17ba55682c15c XiangShan/openLLC/openNCB (546a621)
c0955429aa3d9fe2be560f28599ece700fb0466c XiangShan/openLLC/openNCB/external/rocket-chip (c095542)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/openNCB/external/rocket-chip/cde
(52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/openNCB/external/rocket-chip/hardfloat
(d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/openNCB/external/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/openNCB/external/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/openNCB/external/rocket-chip/torture
(99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/openNCB/external/rocket-chip/torture/env
(ce70afb)
bb4baf85c5bd4b55ffdcda12a75648fef212ab69 XiangShan/openLLC/rocket-chip (bb4baf8)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/openLLC/rocket-chip/hardfloat/berkeley-softfloat-3
(5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/rocket-chip/torture/env (ce70afb)
342e0ad98bf24bf550f4d44099bd64875145d07c XiangShan/openLLC/utility (342e0ad)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/openLLC/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/utility/rocket-chip/torture/env (ce70afb)
ed8c30e1ec8a9c9aa6b116ce54281235d00273fa XiangShan/ready-to-run (ed8c30e)
bea0af5d855a871699b907c1988961141bbf28e7 XiangShan/rocket-chip (bea0af5)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)

06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/rocket-chip/torture/env (ce70afb)
9464aee3c77b021a037f28aa1b3fe53c71b516f6 XiangShan/src/main/resources/aia (heads/main)
a8ca0c6564865ee69b564d431ecbc52074ba8060 XiangShan/utility (a8ca0c6)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/utility/rocket-chip/torture/env (ce70afb)
cadd3c2f43096e253d61296b33ac697be8354e29 XiangShan/yunsuan (cadd3c2)
086a238062fdfb3281b745a537b361983d0154b5 nexus-am (086a238)

1.7.9 原始波形事件记录

以下列出该次构建或运行的记录。

cycle,scope,pc,instruction,vecwen,wdest
19562,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000012c,0xcc0472d7,0,5
19624,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000130,0x2050087,1,1
19624,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000134,0x2058107,1,2
19631,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000138,0x21101d7,1,3
19646,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000013c,0x20601a7,1,32
19646,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000140,0x28513,0,10
19647,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000142,0x8067,0,0
25442,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000012c,0xcc0472d7,0,5
25458,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000130,0x2050087,1,1
25459,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000134,0x2058107,1,2
25466,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000138,0x21101d7,1,3
25481,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000013c,0x20601a7,1,32
25481,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000140,0x28513,0,10
25481,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_2.dpic,0x80000142,0x8067,0,0
32991,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000012c,0xcc0472d7,0,5
33007,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000130,0x2050087,1,1
33008,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000134,0x2058107,1,2

33015,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000138,0x
21101d7,1,3
33030,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000013c,0x
20601a7,1,32
33030,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000140,
0x28513,0,10
33030,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_2.dpic,0x80000142,
0x8067,0,0

1.7.10 向量运算单元事件记录

以下列出该次构建或运行的记录。

cycle,scope,phase,fuOpType,vsew,src0,src1,result
19625,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,in,64,0,0xffffffffffff
fff0102030405060708,0xffffffffffffffff0807060504030201,
19626,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,out,,,,,0xffffffffffffff
f0909090909090909
25460,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,in,64,0,0xffffffffffff
ffff8f9fafbfcfdfeff,0xffffffffffffffff8f9fafbfcfdfeff,
25461,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,out,,,,,0xffffffffffffff
ff0f2f4f6f8fafcfe
33009,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,in,64,0,0xffffffffffff
fffff7f80ff0001ff01,0xffffffffffffffff7f800100ff807f,
33010,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,out,,,,,0xffffffffffffff
ffefe000000007f80

第二阶段：向量点积设计与功能验证

工程组合：0f ＋ firstdot V2 ＋原 C8　材料日期：2026 年 11 月 2 日

本设计在香山昆明湖 V2 现有向量执行单元中接入自定义点积，采用乘积槽 P 和结果槽 R 保存两笔在途请求。下文依次说明指令语义、RTL 实现、正数、负数与边界测试，以及取消和共享资源控制；完成取消关钟修复的最终工程版本记为 0f。

2.1　指令语义与微架构设计

点积将多组乘法结果归约为一个和。本设计在昆明湖 V2 已有向量执行路径中接入 vdot，复用译码、向量寄存器、调度、写回和精确提交机制。数据通路完成 8 项有符号乘加，控制通路维护请求身份、反压及取消。优化围绕独立点积吞吐、结果寄存器开销和与 DIV 共享父级资源的切换行为展开。

2.1.1　点积定义、位宽与结果布局

从指令执行前的源寄存器状态读取：

```
byte_a[i] = vs1_before[8*i+7 : 8*i]
byte_b[i] = vs2_before[8*i+7 : 8*i]      i = 0…7
signed8(x) = x            if x < 128
           x - 256      otherwise
```

源 [127:64] 不参与运算。小端裸机加载将数组第 0 个字节映射到源 [7:0]，后续字节依次对应更高的 8 位字段。向量十六进制打印通常从高位开始，检查时按位段还原 lane 编号，与数组下标逐项对应。

单项乘积范围为 [-16256, 16384]，因此八项精确和为：

```
S_min = 8 × (-128) × 127    = -130048
S_max = 8 × (-128) × (-128) =  131072
```

有符号 18 位的正上限为 131071，八项精确和的正端点达到 131072，因此归约至少使用 19 位有符号表示。架构结果使用 32 位补码，覆盖全部合法输入的精确范围。正端点由八项最大正乘积共同形成，是归约末级位宽的确定性边界。

算术实现由 8 个 signed 8×8 乘积和归约树组成。乘积使用 16 位，二项、四项、八项归并分别保留 17、18、19 位，再扩展为 32 位结果。各级先按符号扩展操作数再相加，保留对应进位。采用 32 位中间和也满足同一数学定义；具体结构改变时，完整结果比较继续使用精确整数参考值。

```
127                          32 31                 0
+-------------------------------------+-------------------------+
|          全部为 0                   |    S 的 32 位补码       |
+-------------------------------------+-------------------------+
```

例如 S=-8 时，完整目的值为 0x00000000_00000000_00000000_FFFFFFF8。补码位于低 32 位结果字段，高 96 位为零。该字段按完整寄存器写回约定解释，结果读回以全部 16 字节为检查对象。

vl=8 限定本指令的合法输入配置，输出则完整覆盖 128 位目的寄存器。通用 tail/mask 合并路径在接入时按这一规则配置或绕开，确保目的高 96 位取零，旧目的值不参与结果。四种 vta/vma 组合使用相同的输出定义。

2.1.2 指令编码与合法配置

本项目选择 32 位 custom-0 主 opcode 0x0b，funct 字段由内部 v0.1 合同定义。标准 opcode 表提供自定义扩展空间，编码分配在团队使用范围内固定。

表 2-1 指令编码与合法配置

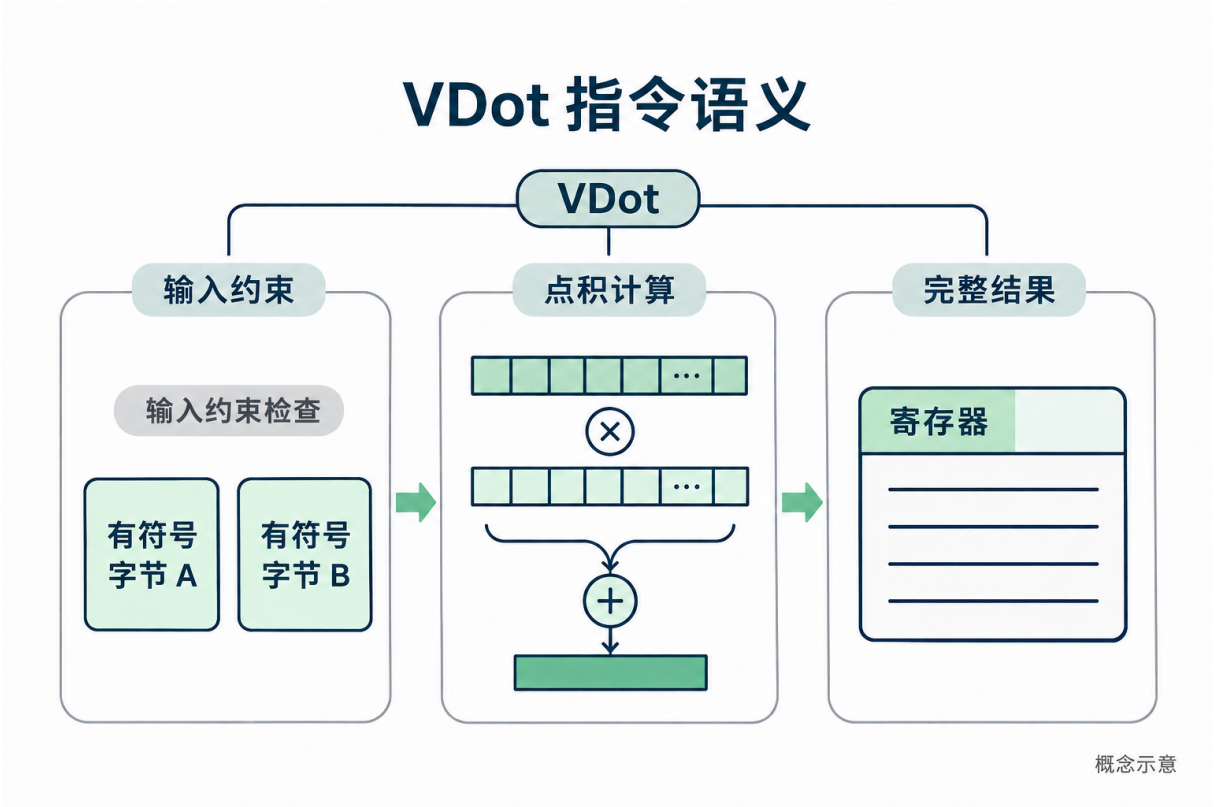
位段	字段	v0.1 取值
[31:26]	funct6	000000
[25]	vm	1；本版只支持无掩码
[24:20]	vs2	第二个编码源寄存器编号，0～31
[19:15]	vs1	第一个编码源寄存器编号，0～31
[14:12]	funct3	000
[11:7]	vd	目的向量寄存器编号，0～31
[6:0]	opcode	0001011（0x0b）

汇编记法沿用向量常用顺序 vdot.vv vd, vs2, vs1。数学公式中 vs1/vs2 的名称与位段一致；点积交换律会掩盖源颠倒，所以编码测试还必须检查字段提取本身。

```
MATCH_VDOT = 0x0200000b
MASK_VDOT  = 0xfe00707f
识别条件： (instruction & MASK_VDOT) == MATCH_VDOT
编码公式： MATCH_VDOT | (vs2 << 20) | (vs1 << 15) | (vd << 7)
BitPat：  b0000001??????????000?????0001011
```

vdot.vv v10, v8, v9 编码为 0x0284850b，小端字节序列为 0b 85 84 02。三个寄存器编号均指向量寄存器。编码测试按位段提取 vs2、vs1 和 vd，并核对译码产生的向量源、目的类型。

vm=0 或固定 funct 字段不符时不匹配本版 vdot。精确 mask/match 只接受已定义模式，其余编码由其他已分配扩展或默认非法指令路径处理，邻近模式分别进入译码测试。



概念示意：输入约束、点积与结果写回。技术依据见图 2-1。

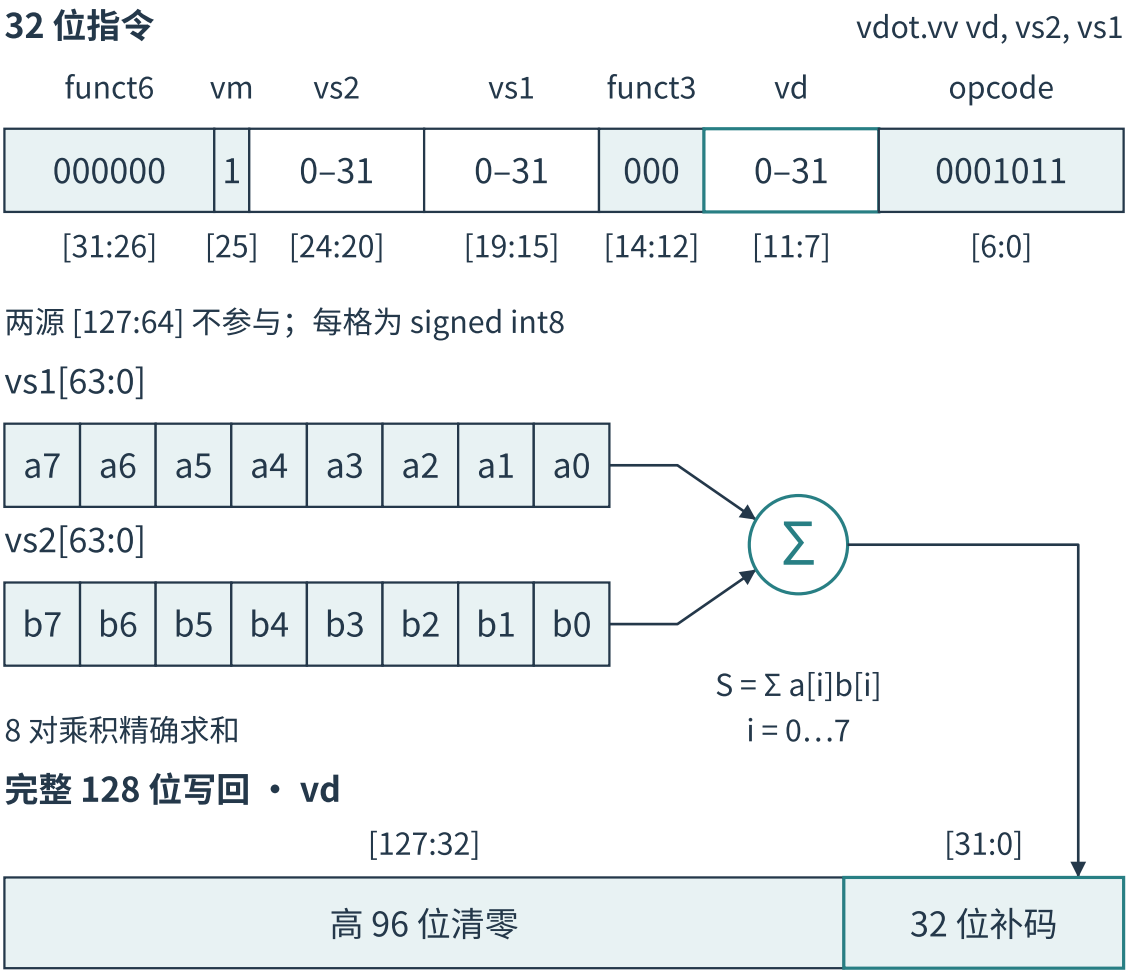


图 2-1　vdot 指令字段与完整结果布局

低 8 个有符号字节逐项相乘并求和；结果低 32 位保存补码，高 96 位清零。内部合同 vdot-s8x8-r32-v0.1；Q01–Q07 待官方澄清；先读源、后覆盖 vd（允许别名）。

表 2-2　指令编码与合法配置

条件	指令行为与测试条件
VLEN	固定为 128；RTL 构建与 NEMU 配置使用相同常量，其他 VLEN 采用独立版本约定
向量状态	运行环境支持并启用 V，mstatus.VS 不为 Off；沿用既有特权态检查
vtype	vill=0、vsew=000（e8）、vlmul=000（m1）；域外配置进入非法指令路径
vl	等于 8；vl=0、1、7、9、16 等值属于域外配置，其中 vl=0 按非法配置处理
vstart	等于 0；非零值进入非法指令路径

表 2-2 指令编码与合法配置（续）

条件	指令行为与测试条件
vm	等于 1；vm=0 相邻编码由默认译码及其他已分配扩展规则处理
vta/vma	tu/mu、tu/ma、ta/mu、ta/ma 均合法；输出仍精确遵循“点积定义、位宽与结果布局”小节

基础测试使用 vsetivli zero, 8, e8, m1, tu, mu 建立配置，读取实际 vl/vtype 并核对 vstart=0，再执行已匹配的 vdot 机器字。配置准备与自定义指令译码是两个步骤，测试分别保留其状态与指令身份。

v0.1 内部合同采用固定的 custom 指令合法域，并定义完整 128 位结果写回。标准向量状态、策略位和重启机制用于现有体系接入；vdot 的低 8 项读取、域外异常和目的格式由本合同明确规定。相关背景见RISC-V Vector 1.0 文档。

vd/vs1/vs2 均为 v0…v31，每个操作数只占一个寄存器，无偶数寄存器对齐要求。v0 在本版 vm=1 下可作为普通源或目的，不是零寄存器。

源值取自指令执行前的寄存器快照，再写回目的。NEMU 先缓存所需源数据，RTL 通过重命名维护源版本与目的映射；vd 与任一源相同、两个源相同以及三者相同都遵循这个顺序。旧 vd 不参与数学累加，通用旧目的端口也不改变该结果。

成功执行时：

- 只更新一个向量目的寄存器的完整 128 位；其余向量寄存器、标量寄存器、浮点寄存器、内存不变。
- vl、vtype、vlenb、vxrm、vxsat、vcsr 保留原值；合法初态下 vstart 仍为 0，原有 vxsat=1 同样保留。
- 沿用现有向量状态置脏机制；不产生浮点异常标志，不访问数据存储器。
- 正常进入下一条指令（本编码长度 4 字节），退休与计数行为沿用处理器正常指令机制。

编码匹配而配置不合法时，合同规定精确非法指令异常，异常码为 2，陷阱 PC 指向该指令。目的寄存器、运算相关向量 CSR 和内存保持指令执行前状态。陷阱入口对特权 CSR 的正常更新、tval 填写和异常优先级沿用现有机制。测试优先构造单一可达非法条件；vill 与 vl 等联动条件按“数据布局、别名与异常测试”小节成组记录。

VLEN 一致性在构建或加载配置时检查。跨 VLEN 扩展需要独立规定读取范围、结果布局与异常条件，并以新的合同版本组织实现和测试。

redirect/flush 按 ROB 年龄取消乱序请求，被取消事务不产生已提交的目的更新。指令采用完整操作的精确提交，沿用现有中断与取消机制，不提供中间 lane 的架构恢复状态。全核验证以异常、取消和正常恢复用例核对这些行为。

2.1.3 架构参考伪代码

以下伪代码定义架构效果。它使用精确整数求和、先读源后写目的，并把合法性检查与数据运算分开；实际握手和时钟行为见“逐拍状态转移与握手”和“时钟门控、唤醒与写回”。

```
def execute_vdot(inst, state):
  # 必须先完成编码、向量状态和配置检查；失败则走非法指令路径。
  require_vdot_encoding_and_supported_state(inst, state)

  src1 = read_full_vector_before_write(inst.vs1)  # 128-bit snapshot
  src2 = read_full_vector_before_write(inst.vs2)  # 128-bit snapshot
  total = 0                              # 数学整数，不模拟 RTL 截断
  for i in range(8):
    a = (src1 >> (8 * i)) & 0xff
    b = (src2 >> (8 * i)) & 0xff
    a = a if a < 128 else a - 256
    b = b if b < 128 else b - 256
    total += a * b

  result128 = total & 0xffffffff             # 仅编码低 32 位补码
  write_full_vector(inst.vd, result128)      # 高 96 位因此为零
  # vl/vtype/vxrm/vxsat/vcsr 不变；vstart 仍为 0；向量状态按写回置脏。
```

独立数学模型负责输入数据与结果计算；完整指令模型在其外层加入编码匹配、向量状态、配置检查和精确异常处理。模型的输入检查用于识别测试数据是否合法，处理器异常则按指令模型和已有陷阱机制产生。

2.1.4 P/R 两槽数据通路

两槽实现采用 FuConfig.piped=false 与 UncertainLatency() 框架接口。piped 字段表达框架对确定时延的要求，P/R 则表达 FU 内部缓冲状态；两槽弹性推进与不定时延响应在同一接口下配合。反压、取消和父级资源互斥共同决定实际完成时机。

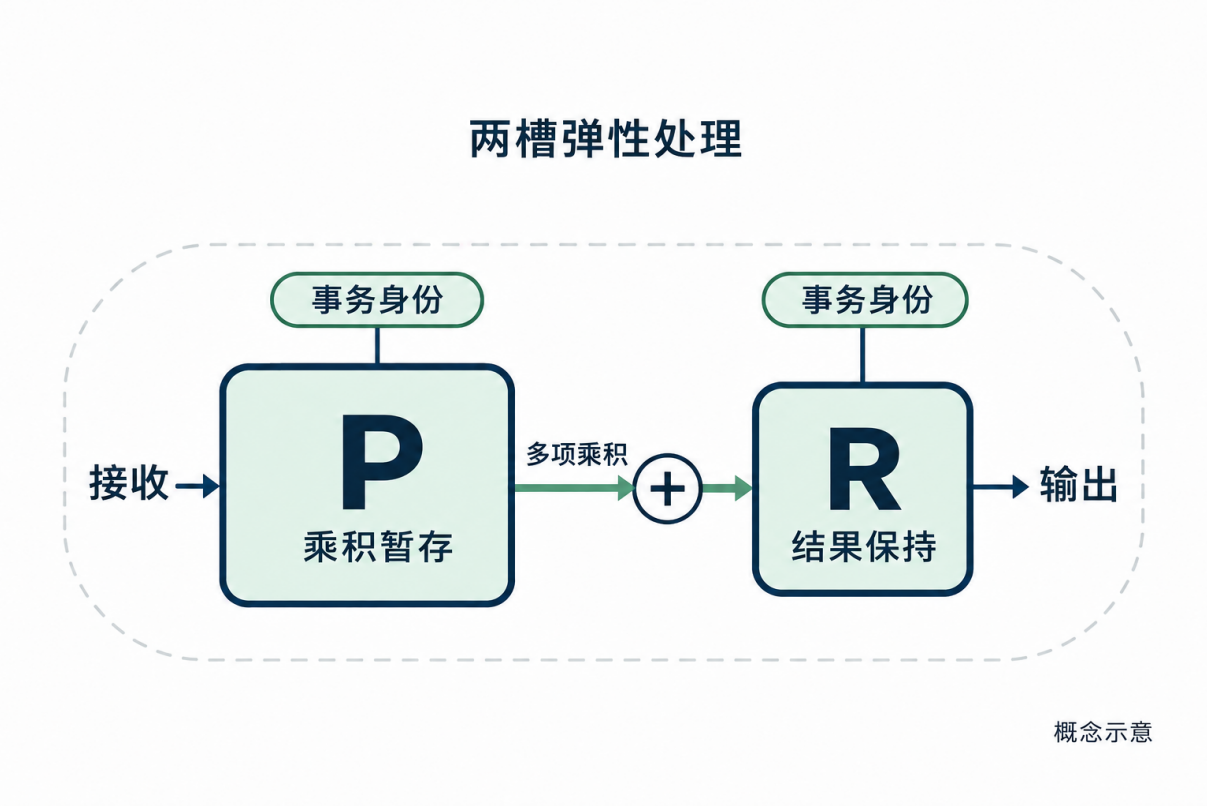
VDot 使用独立 FuncUnit 维护两笔请求的状态，输入可复用 VecFuncUnitAlias，P、R 分别保存各自的事务包。原单份 RegEnable/DataHoldBypass 身份结构按单在途设计，两槽路径因此独立管理有效位、结果和控制。原 O2 合同以完整 128 位结果定义逻辑接口；当前 H1/0f 将结果状态物理保存为 32 位，输出拼接高 96 位零。

表 2-3 P/R 两槽数据通路

槽位	有效状态与数据	必须一同保存的身份
P：乘积槽	valid、8 个 16 位有符号乘积、输入合法性	接受时的 ROB、pdest、vecWen/v0Wen、需要输出的完整 vpu 控制及 perfDebugInfo；字段清单按 FuncUnitOutput 逐项核对
R：结果槽	valid、32 位结果状态、异常向量；输出端拼接为完整 128 位	从 P 转移的同一包身份，与该结果状态一同保存

被接受的非法配置请求携带异常状态进入槽，并以对应身份终结一次。v0 与普通向量目的分别使用匹配的写使能和 pdest。输出数据、异常、vpu 控制及 perf 信息统一取自 R 槽，在反压期间保持完整包稳定。

父级通过 P、R 真实 valid 或精确 occupancy/empty 归约获知 VDot 占用，派发和门控据此决策。观察 seq 用于关联日志，保持在硬件功能之外。若验证端另有计数副本，逐拍断言其与真实槽状态一致，使功能状态、父级选择和检查器账本可相互核对。



概念示意：乘积暂存与结果保持。技术依据见图 2-2。

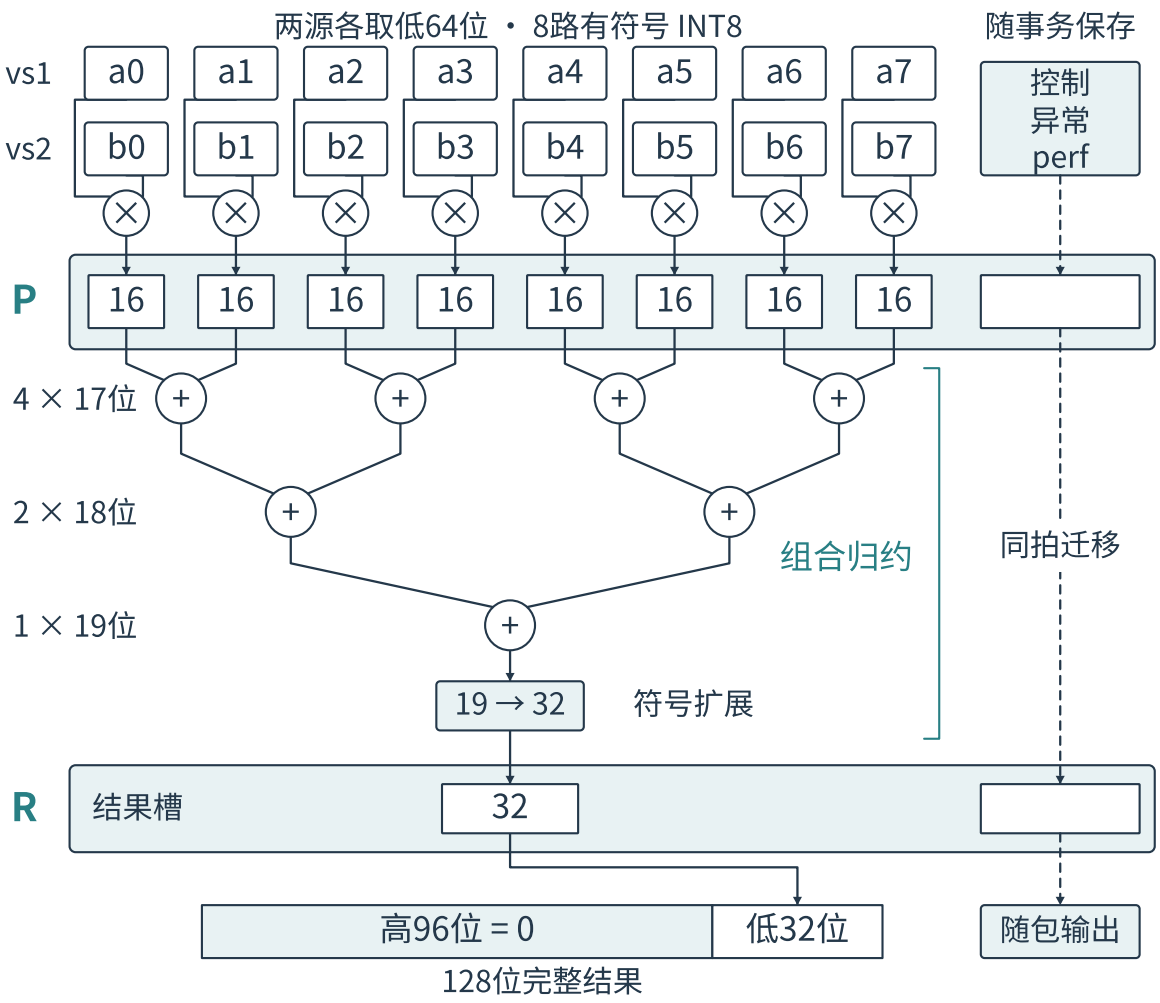


图 2-2 0f 点积算术与 P/R 两槽数据通路

8 路乘积经 P 槽、组合归约和 R 槽输出，事务身份随数据传递。持续 ready、允许接收且无取消时，局部 FU 接口最早 2 拍响应，连续接收间隔为 1 拍。

```scala
class VDotProducts extends RawModule {
  val io = IO(new Bundle {
    val vs1 = Input(UInt(128.W))
    val vs2 = Input(UInt(128.W))
    val products = Output(Vec(8, SInt(16.W)))
  })

  for (lane <- 0 until 8) {
    val a = io.vs1(8 * lane + 7, 8 * lane).asSInt
    val b = io.vs2(8 * lane + 7, 8 * lane).asSInt
    io.products(lane) := a * b
  }
}

/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
```

(a) VDotProducts 乘积计算

```scala
/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
  val io = IO(new Bundle {
    val products = Input(Vec(8, SInt(16.W)))
    val result = Output(UInt(128.W))
  })

  private val sums17 = Wire(Vec(4, SInt(17.W)))
  private val sums18 = Wire(Vec(2, SInt(18.W)))
  private val sum19 = Wire(SInt(19.W))

  for (pair <- 0 until 4) {
    sums17(pair) := io.products(2 * pair) +& io.products(2 * pair + 1)
  }
  for (pair <- 0 until 2) {
    sums18(pair) := sums17(2 * pair) +& sums17(2 * pair + 1)
  }
  sum19 := sums18(0) +& sums18(1)

  private val result32 = Wire(SInt(32.W))
  result32 := sum19
  io.result := Cat(0.U(96.W), result32.asUInt)
}

/** Combinational arithmetic top for independent verification.
  * The future functional-unit wrapper instantiates the same two submodules
  * separately, with registers between them; this top has no cycle protocol.
  */
```

(b) VDotReduce 归约求和

图 2-3 VDot 乘法与归约代码

VDotProducts 生成 8 路有符号乘积，VDotReduce 完成归约求和。

2.2 RTL 功能单元扩展与父级接入

2.2.1 译码、微操作与配置依赖

译码按照精确 mask/match 识别 vdot，将 vs1、vs2 和 vd 解释为向量寄存器字段，并建立与本条指令对应的配置依赖。执行采用单个微操作、单次完整向量写回。编码测试同时检查字段位置和寄存器类别，使机器字、源读取与目的写回保持一致。

实际 VL 通过重命名后的依赖传入执行端，SEW、LMUL、vill 等控制信息随微操作传播。每条 vdot 据此使用自己的配置快照，支持 vset 与向量操作交错执行。配置测试覆盖密集切换、合法与非法状态交替及取消后的恢复，逐层核对配置、源值和事务身份的配对关系。

旧目的寄存器不参与点积累计，译码需要避免无用旧目的依赖。目的可能是 v0，也

可能与某个源相同，因此源快照和目的写回类型必须明确。完成标记、异常信息和写使能与这一次微操作绑定，防止推测执行在取消后留下架构可见更新。

完整目的覆盖也使 H1 可以把结果寄存器收窄为 32 位。高 96 位对所有合法结果恒为零，可由输出端直接构造；ROB、pdest、异常和控制字段仍保存各自事务值。物理状态位数由实现结构决定，软件可见的 128 位结果宽度由指令合同决定，两者在写回接口处连接。

2.2.2 逐拍状态转移与握手

以下量在同一时钟边沿前计算，ready/valid 遵循 Decoupled 握手。P、R 各自调用该请求的 robIdx.needFlush(io.flush)，保留硬件 RobPtr 的 flag 与环绕语义。

killP = P.valid && P.rob.needFlush(flush)

killR = R.valid && R.rob.needFlush(flush)

liveP = P.valid && !killP && !reset

liveR = R.valid && !killR && !reset

out.valid = liveR

out.fire = out.valid && out.ready

rReady = !liveR || out.ready

move = liveP && rReady

pReady = !liveP || rReady

in.ready = pReady && parentAllowsVDot && !flush.valid && !reset

in.fire = in.valid && in.ready

nextR.valid = (liveR && !out.fire) || move

nextP.valid = (liveP && !move) || in.fire

move 为真时，R 锁存边沿前 P 的归约结果与身份；in.fire 为真时，P 锁存新输入的乘积与身份。两者同拍时分别使用旧 P 与新输入。reset 清除两个 valid 及父级占用，并抑制当拍握手；无效槽中的旧数据由有效位隔离。

表 2-4 逐拍状态转移与握手

边沿	接受	槽间转移	输出握手
E0	A 进入 P	无	无
E1	B 进入 P	A 进入 R	无
E2	C 进入 P	B 进入 R	A
E3	D 进入 P	C 进入 R	B

表中给出空管线、持续 ready 且无取消条件下的状态推导：E0 接收 A，E1 推进 A 并接收 B，E2 开始逐拍响应。相应测量在本章无阻塞父级事件图中展示。R 阻塞时由空 P 吸收一笔请求；两槽满后反压输入。部分取消保留未匹配事务，并允许幸存 P 进入已腾空 R。

重定向当拍抑制新请求接收，幸存旧 R 可握手，幸存 P 可推进。终结优先级为 reset、匹配取消、正常响应；同一事务只归入一个终结类别。reset 当拍输入、输出均无握手，复位丢弃数取沿前有效槽数；effectiveKillP=killP && !reset、effectiveKillR=killR && !reset 使取消计数与复位计数互斥。

概念示意：保留、取消与继续处理。技术依据见图 2-4。

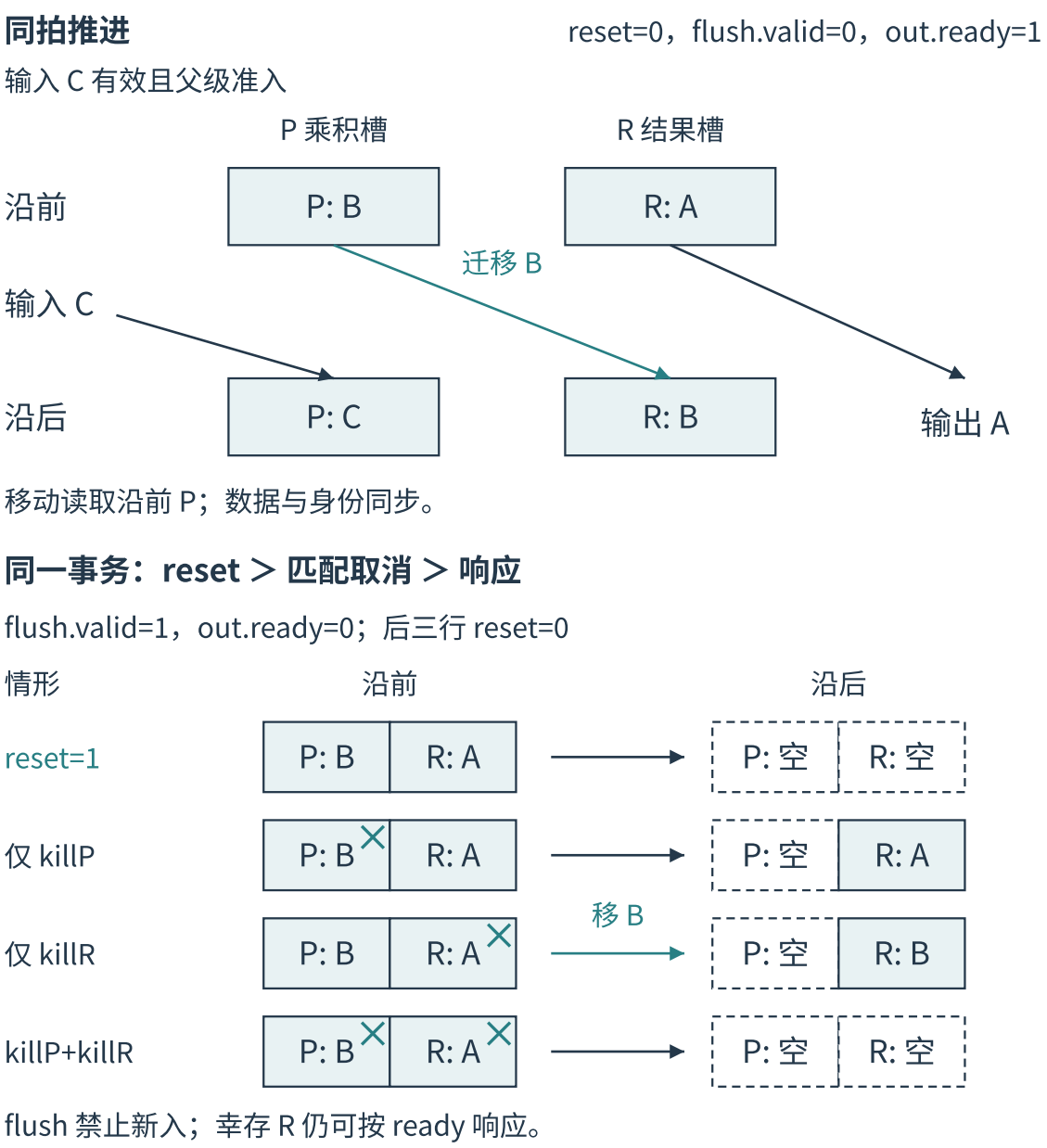


图 2-4　P/R 弹性状态与取消优先级

固定 0f 两槽行为：输入、旧 P 迁移和旧 R 响应可在同一边沿发生；move 读取沿前 P。P、R 各按自身 ROB 判断取消。reset 优先并抑制新入、迁移及响应；重定向当拍禁止新入，幸存 P 仍可填补空 R，幸存 R 仍可按 ready 响应。

```
    require(!cfg.writeFflags && !cfg.writeVxsat && !cfg.flushPipe && !cfg.replayInst)

    val pipelineStatus = IO(Output(new VDotElasticStatus))

>   private val pValid = RegInit(false.B)
    private val rValid = RegInit(false.B)
    private val pProducts = Reg(Vec(8, SInt(16.W)))
    private val pCtrl = Reg(chiselTypeOf(io.out.bits.ctrl))
    private val pPerf = Reg(chiselTypeOf(io.out.bits.perfDebugInfo))
    // H1 stores only the architectural low word; the public result stays 128 bits.
    // Keep each slot's complete control/perf identity on the same transfer edge.
    private val rPacket = Reg(new Bundle {
      val ctrl = chiselTypeOf(io.out.bits.ctrl)
      val res = new Bundle {
        val data = UInt(32.W)
      }
      val perfDebugInfo = chiselTypeOf(io.out.bits.perfDebugInfo)
    })

    pipelineStatus.pValid := pValid
    pipelineStatus.rValid := rValid
    pipelineStatus.occupancy := pValid.asUInt +& rValid.asUInt

    private val illegalInput = vill || vsew =/= VSew.e8 || vlmul =/= VLmul.m1 ||
      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
```

(a) 槽位寄存器与 32 位结果保存

```
PS C:\WINDOWS\system32> wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -Pattern 'private val rReady
=' -Context 5,12

      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
    private val killP = pValid && pCtrl.robIdx.needFlush(io.flush)
    private val killR = rValid && rPacket.ctrl.robIdx.needFlush(io.flush)
    private val liveP = pValid && !killP && !reset.asBool
    private val liveR = rValid && !killR && !reset.asBool
>   private val rReady = !liveR || io.out.ready
    private val move = liveP && rReady
    private val pReady = !liveP || rReady

    // Parent admission is enforced by its atomic child dispatch. The FU also
    // suppresses every new accept on redirect/reset, including unrelated flush.
    io.in.ready := pReady && !io.flush.valid && !reset.asBool
    io.out.valid := liveR
    io.out.bits.ctrl := rPacket.ctrl
    io.out.bits.perfDebugInfo := rPacket.perfDebugInfo
    io.out.bits.res := 0.U.asTypeOf(io.out.bits.res)
    io.out.bits.res.data := Cat(0.U(96.W), rPacket.res.data)
```

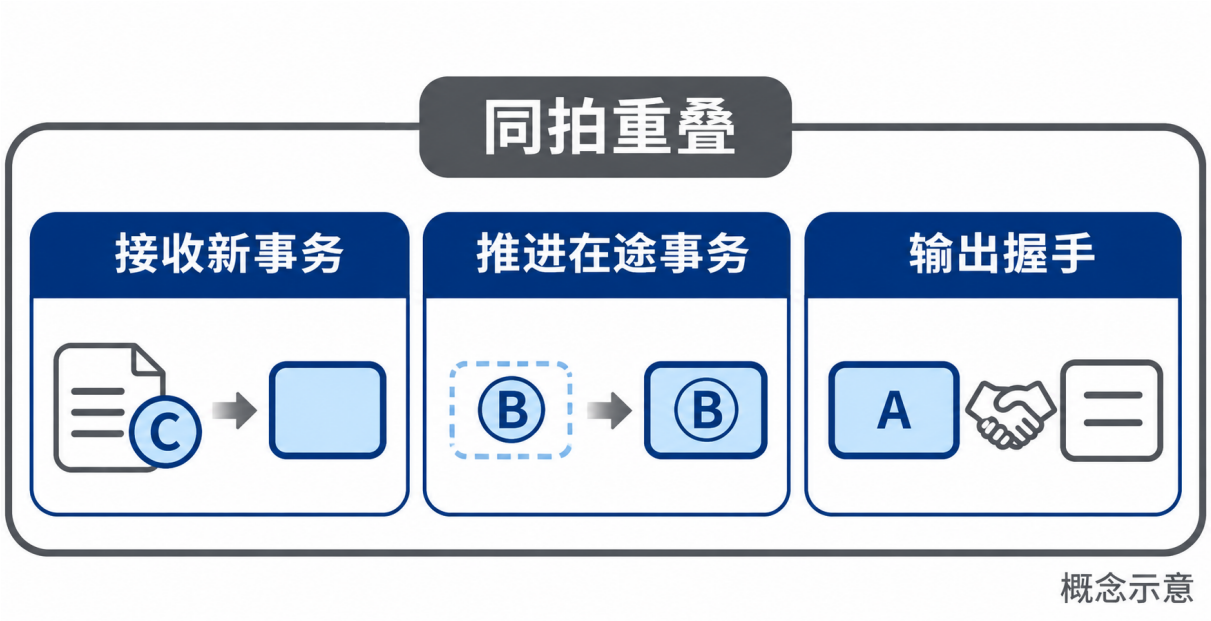
(b) 握手与取消条件

```
PS C:\WINDOWS\system32>
                       > wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -SimpleMatch 'pValid := (live
P && !move) || io.in.fire' -Context 2,16

    acceptedCtrl.exceptionVec.get(ExceptionNO.illegalInstr) := illegalInput

>   pValid := (liveP && !move) || io.in.fire
    rValid := (liveR && !io.out.fire) || move

    when(io.in.fire) {
      pProducts := products.io.products
      pCtrl := acceptedCtrl
      pPerf := io.in.bits.perfDebugInfo
    }
    when(move) {
      // Reads pre-edge P even when a new request replaces it at this same edge.
      rPacket.ctrl := pCtrl
      rPacket.perfDebugInfo := pPerf
      rPacket.res.data := Mux(pCtrl.exceptionVec.get(ExceptionNO.illegalInstr),
        0.U(32.W), reduce.io.result(31, 0))
    }

    // Read-only handles for the separate G1/G2 observation harness. These carry
```

(c) 状态更新与旧 P 迁移

图 2-5 两槽寄存器与控制代码

依次展示寄存状态、握手与取消条件，以及读取旧 P 数据的状态更新。



概念示意：接收、推进与输出的重叠。技术依据见图 2-6。

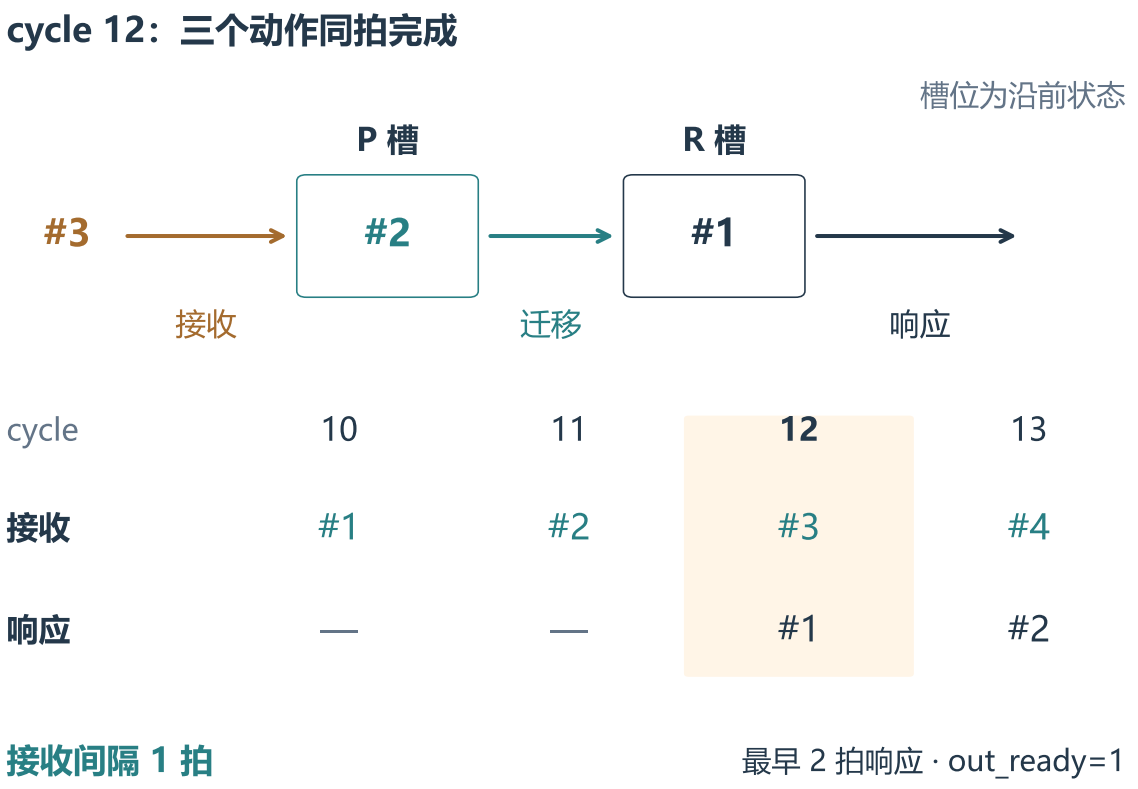


图 2-6　两槽在同一拍的接收、移动与输出

cycle 12 同时完成旧 R 响应、旧 P 迁移和新请求接收。槽位显示沿前状态，seq 表示测试事务号。

```
line 146 cycle=10984 O2_STATE
  pvalid=0 pseq=0 rvalid=0 rseq=0 occupancy=0 invalid=1 inready=1 outvalid=0 outready=1 move=0
line 148 cycle=10984 O2_ACCEPT
  seq=0 rob=1:31 pdest=110 vl=8 illegal=0 vecwen=1
line 149 cycle=10985 O2_STATE
  pvalid=1 pseq=0 rvalid=0 rseq=0 occupancy=1 invalid=1 inready=1 outvalid=0 outready=1 move=1
line 151 cycle=10985 O2_ACCEPT
  seq=1 rob=1:32 pdest=111 vl=8 illegal=0 vecwen=1
line 152 cycle=10985 O2_MOVE
  seq=0 rob=1:31 pdest=110 illegal=0 vecwen=1
line 153 cycle=10986 O2_STATE
  pvalid=1 pseq=1 rvalid=1 rseq=0 occupancy=2 invalid=1 inready=1 outvalid=1 outready=1 move=1
line 155 cycle=10986 O2_ACCEPT
  seq=2 rob=1:33 pdest=112 vl=8 illegal=0 vecwen=1
line 156 cycle=10986 O2_MOVE
  seq=1 rob=1:32 pdest=111 illegal=0 vecwen=1
line 157 cycle=10986 O2_RESPONSE
  seq=0 rob=1:31 pdest=110 data=0x00000000000000000000000000020000 vecwen=1

At cycle 10986: ACCEPT seq=2, MOVE seq=1, RESPONSE seq=0.
```

图 2-7　两槽重叠事务日志

clean-02 dense-batch128 日志中，10986 周期同时发生 ACCEPT seq2、MOVE seq1 与 RESPONSE seq0，分别对应新请求接收、旧 P 迁移和旧 R 响应。

2.2.3　VFEX4 派发与共享资源

VFEX4 包含 VDot、VIDiv 和 VFDivSqrt，共享输出采用 one-hot 选择和 ready 广播。两槽路径沿用互斥派发规则，确保任一时刻只有一个可完成来源：

1. 有任意 vdot 在途时，只能向 vdot 追加请求；DIV 禁止接收。

2. 任意 DIV 在途时，vdot 禁止接收，保持已有 DIV 串行行为。

3. vdot 最后响应或取消当拍不切换 DIV；在下一拍确认两个槽确实空、无旧 vdot 输出后，才可接受 DIV。

4. DIV 完成或取消后，确认旧输出撤销且子单元具备合法接收条件，再放行 vdot；切换根据实际有效状态与握手推进。

5. 跨 FU 切换保持输出有效来源唯一，DIV 的取消当拍行为按对应历史运行记录及全核合同核对。

6. 持续 vdot 供给可能推迟排空切换，沿用现有调度策略；混合负载记录等待，有限输入停止后检查排空与 DIV 最终完成。

O2 在 VFEX4 局部选择被选 FU 的 ready，并保持父子握手原子性。原 Dispatcher 的总 ready 由多个 ready 相与形成，child.valid 不与总 ready 同步相与，局部接入因此同时约束父级接收与子级接收。原有其他 EXU 派发语义保持不变，接口满足：

parent.in.fire == OR(all child.in.fire)

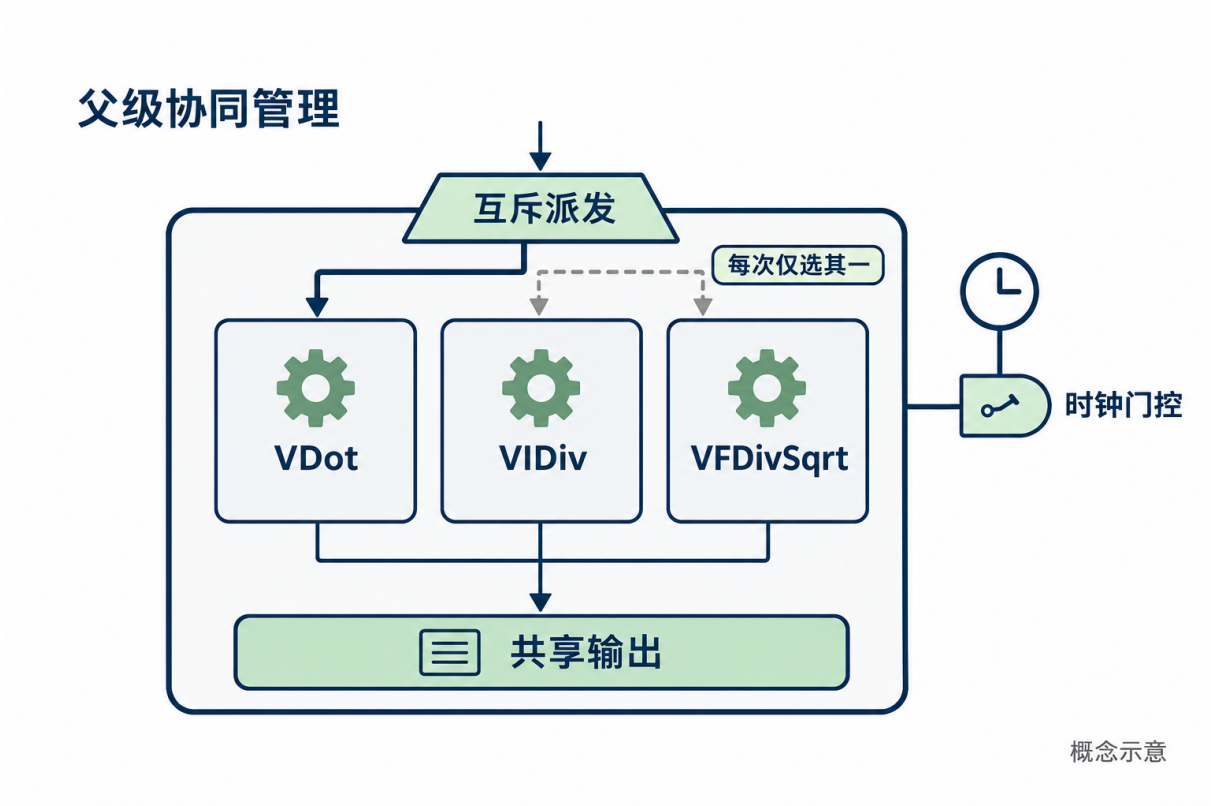
PopCount(all child.in.fire) <= 1

child.in.fire -> parent.in.fire && selected(child)

VDot 占用从两个真实槽状态取得，DIV 占用与身份独立保存。每次响应或取消只释放其对应事务，其他幸存槽继续有效。父级据此判断追加 VDot、等待排空或切换 DIV，

保证双在途身份与资源占用一致。

输入选择依赖已注册占用、当前合法选择和下游 ready，valid/ready 路径保持无组合环。请求被选择但尚未握手时仍由上游保存，FU 在 in.fire 后才更新槽数据与身份。



概念示意：VFEX4 父级派发与共享输出管理。技术依据见图 2-8。

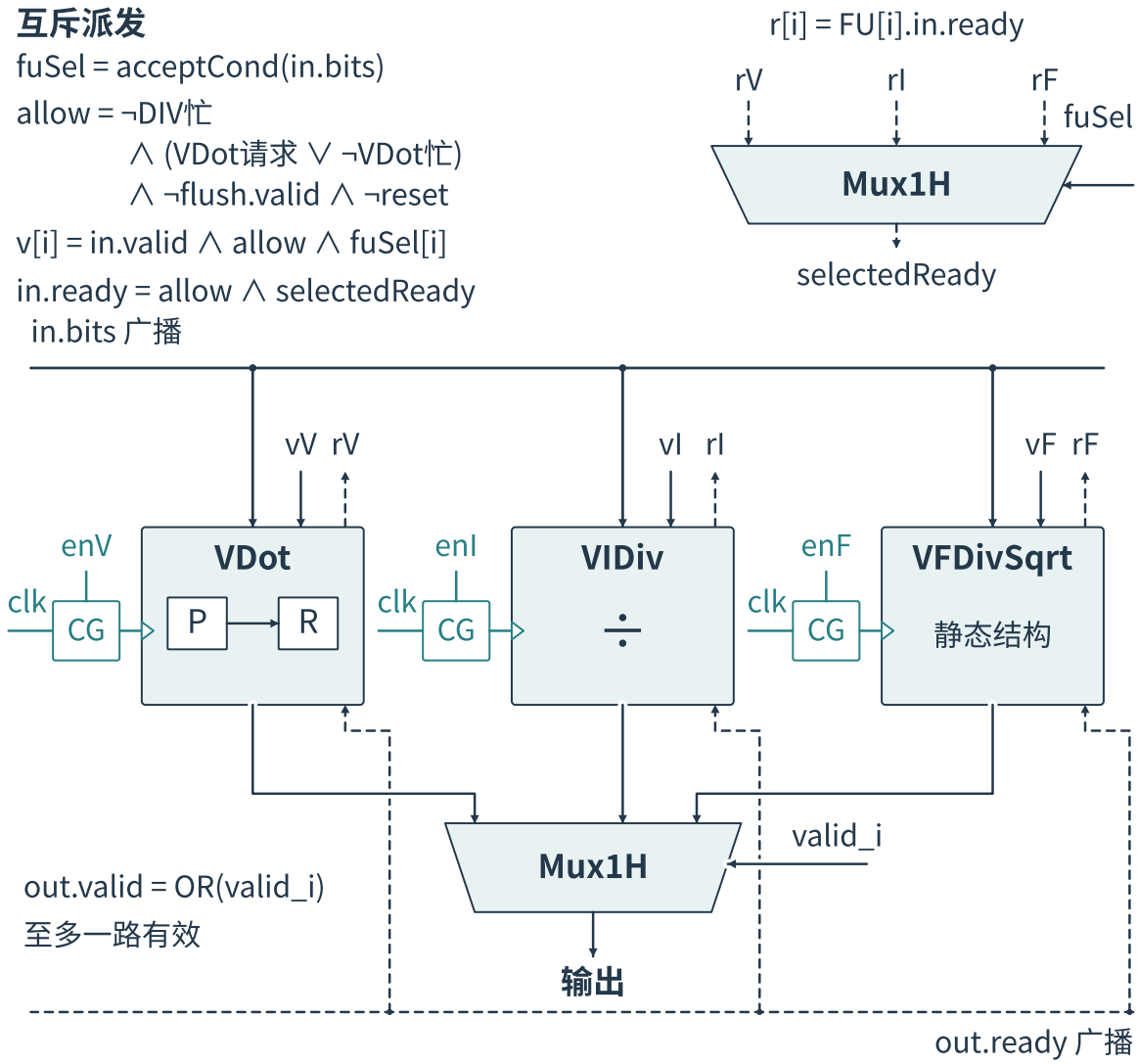


图 2-8　VFEX4 父级的派发、互斥与门控结构

固定 0f 且 VDotO2.enabled 与 EnableClockGate 开启时，VDot、VIDiv 和 VFDivSqrt 共享派发与写回资源，分别通过独立子时钟门控工作。VFDivSqrt 列仅表示静态结构，不能据此增加动态验证结论。

2.2.4　时钟门控、唤醒与写回

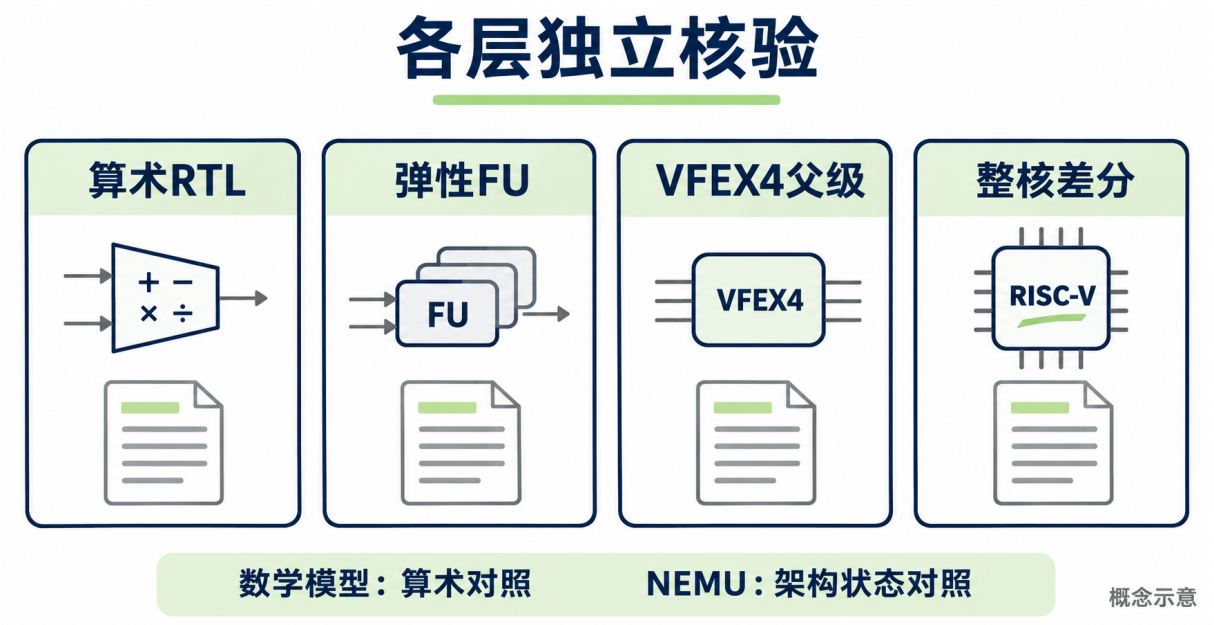
- 原单在途路径使用 uncer_en_reg，在 out.fire 且没有 in.fire 时清零。O2 可能仍有 P 事务，专用使能因而覆盖任一槽有效、当前输入接收以及必要复位和清理边沿。
- 当前槽有效时，即使 flush 将其全部取消，也必须保留处理本次清除所需的时钟边沿；排空后再关门。父级状态逻辑位于未门控时钟域。
- 原 O2 接入首先扩展 VDot 占用与门控，保留既有 DIV 路径；后续 0f 修复匹配 DIV 取消后的保持状态释放。真实父级验证使用 ClockGate，检查已关钟空槽复位、非空复位、cancel 与 reset 同拍以及重新唤醒，使复位传播和门控寄存器状态在实际时钟路径上对应。
- 完成通知、调度唤醒和写回仲裁沿用既有不定时延机制。输出尚未被接受时，R 持

续保持完整数据和控制包，等待真实写回 ready。

- 接入保持现有完成与唤醒关系，启动间隔通过实际父级输入握手测量；issue 反馈或写回竞争引起的等待记录在相应运行窗口中。
- 执行路径如需改为确定时延结构，需同时评审 FuConfig、控制流水、调度唤醒、寄存器读端口和写回预留；这些属于独立架构方案，采用单独合同。

2.3 基础功能验证

基础验证覆盖独立数学模型、算术 RTL、弹性 FU、真实 VFEX4 父级和全核差分五个层次。算术集合包含 22 组定向与 10000 组随机输入；全核矩阵包含 117 个不同用例，按原矩阵分片执行 124 次。0f 的 FU 与真实父级原记录及 4 ＋ 16 项负控纳入复核，R5/G5 采用 13 项原记录与 1 项新控制诊断的组合验收。



概念示意：不同层级的验证视角。技术依据见图 2-9。

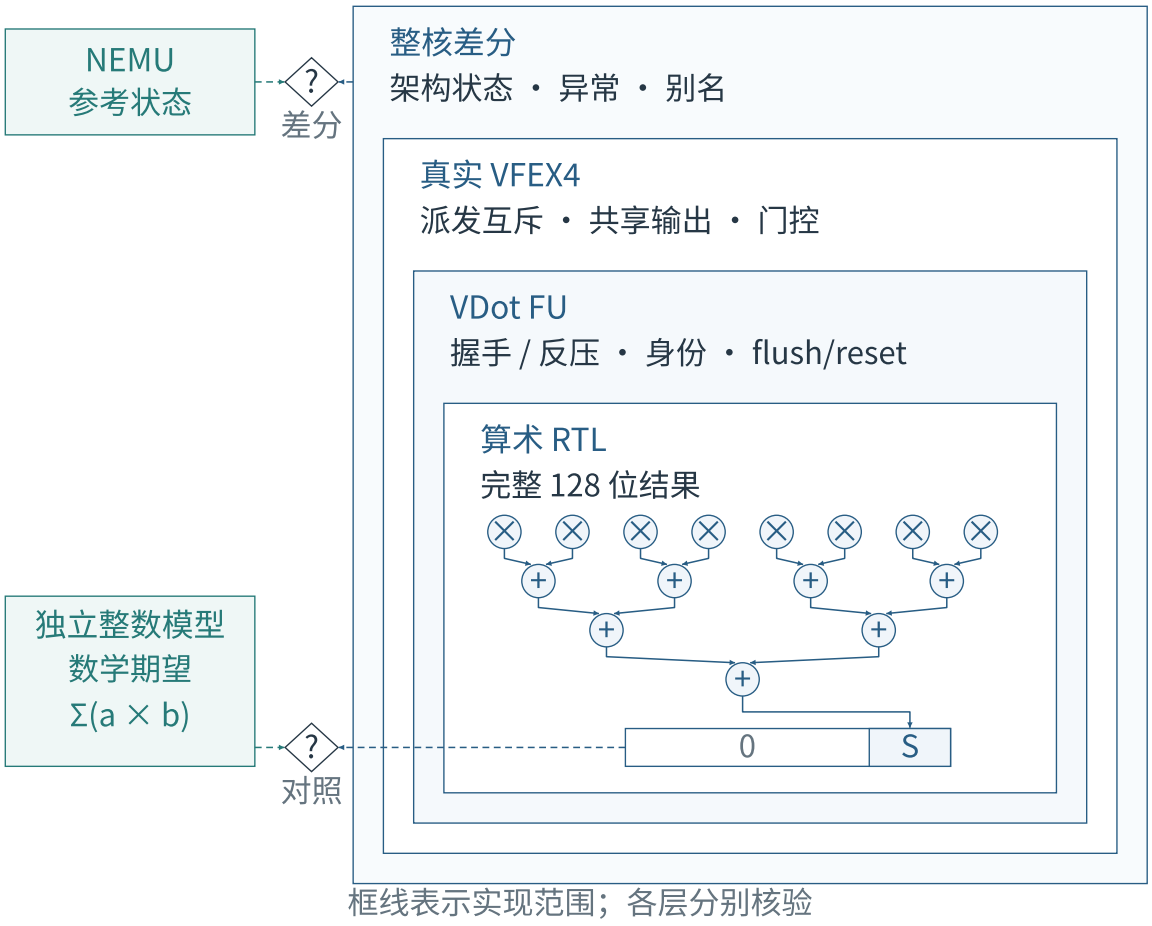


图 2-9 基础验证层级与覆盖范围

嵌套框表示 VDot 相关实现范围由算术 RTL 扩展到 FU、真实 VFEX4 父级和整核；各层分别核验并保留原证据。独立整数模型用于完整 128 位算术对照，NEMU 用于对应架构状态比较。实现包含不表示测试集合全包含，也不合并为一次新的 PASS。

图 2-10 主机参考模型测试结果

主机模型及测试数据检查共 35 个测试方法通过。

2.3.1 正数点积测试

正数用例将 [1,2,3,4,5,6,7,8] 与自身相乘并求和，精确结果为 204，目的低 32 位为 0x000000CC，高 96 位为零。该输入用于检查八项乘法、归约和完整向量写回；对应的正数事务响应与期望值在本节结果图中展示。

2.3.2 负数点积测试

负数用例计算 8 个 −1 与 8 个 1 的点积，精确结果为 −8，目的低 32 位为 0xFFFFFFF8，高 96 位为零。检查同时覆盖有符号输入解释、低 32 位补码以及完整结果格式；结果图保留对应负数事务的完整响应。

2.3.3 边界点积测试

八个 −128 与自身点积得到正端点 131072，目的低 32 位为 0x00020000；该值超过 18 位有符号数的正上限，用于检查归约末级位宽。八个 −128 与八个 127 点积得到负端点 −130048，低 32 位为 0xFFFE0400。零输入、端点及其他定向向量的精确预期集中列于下表；结果图展示正端点事务的完整响应。

```
VDOT_DATAPATH_SUITE_PASS suite=directed count=22
VDOT_DATAPATH_SUITE_PASS suite=random count=10000
VDOT_DATAPATH_PASS directed=22 random=10000 total=10022
```

(a) 组合算术 RTL 测试统计

```
[POSITIVE] functional-2 seq=1 ROB=0:39 pdest=103
vs1 (128-bit)     0xa5a5a5a5a5a5a5a50807060504030201
vs2 (128-bit)     0xa5a5a5a5a5a5a5a50807060504030201
expected sum      204
expected (128-bit) 0x000000000000000000000000000000cc
actual  (128-bit) 0x000000000000000000000000000000cc
ACCEPT cycle=339296 line=7030 | RESPONSE cycle=339298 line=7034
```

(b) 正数事务：期望值 204 与完整响应

```
[NEGATIVE] functional-0 seq=4 ROB=0:12 pdest=143
vs1 (128-bit)     0xa5a5a5a5a5a5a5a5ffffffffffffffff
vs2 (128-bit)     0xa5a5a5a5a5a5a5a50101010101010101
expected sum      -8
expected (128-bit) 0x0000000000000000000000000000fff8
actual  (128-bit) 0x0000000000000000000000000000fff8
ACCEPT cycle=420291 line=7927 | RESPONSE cycle=420293 line=7931
```

(c) 负数事务：期望值 -8 与完整响应

```
[BOUNDARY] functional-0 seq=5 ROB=0:43 pdest=66
vs1 (128-bit)     0xa5a5a5a5a5a5a5a58080808080808080
vs2 (128-bit)     0xa5a5a5a5a5a5a5a58080808080808080
expected sum      131072
expected (128-bit) 0x00000000000000000000000000020000
actual  (128-bit) 0x00000000000000000000000000020000
ACCEPT cycle=448417 line=8400 | RESPONSE cycle=448419 line=8404
```

(d) 边界事务：期望值 131072 与完整响应

图 2-11 正数、负数与边界用例的完整 128 位结果

组合 RTL 测试统计与三类全核观察器事务摘录。actual 取自响应记录，expected 按低 8 个 signed int8 输入计算。

2.3.4 完整定向向量与精确预期

[x]×8 表示重复 8 次。列表按 lane 0→7 排列，未另行说明时源高 64 位为零；输出高 96 位在所有用例中均为零。

表 2-5 完整定向向量与精确预期

ID	源 A 的低 8 项	源 B 的低 8 项	S（十进制）	目的低 32 位（十六进制）
D00	[0]×8	[127]×8	0	00000000
D01	[1]×8	[1]×8	8	00000008

表 2-5 完整定向向量与精确预期（续）

ID	源 A 的低 8 项	源 B 的低 8 项	S（十进制）	目的低 32 位（十六进制）
D02	[1,2,3,4,5,6,7,8]	同 A	204	000000CC
D03	[-1]×8	[1]×8	-8	FFFFFFF8
D04	[-1]×8	[-1]×8	8	00000008
D05	[127]×8	[127]×8	129032	0001F808
D06	[-128]×8	[127]×8	-130048	FFFE0400
D07	[-128]×8	[-128]×8	131072	00020000
D08	[1,-2,3,-4,5,-6,7,-8]	[8,7,6,5,4,3,2,1]	0	00000000
D09	[-128,0,0,0,0,0,0,0]	[-1,0,0,0,0,0,0,0]	128	00000080
D10	[0,0,0,0,0,0,0,-128]	[0,0,0,0,0,0,0,127]	-16256	FFFFC080

2.3.5 数据布局、别名与异常测试

表 2-6 数据布局、别名与异常测试

ID	测试内容	判定标准
L01	逐 lane 一热：只令 A 的 lane i 非零，B 的 8 项互不相同，i 遍历 0～7	每次等于对应那一项乘积，检测遗漏和单侧错位
L02	保持低 8 项不变，两个源高 64 位分别换成随机值、全 FF、交替 55/AA	结果完全相同
L03	独立目的寄存器预填全 FF，再执行 D03	目的完整值为 00000000_00000000_00000000_FFFFFFF8
L04	不重叠、vd=vs1、vd=vs2、vs1=vs2、三者相同；含 v0/v31	使用实际执行前源快照算期望；别名不能使源被提前破坏
L05	四种 vta/vma 组合	每种均得到相同完整目的值，保留原配置

表 2-6 数据布局、别名与异常测试（续）

ID	测试内容	判定标准
S01	vxrm 四种值、vxsat 0/1 组合，预置后执行合法指令	算术结果不变；上述状态及其 vcsr 映射保留
S02	检查 vl/vtype/vstart、非目的寄存器与内存	除正常提交、向量置脏等既有机制外，无额外副作用
E01	逐项改变 SEW、LMUL、vl、vstart、vill，或关闭向量状态	模型显式拒绝；完整指令实现发生非法指令异常，无部分结果
E02	vm=0、逐个翻转固定 funct 位；合法寄存器字段遍历	只识别精确 mask/match；确认相邻模式未被误译码为 vdot
C01	连续请求、输出反压、结果依赖、同一目的连续写、flush	无丢失/重复/错序提交，取消后无已提交写回
R01	原 hello/vadd 及相关向量回归	保持正确；vdot 新路径不破坏旧指令

L01 针对遗漏 lane 与单侧错位。两源同时采用相同排列时数学点积保持不变，这一情况归入模型不变量检查；源字段位置与寄存器依赖由译码、重命名观察点核对。CSR 及非法配置用例使用合法准备指令，保存目标 vdot 执行前的实际 CSR 状态。

非法配置通过架构可达状态构造。标准 vset 指令设置非法 vtype 时，vill=1 与 vl=0 联动，记录按这一联合状态核对目标 vdot 的非法异常及无部分更新。非零 vstart 在向量配置和加载完成后写入；向量关闭用例先准备数据，再关闭 VS，保证被测异常来自目标指令。

2.3.6 随机输入与完整结果读回

- 初始数学模型和算术计划采用固定种子 0x20261011 及至少 10000 组 signed int8 输入，全核冒烟使用定向向量和 32 组固定随机输入。该计划记录与本章 22+10000 算术集合、117 不同全核用例及 124 次执行的实际统计分别标识。
- 保存生成器版本、实际输入、完整期望值和合同版本，种子用于重新生成同一算法下的序列；跨语言复现直接对照归档输入。
- 交叉检查交换两个源、同时重排两边低 8 项等不变量。涉及 int8 取负时排除 -128，避免把不可表示的 +128 当成合法输入。
- 失败应输出指令字、PC、配置、源/目的编号、指令前源值、完整实际/期望目的值；定向用例输出用例 ID。

- 独立模型、算术 RTL、FU/父级、NEMU 和全核 DiffTest 分别记录观察对象与结果，使用同一数学定义连接各层检查。

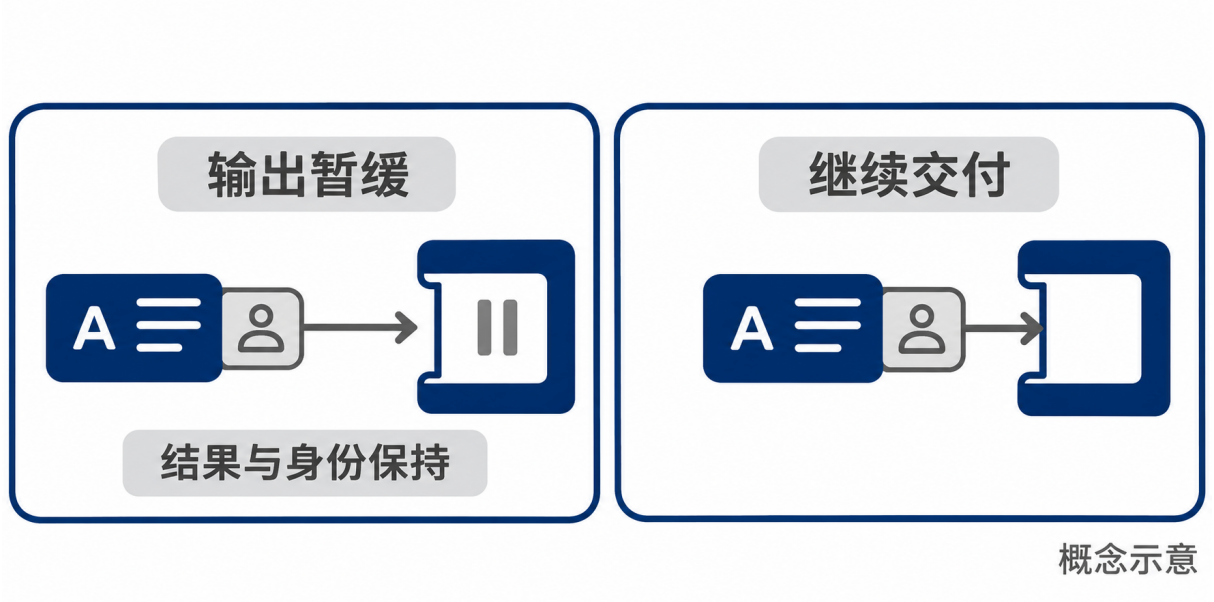
软件读取结果覆盖全部 16 字节。例如指令执行后切换到 e32,m1,vl=4，保存四个 32 位元素，再恢复约定的 vl/vtype。读回程序记录准备、结果保存和状态恢复三个阶段；被测指令的副作用检查以执行前后指定观察点比较。

2.4 协议与系统验证

2.4.1 反压期间的结果与身份保持

io.out.valid 表示 R 中存在可响应事务，io.out.ready 决定该拍能否完成握手。反压期间，R 槽的结果、ROB 标识、目的物理寄存器、异常向量、写使能和性能字段作为一个整体保持。检查器逐字段核对这个输出包，使数值与写回目标同时受约束。

验证为每个接受事务记录独立身份，响应时与该身份的参考值对应。负控包括交换两事务 pdest、改变 ROB、篡改高位、制造重复响应或漏响应。实际硬件协议测试与检查器日志变异分别记账，前者验证设计，后者验证证据审查能力。



概念示意：等待期间保持事务。技术依据见图 2-12。

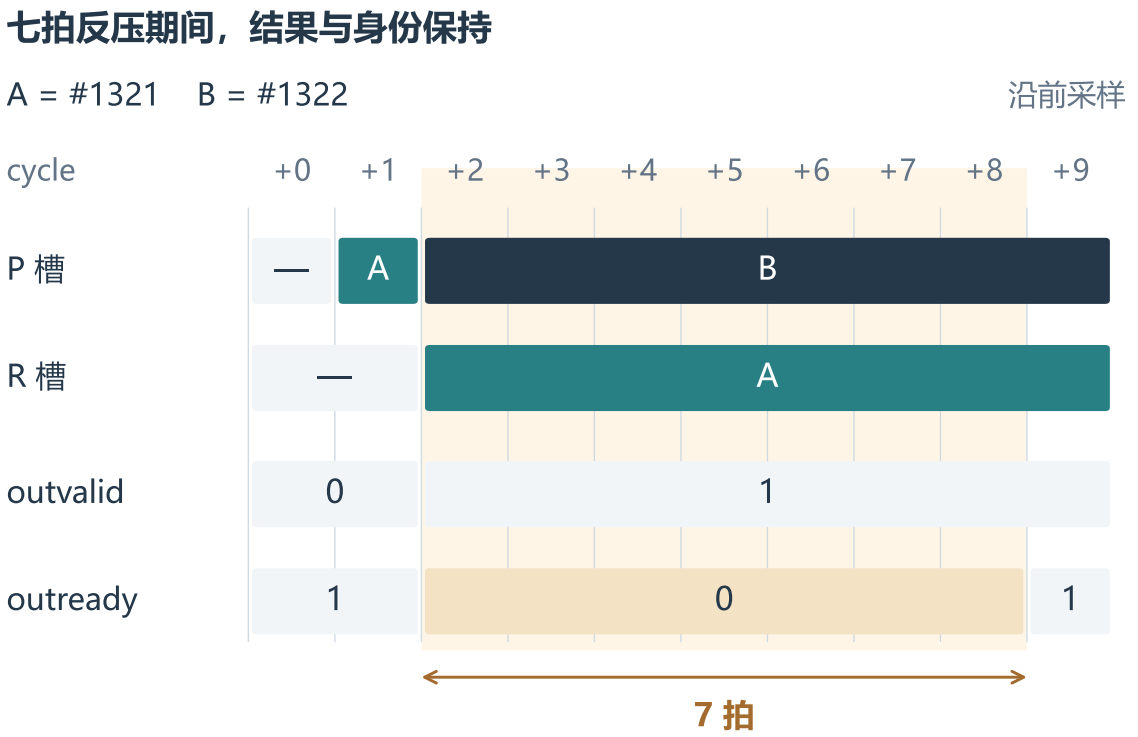


图 2-12 反压期间的结果与事务保持

seq1321 在 181599—181605 连续反压 7 拍，完整结果为 0x00000000000000000000000000001A87。181606 响应一次，同时旧 P 事务 1322 转入 R；FU vready 恢复，父级 inready 仍为 0。

2.4.2 ROB 取消与复位优先级

redirect 按 RobPtr 的 index 与环绕 flag 共同判断 P、R 的程序年龄。测试覆盖 flag 变化、flushItself 两种取值以及接受顺序与年龄顺序不同的情况。每个窗口先确定应保留和应取消的身份集合，再核对幸存事务输出与被取消槽的有效位清除。

同一拍发生输出 ready 和匹配取消时，取消决定该事务是否仍可形成响应。reset 则清除槽有效状态，并检查接口握手被抑制。真实父级还需验证已经关钟的空槽可以正确复位，以及取消后重新唤醒的第一笔请求不丢失。

任意前缀都满足事务守恒。接受数等于已响应数、已取消数、复位丢弃数和当前在途数之和。当前在途数最大为 2。这个关系能够发现尾部漏计、幽灵响应和被取消事务再次输出等错误，并要求正常结束时排空。

例如沿前两槽有效且 reset 与匹配 redirect 同拍时，优先级使两项计入复位丢弃，取消和响应不再重复计数。仅 R 取消而 P 幸存时，取消集合增加一项，P 转入 R 保持原身份，在途数随之减一；槽间移动本身不增加接受数。守恒式把这些不同动作统一到同一个数量关系中。

概念示意：取消与复位的处理边界。技术依据见图 2-13。

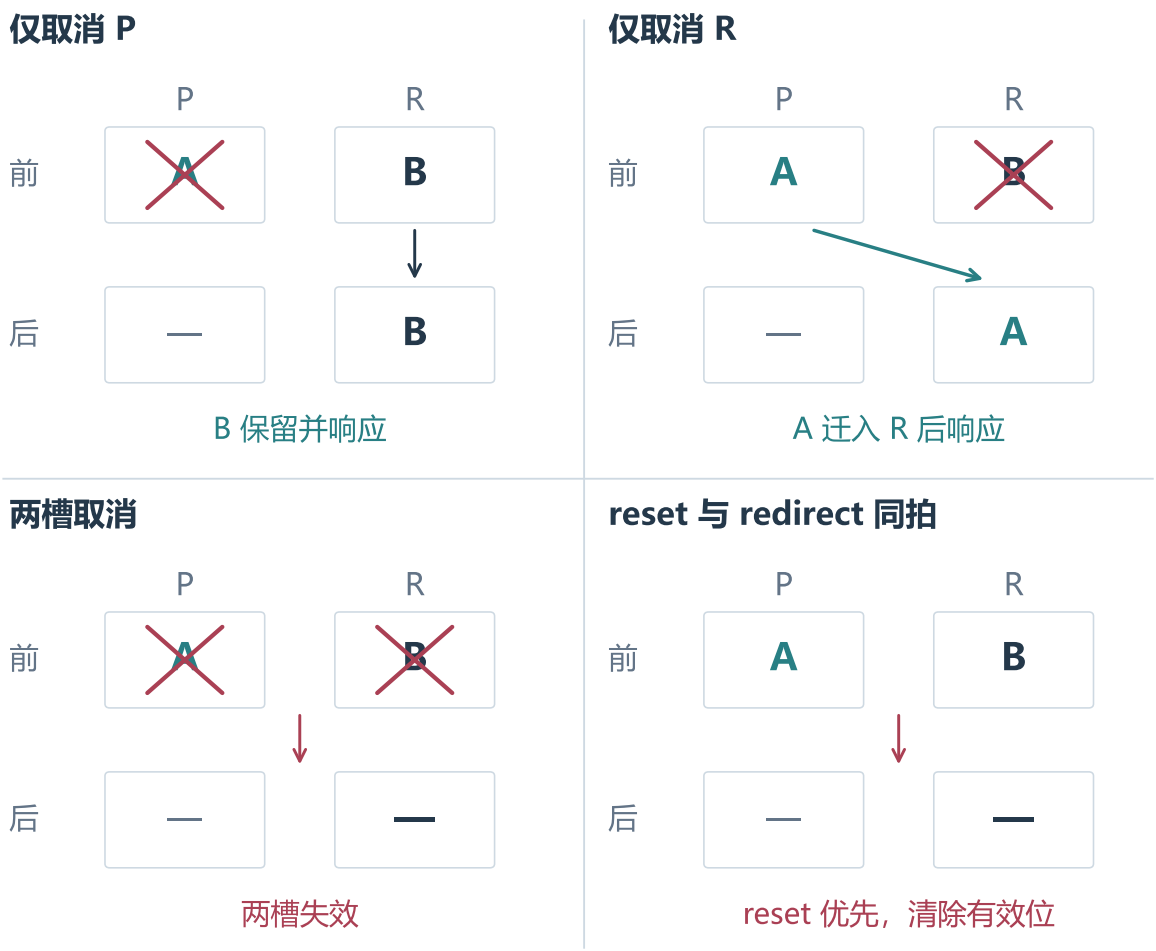


图 2-13　取消与复位的槽位变化

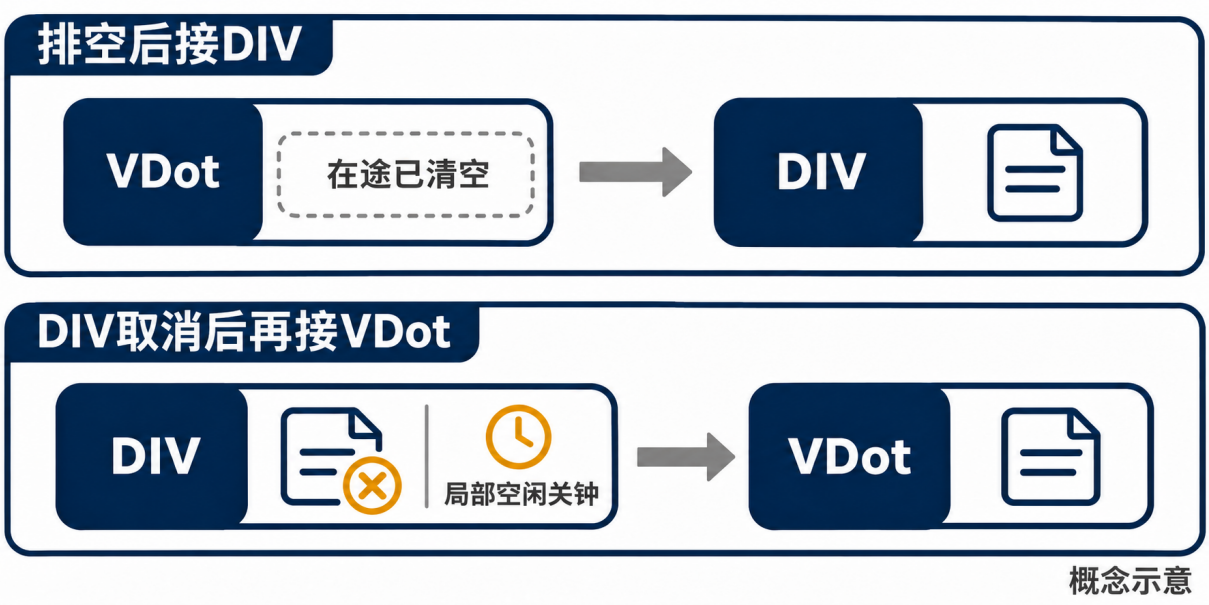
四个独立场景分别展示仅 P 取消、仅 R 取消、两槽取消和 reset 优先清除。A/B 为各场景内的事务标记，取消依据 ROB 年龄。

2.4.3　DIV 切换、取消清理与关钟

VFEX4 负责 VDot 与 DIV 的互斥派发、唯一完成源选择和占用释放。动态验证使用真实 ClockGate，覆盖 VDot、整数 DIV、浮点 DIV 之间的切换，以及反压、取消、复位和重新唤醒；SQRT 取消路径采用静态检查。图中以时钟上升沿后的子时钟采样状态展示取消清理和后续空闲关钟。

资源切换可按占用、派发和完成三个观察点理解。占用说明旧操作是否仍在途，派发说明新操作是否真正握手，完成说明共享输出当前来自哪一类 FU。切换前核对旧输出撤销与子单元接收条件，切换后核对首次接收与响应身份，能够把 busy 释放问题与结果选择问题分开定位。

0f 修复匹配 DIV 取消之后保持状态的释放：取消当拍保留完成清理所需的时钟边沿，随后解除旧占用；仍在途的有效事务继续获得时钟。测试同时观察空闲采样与幸存事务完成，核对取消、资源释放和新请求进入之间的顺序。



概念示意：功能切换与时钟管理。技术依据见图 2-14。

图 2-14　DIV 切换、取消清理与关钟

VDot 排空后接收整数或浮点 DIV，取消清理完成后子时钟归零，再接收新 VDot。图中 clocks 为沿后采样的子时钟位掩码。

2.4.4　写回身份与全核结果检查

向量点积的数学参考值可以由两个源向量独立计算，但它不能告诉检查器该值是否写入了正确目的。测试平台因此为每个接受事务绑定一个身份记录，包含目的物理寄存器、ROB 位置、指令控制及期望结果。响应到达时先核对身份，再核对完整一百二十八位数据。若先按结果值寻找匹配事务，两笔恰好得到同一结果的请求可能被错误互换而不被发现。零结果、相同源向量和不同目的寄存器的交错请求尤其适合暴露这一盲点。

写回使能与异常字段同样需要随身份核查。异常指令可以在执行路径中占有状态，但不能以正常结果覆盖架构目的；被取消的指令在局部槽位里保留旧数据，也不能因 ready 恢复而重新输出。检查器在每次接受、响应和取消时更新账本，并在结束时核对所有身份的去向。该账本提供了比单纯比较输出顺序更强的约束，也能分辨协议漏响应与结果错误。

完整路径复验还应覆盖目的别名和连续配置变化。若两条指令的源与目的存在交叉别名，重命名阶段必须固定各自的源版本；若配置指令夹在两条点积之间，执行端必须持有各自对应的 VL 与 vtype 快照。最终软件结果可能在少数输入上偶然相同，因而需要结合调度记录、响应身份和异常状态判断。只有这些字段与数学结果共同成立，才可将局部算术通过提升为全核写回路径通过。

2.4.5 控制场景的组合与回归覆盖

回归矩阵按失效机理组织：极值与随机输入覆盖算术，配置交错覆盖依赖，别名覆盖源快照，反压覆盖包稳定，取消覆盖年龄和复位优先级，混合操作覆盖共享资源。每项记录主断言及额外观察点，失败时据此定位输入和轨迹，实现变化时据此选择受影响的回归范围。

为了避免组合爆炸，可先对每一维构造最小可区分情境，再挑选交叉点。比如正常单事务可检查基础算术和响应格式；双事务且不同目的寄存器可检查连续接受；双事务加反压可检查载荷稳定；环回 ROB 加选择性取消可检查年龄语义；取消后首笔 DIV 和点积交替可检查父级释放。并不是所有输入值都要与所有控制状态笛卡尔相乘，但每个高风险交叉都应能在矩阵中定位到明确的用例和预期。

随机测试以种子、生成规则和实际输入共同确定可重放序列，并覆盖正负数、零、高位哨兵及别名组合。精确的 18 位溢出正端点由定向用例构造，随机输入补充一般数据分布。两类记录在同一数学定义下分别提供可解释的边界检查和较宽的数据覆盖。

2.5 设计演进、参数扩展与取舍

O2 通过两槽容量支持连续接收，H1 降低结果寄存器开销，0f 完善 DIV 取消后的关钟行为。冻结 0f 构建按 clean-02 配置启用 O2/T14、关闭 O1；通用源码默认开关与该冻结构建分别记录。一拍 P4 候选完成局部验证后进入 47 项同约束筛选，初版 6 项、唯一修订 10 项未通过，因此生产版本选择 0f。

参数扩展实验实现 signed INT16×8 和 64 位结果，完成 94 组定向及 10000 组随机验证，用于检查算术与包装层的参数化方法。当前生产 ISA 仍采用 INT8×8、低 32 位结果及高 96 位清零的合同。

指令合同 • 稳定 FU 源码 • R5 验收矩阵 • 参数扩展

2.5.1 吞吐与实现成本

弹性槽提供的是输入与输出之间的缓冲容量，它改善持续请求或短暂下游阻塞时的利用率，却引入额外的状态寄存器和 ready 路径。对独立指令流，流水重叠可以降低平均每条指令的间隔；对严格依赖链，下一条必须等前一条写回，缓冲容量不一定产生同样收益。独立流与依赖链分别测量，使用各自的周期结果解释流水收益。

综合成本也有层级。乘法树位宽、P/R 寄存器规模、控制比较器以及父级选择逻辑分别贡献不同路径和面积。H1 收窄结果状态可能减少寄存器位数，但若增加末端拼接或改变布线，最差时序端点可能转移。P4 缩短内部算术拍数也可能使组合路径穿越更多逻辑，导致父级 ready 或门控使能的余量变差。综合记录同时列出路径端点、约束、面积和裕量，解释各项成本变化。

2.5.2 本阶段小结与接口衔接

阶段间接口以架构语义冻结：参考模型接收指令字、源向量、控制状态和异常条件，返回完整目的向量及异常结果。数学和指令模型独立于 P/R 具体拍数，允许硬件在保持架构效果的前提下调整内部流水。性能记录则绑定实际硬件版本和开关，描述该实现的周期与成本。

软件输入镜像、编译器与汇编路径、配置前置条件一同冻结。C 接口或链接布局变化后，检查最终指令字与调用、返回约定，分别记录裸指令部分和软件包装部分的开销。差分验证沿用数学与异常合同，性能测量明确入口、出口和状态恢复要求，两类记录通过同一指令语义衔接。

2.6 复现资料与接口索引

2.6.1 汇编封装与编码冲突记录

软件通过独立.S 文件中的.word 0x0284850b 或校验后的编码宏生成机器字，宏同时检查寄存器编号范围。反汇编工具可显示机器字或识别后的名称；指令身份以机器字、字段和最终链接映像为准。C 接口的调用和返回约定见第三阶段软件集成说明。

C 内联汇编需要明确寄存器绑定、clobber 和编译器重排约束，使编译器可见的读写集合与实际向量操作一致。独立汇编封装则通过显式入口、出口与保存恢复约定表达这些关系。两种路径都检查最终目标指令和结果读回。

2026-10-11 在固定 XiangShan 实际译码表和 NEMU 主译码表进行的静态检查未发现与 vdot mask/match 交叠的已接入模式。已有 nemu_trap/TRAP 使用 0x6b。rocket-chip 依赖中 CUSTOM0 通配项属于 Rocket RoCC 译码体系，当时未接入香山实际译码表；启用该路径时需重新核对分配与仲裁。这份记录说明接入前的编码空间，后续译码和全核结果由本章功能验证给出。

2.6.2 源码接入位置与职责

表 2-7 源码接入位置与职责

子系统	相对路径	接入职责
XiangShan 指令模式	src/main/scala/xiangshan/backend/decode/Instructions.scala	精确 vdot 模式、相邻编码及既有扩展的模式关系
XiangShan 向量分类	src/main/scala/xiangshan/backend/decode/isa/bitfield/RiscvInst.scala	将精确 vdot 模式纳入向量分类；同时保留其他 custom 编码的独立分派
XiangShan 译码控制	src/main/scala/xiangshan/backend/decode/DecodeUnit.scala	正确的向量源/目的类型、配置依赖、合法性检查、微操作和写回控制
XiangShan 执行接入	src/main/scala/xiangshan/backend/fu/FuConfig.scala，fu/wrapper/，fu/vector/	执行资源、结果位宽、完整目的写回、取消和不定时延接口
NEMU 主译码	src/isa/riscv64/instr/decode.c	custom 主 opcode 分派、向量状态检查及独立 vdot 执行入口
NEMU 向量实现	src/isa/riscv64/instr/rvv/decode.h、exec.h、vcompute.h、vcompute_impl.c	字段解析、配置检查、精确算术及本指令的完整目的写回规则
NEMU 寄存器访问	src/isa/riscv64/instr/rvv/vreg.h、vreg_impl.c	有符号源读取、先读后写、完整 128 位写入
NEMU 声明与差分状态	src/isa/riscv64/include/isa-all-instr.h、src/isa/riscv64/difftest/ref.c	执行入口声明、向量和 CSR 状态同步，确认实际比较范围

表中路径用于定位接入职责，源代码行号对应当时固定的硬件和 NEMU 提交。参考入口：XiangShan 译码目录、NEMU 主译码、NEMU RVV 目录。

固定版本源码显示以下接入关系：

- RiscvInst.scala:93 的向量分类需包含精确 vdot 模式；DecodeUnit.scala:897 的向量禁用检查同时依赖 FU 分类。编码匹配与功能分类分别完成后，合法性检查才能覆盖该指令。
- DecodeUnit.scala:775 的输入带 vtype/vstart，未直接带实际 vl；vl=8 限制须通过正确的 VL 依赖，在能取得 srcVConfig.vl 的路径检查，例如 fu/vector/VecPipedFuncUnit.scala:54，并接通非法指令异常输出。
- DecodeStage.scala:236 对 v0 目的使用独立写使能，重命名也有相应通路；寄存器测试因此包含 v0 及 v31，并核对目的物理映射。

- 原 NEMU 的 instr/rvv/decode.h:167 在 vopivv 子表中将 funct6=0 解释为 vadd，不再检查主 opcode。vdot 使用独立执行分派，复用字段解析和寄存器访问；向量状态检查与置脏沿用 instr/vp.c 的 vp_enable() 和 vp_set_dirty()。

2.6.3 参考模型的构建与运行身份

独立数学模型按精确整数定义计算参考值，NEMU 在其外层实现编码、配置、异常和架构状态更新。差分比较包含完整目的结果及约定状态；可控故障参考与正常参考恢复分别检查错误检出和正常运行。参考模型与 DUT 使用独立的数据计算路径。

共享参考库与 standalone NEMU 分别构建，记录各自提交、配置、产物摘要和实际加载路径。团队维护可访问的源码版本或可复现补丁，运行记录绑定所加载的具体库，保证其他工作区能够还原同一指令模型。

2.6.4 事件记录与检查不变量

O2_SCHEMA version=1 按未门控全局 cycle 记录两槽事件。每次父级接收产生唯一观察 seq，P 至 R 转移保持同一 seq，检查器以此串联接收、等待、取消与响应；seq 只服务于观察，功能决策使用真实槽状态和 ROB 身份。

事件集合包含父级接收与等待、请求种类及选择、ready 及阻塞原因、P_ACCEPT、P_TO_R、R_STALL、R_RESPONSE、P_KILL、R_KILL、RESET 丢弃、槽占用、DIV 开始结束和重定向。检查按统一 cycle 处理旧事务终结、移动和新接收，同时保留原始输出顺序以便定位日志。

必须满足：

- 任意前缀：接受数 = 响应数 + 取消数 + reset 丢弃数 + 当前在途数；同一事务恰好属于一个终结类别。
- 当前在途数在 0..2；两个槽不能持有同一 seq；输出来自 R 中那个身份。
- 幸存输出按接受顺序，不允许重复/幽灵响应；完整 128 位及异常/写使能与独立模型一致。
- 多槽取消逐笔按 ROB 年龄判断，覆盖年龄顺序与接受顺序相反、flag 环绕、flushItself 两种情况。
- 反压保持整个 R 包稳定；只有匹配取消/reset 可以撤销旧响应。
- 父子握手一致，父级 out 与被选子输出一致，FU 输出 one-hot，门控不能冻结未终结请求。
- 正常运行结束时所有已接收事务均有终结记录；主动故障终止保留已检查前缀与停止位置，单独记录运行状态。

2.6.5 原 O2 分阶段判据与负控

下表保留原 O2 从设计到采用的分阶段判据，说明各类验证之间的衔接。当前 0f 的完成情况由本章功能验证汇总及 R5 验收矩阵记录；动态父级记录覆盖 VDot、整数 DIV 和浮点 DIV，SQRT 保留静态取消路径检查的状态。

表 2-8 原 O2 分阶段判据与负控

门禁	核验内容	判据
G0 评估/合同	旧轨迹复算、两条接入路线审查、候选范围及停止条件	形成隔离原型的输入合同与停止条件
G1 算术与弹性 FU	原 22+10000 完整 128 位向量；连续满管线、两槽反压/腾空；合法非法交错、v0 交错；部分/全部取消、reset 和环绕	无反压时真实 II=1 且最早响应 2 拍；事务守恒与反例检出通过
G2 真实父级	真 ExeUnit+VDot+VIDiv+VFDivSqrt+ClockGate；全部 FU 切换、满管线反压、cancel 与切换重合、持续供给及最终排空	无误派发/双完成/误关钟；其他 FU 受影响范围回归通过
G3 全核	新 O2 EMU、正常 NEMU；hello/vadd、117 唯一功能/异常矩阵、T12 等价控制压力；故障参考库及正常恢复	全部 DiffTest/完整状态通过，新观察事件覆盖达标
G4 公平性能	冻结相同 16 镜像、参考库、工具配置，O1/O2 各自实测 custom 与 RVV；同合同四场景和相同样本规则	有实际净收益、退化可解释、结果与身份绑定可复算
G5 采用与交付	O3 应用数据、O4 结构/时序证据、O5/T17 独立干净检出构建及相应复核记录	形成最终模式、冻结版本与交付记录

定向负控覆盖 P/R 身份交换、结果高位篡改、漏取消或误杀幸存事务、重复或丢失响应、反压字段变化、父子握手不一致、双 FU 有效及在途误关钟。硬件注入与日志/检查器注入分别记录故障位置、预期断言和实际检出，保持各自的观察层级。

原 O2 成本筛选将冻结 batch128 相对 O1 降低至少 10% 设为工程目标，即由 362.50 降至不高于 326.25 cycles，同时检查其他保留场景是否出现超过 2% 的均值退化。若 RVV 为 678，该目标约对应 2.078×；正式加速比使用同一候选自身实测 RVV。该目标属于当

时的性能筛选条件，采用判断还结合功能、协议、资源、时序和复现结果。

2.6.6 RVV 对照的功能等价条件

RVV 对照以 8 位有符号输入、16 位精确乘积和 32 位扩宽归约完成相同功能。序列围绕 vwmul.vv 与 vwredsum.vs 组织，中间 SEW/LMUL、寄存器分组、零初值和最终高位清零共同构成功能等价条件。具体冻结序列与计时结果见第三阶段“RVV 对照与周期测量方法”。

基线按以下条件核对：

1. 只计算源低 8 项，从零开始，不把旧目的或源高半部分加入。

2. 不使用会在 8/16 位提前截断的组合；D06、D07 必须通过。

3. 最终目的低 32 位及高 96 位与“点积定义、位宽与结果布局”小节完全相同；普通归约指令的尾部策略不能代替显式验证。

4. 双方在共同入口、出口比较完整 128 位目的结果及约定向量 CSR，vl/vtype 恢复实际入态。临时 scratch 寄存器预先列明并排除于该比较集合，源目的别名用例单独核对源快照。单条 vdot 的严格副作用规则与多指令序列的临时寄存器约定分别定义。

5. 分别记录核心算术窗口和完整等价序列的周期；清零、配置切换、恢复及搬运按共同边界计入，两边的装载、存储计时口径一致。

6. 使用同一硬件基线、输入和测量边界，分别说明依赖链延迟与独立流吞吐。性能指标采用 DUT 周期，主机运行时间用于执行资源记录。

冻结测量记录保存具体序列、入口出口、计时事件和样本规则，详见第三阶段“RVV 对照与周期测量方法”。设计语义确定功能比较集合，测量约定确定周期比较范围，两者共同连接裸指令测试与软件调用实验。

2.6.7 待确认规格与变更记录

下表集中列出内部 v0.1 合同涉及的赛事待确认问题。现有归档尚无命题方正式答复，当前实现和测试依表中内部约定组织；官方口径如有更新，再按变更流程调整对应合同。

表 2-9 待确认规格与变更记录

编号/优先级	需要确认的问题	本版约定及受影响内容
Q01 / P0	图中的 8 项是否为最终验收语义？是否明确取低编号 8 项？	固定低 8 项；影响读取范围、算术规模、测试和基线

表 2-9 待确认规格与变更记录（续）

编号/优先级	需要确认的问题	本版约定及受影响内容
Q02 / P0	“vd 第一个元素”的位宽是什么？是否接受低 32 位精确结果、高 96 位清零？	“点积定义、位宽与结果布局”小节；影响目的格式、写回和完整结果比较
Q03 / P0	是否要求精确结果？是否另有截断、饱和或旧 vd 累加规则？	精确从零求和；影响数据通路与边界结果
Q04 / P0	vl/LMUL/mask/vstart 的必测范围是什么？是否接受“指令编码与合法配置”小节的受限原型域？	固定 e8/m1/vl8/vstart0、无掩码；影响控制及异常范围
Q05 / P1	vta/vma 是否影响“其他通道清零”？是否要求源目的重叠支持？	策略位不改变完整结果，全部单寄存器别名支持
Q06 / P1	是否有指定或禁止的指令编码？是否允许使用 custom-0？	“指令编码与合法配置”小节；影响 RTL、NEMU 与汇编封装同步
Q07 / P1	延迟比较是否有指定输入/出口状态、计时边界或基线要求？	按“RVV 对照的功能等价条件”小节先验证等价，再分别报告各项开销

正式答复按日期、来源及原文归档到 team/spec/clarifications，并与 Q01～Q07 逐项关联。记录同时保留现有约定、答复后的约定和受影响测试，便于重新核对实现与评分口径。

行为变更记录旧行为、新行为、依据和受影响用例，经评审后升级合同标识，同步更新 RTL、NEMU、编码封装与数据。原版本输入和结果继续保留原标识，新版本使用自己的验证与性能记录。

2.6.8 技术资料索引

内部 ISA 合同、数学边界和测试向量；两槽弹性状态、父级协议及验证不变量。

第三阶段：协同仿真与性能分析

工程组合：0f ＋ firstdot V2 ＋原 C8　材料日期：2026 年 11 月 2 日

本阶段完成 NEMU 指令扩展与 RTL 协同仿真，使用现有 RVV 指令构造等价对照，并评估点积及矩阵向量计算的周期与实现成本。最终硬件版本记为 0f，包含两槽弹性执行结构（O2）、低 32 位结果保存（H1）和 DIV 取消关钟修复。软件中的 firstdot 表示首段点积初始化方法，C8 表示 8×32 调用中的向量配置复用入口。

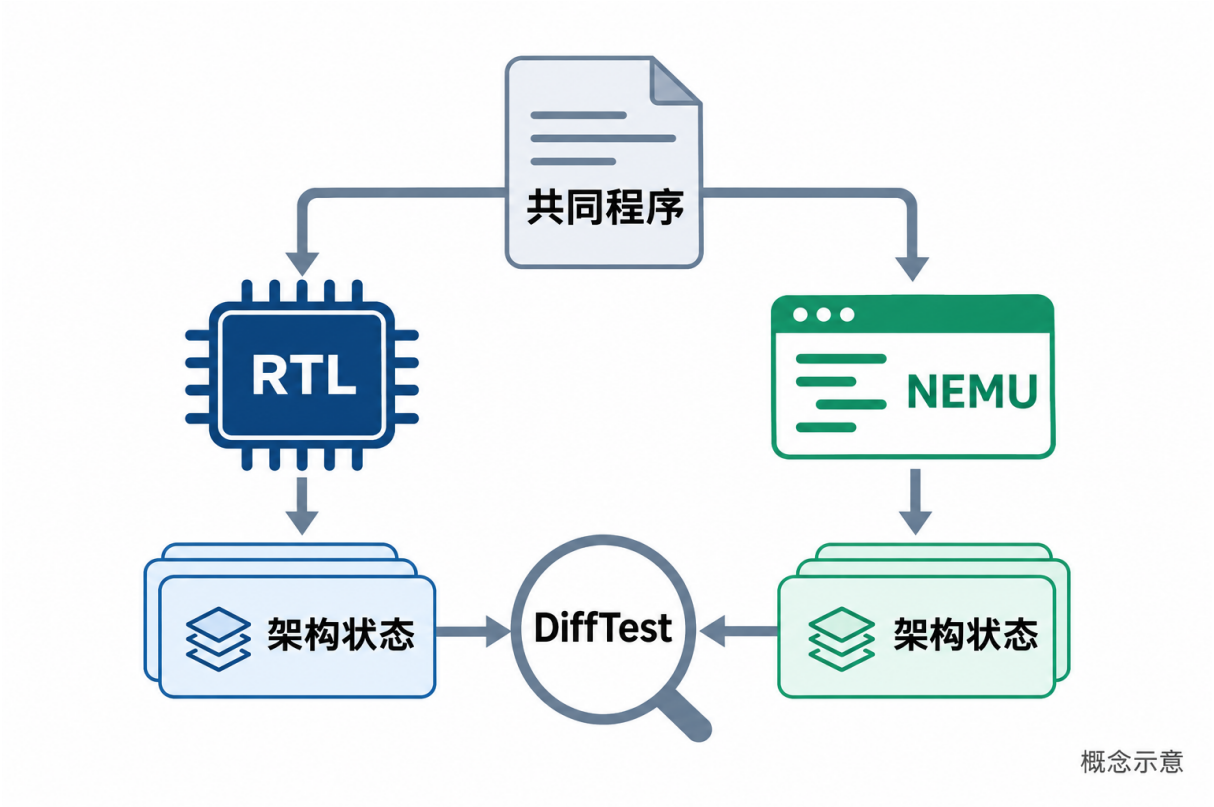
3.1 NEMU 扩展与协同仿真

3.1.1 独立参考模型与架构行为

VDot 的硬件实现和 NEMU 参考模型共享同一份指令语义。输入是两个向量寄存器低 64 位中的八对有符号字节，输出是八项乘积之和：低 32 位保存二补码结果，高 96 位清零。硬件通过并行乘法、归约和两槽控制产生响应，NEMU 则按软件顺序提取元素并求和。两种实现采用不同的计算组织，使比较同时覆盖算术结果和结果布局。

参考模型的取数过程从架构寄存器开始。每个字节先解释为 signed int8，再扩展到能够容纳乘积与总和的有符号类型；求和初值为零，不读取目的寄存器的旧值。读取两个源的快照后再写回目的寄存器，因此 vd 与 vs1、vs2 重合时仍使用同一条指令执行前的源数据。这样的步骤与软件可见语义直接对应，也便于检查 C 语言整数提升、符号扩展和移位处理是否符合预期。

参考侧使用完整 128 位向量值承载结果。对负数而言，符号只保存在低 32 位二补码中，高 96 位仍为零；这是本指令约定的结果格式。若把 32 位结果符号扩展到整个向量，低位算术可能正确，高位比较仍会立即暴露布局错误。硬件内部采用 32 位结果寄存器后，在公开输出处补零，正是利用了这一固定的架构属性。



概念示意：独立求值与架构状态比较。技术依据见图 3-1。

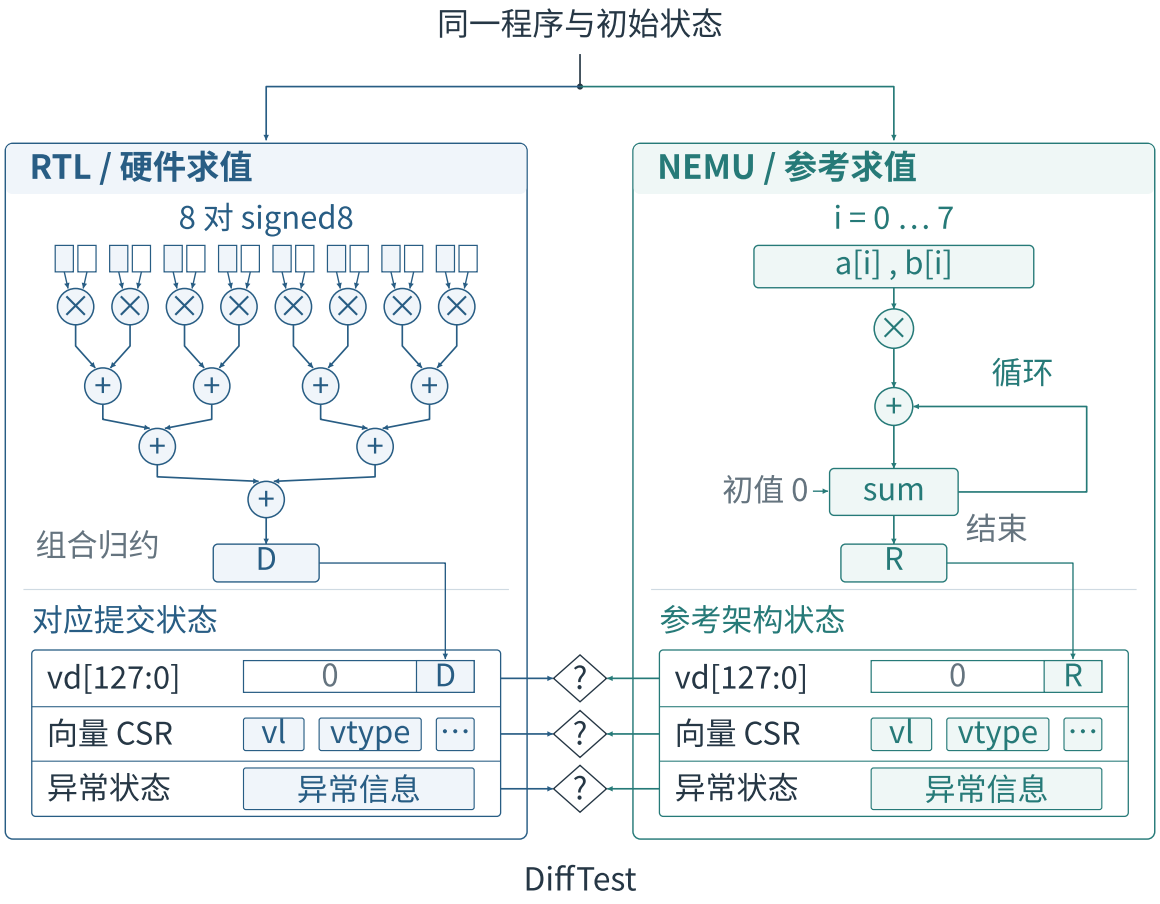


图 3-1 RTL 与 NEMU 独立模型的 DiffTest 框架

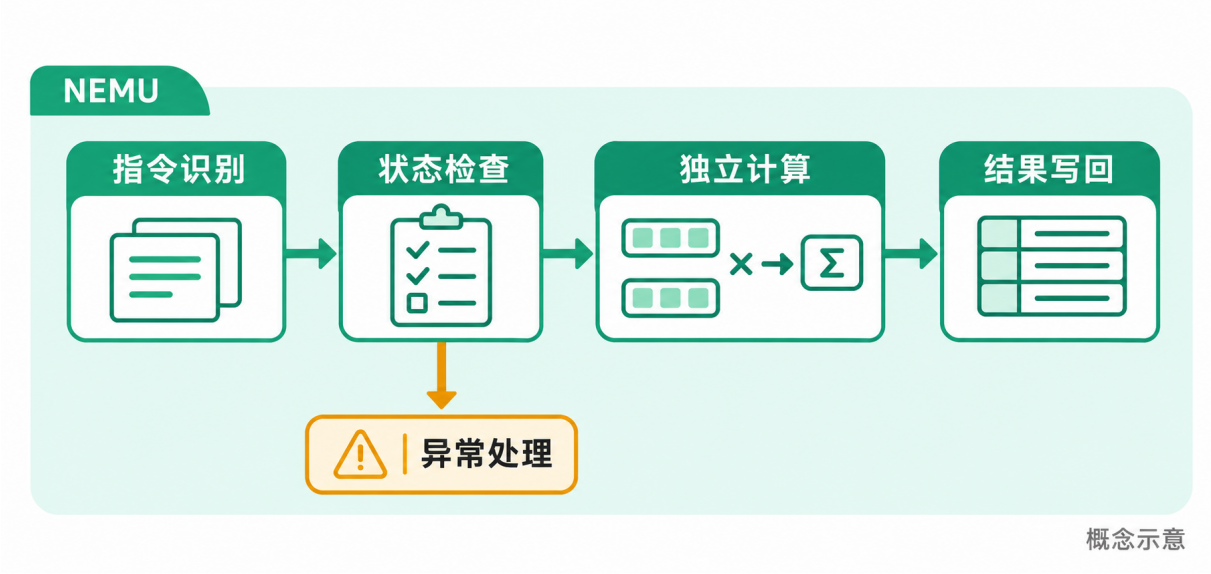
相同程序与初始状态分别进入 RTL 和 NEMU。左侧八路乘法和组合归约、右侧 C 循环仅表示独立求值结构，树层数不是流水级数，循环次数不是 DUT 周期。算术结果写入各自向量状态，CSR 与异常来自模型整体；DiffTest 按对应架构提交比较完整结果及状态。D、R 仅为两端结果标签，FU 响应不等于 ROB 提交。

3.1.2 指令译码与异常处理

NEMU 先按 custom-0 指令字匹配项目编码，再检查向量配置。正常路径使用 SEW8、LMUL1、VL8、vstart0 和 VLEN128；非法配置进入异常处理，目的向量保持原值。译码与执行检查共同决定指令能否生效，因此测试既覆盖操作码和功能字段，也覆盖同一合法机器字在不同向量状态下的行为。

香山固定版本中，译码阶段识别的非法指令与功能单元发现的 VL 非法具有不同的 xTVAL 表现。平台适配按对应的指令和架构状态描述这些条件，并比较异常码、陷阱 PC、xTVAL 及目的状态。报告中的特权执行记录来自 M 模式。该平台规则集中保存在 reference 配置中，正常算术路径仍由独立求和得到结果。

异常测试中的初始状态与正常测试同样重要。将目的向量设置为可辨认的非零值，能观察非法操作是否错误清零或部分更新目的；源寄存器和 CSR 快照则用于核对异常前后的状态变化。把这些观察与异常 PC 对应起来，可以区分译码错误、执行阶段非法配置和异常后的状态恢复问题。



概念示意：识别、检查与执行。技术依据见图 3-2。

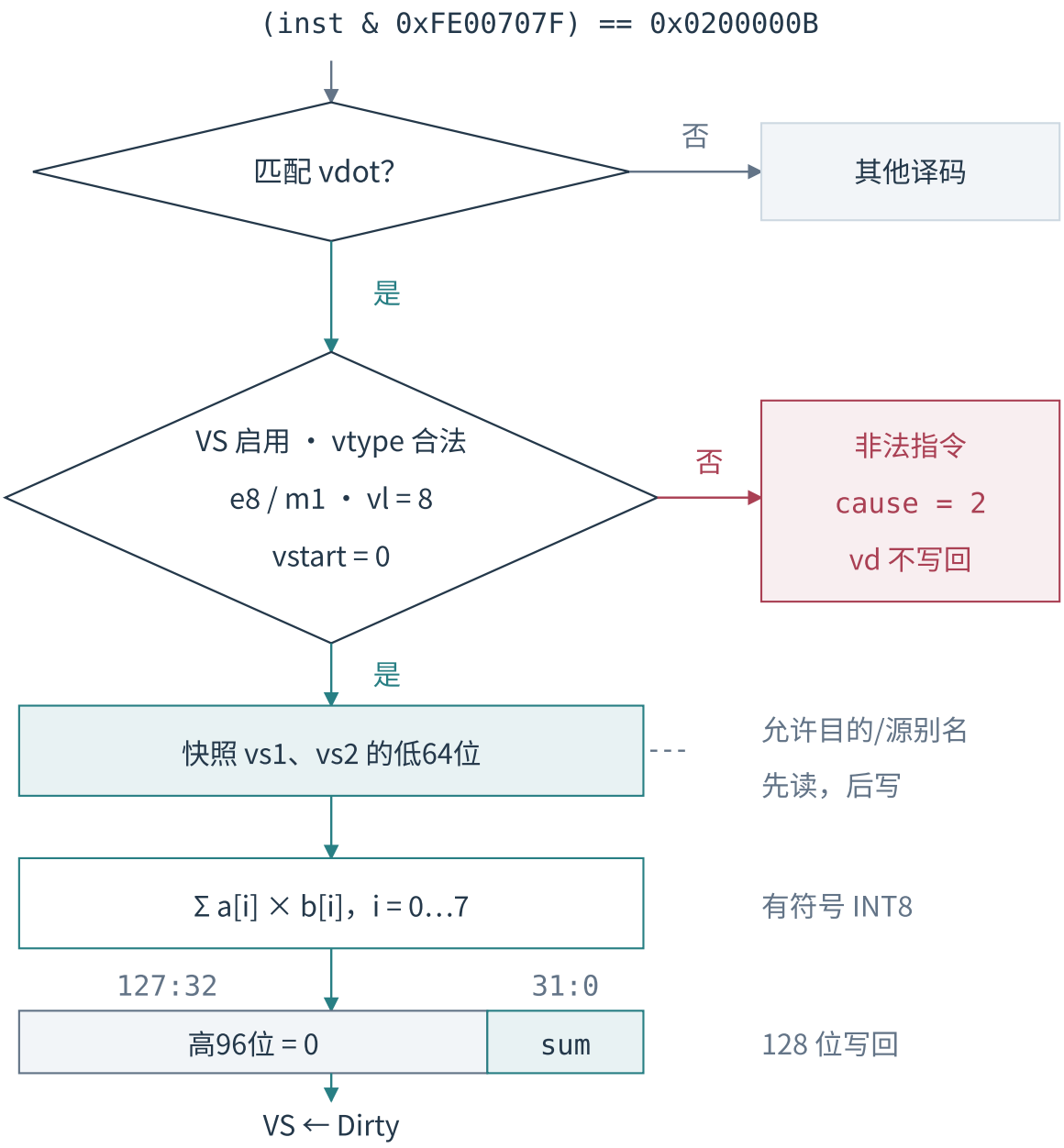


图 3-2 NEMU 中 vdot 的识别、状态检查与独立执行

精确匹配 vdot 编码后，所有向量状态条件均须满足；vtype 合法包括 vill 与保留位检查。合法路径先快照两源，再求八项有符号 INT8 乘积之和，完整写回 128 位（高 96 位为零、低 32 位为和的补码）并将 VS 置脏。其他译码与状态非法分支分别处理。NEMU a48a11f、补丁 1.1.0；xTVAL 沿用固定平台策略，已验证范围为 M 模式。

3.1.3 提交边界上的差分比较

DiffTest 在架构提交边界比较 DUT 与参考模型。前端取指、功能单元响应和 ROB 提交分别代表不同的进度，比较器以最终提交的指令与架构状态建立对应关系。对 VDot 而言，关键内容包括机器指令、目的寄存器、完整向量结果及向量 CSR；对异常指令，还包括陷阱入口和相关异常字段。

运行记录按启动、指令执行、差分检查、程序结束四个阶段组织。正常用例同时检

查计划中的测试数量、完整结果、GOOD TRAP 和进程退出状态。这样可以把程序自行报告的功能结果与外部差分检查联系起来：前者核对测试期望，后者核对 DUT 提交状态与独立模型的一致性。保留 stdout、stderr 和退出码，也便于定位构建、装载和目标程序执行之间的故障位置。

不同层次的测试采用各自的计数单位。主机模型以输入向量和测试方法计数，局部 RTL 以事务与控制场景计数，全核以程序、用例和窗口计数。R5 全核功能矩阵的 117 个不同用例与 124 次分片执行分别描述用例覆盖和实际执行安排；新控制诊断中的 131072 字节则描述被核对的寄存器结果量。把这些单位列清楚，能够准确反映每一层验证所完成的工作。

点积测试检查整个目的向量，同时记录其他向量寄存器与 CSR。VDot 路径仅更新约定的目的寄存器；RVV 等价路径需要中间向量寄存器，检查器据预先声明的 scratch 集合判断这些变化。由此既能检查计算结果，又能区分允许的临时状态与意外破坏。

源与目的别名是测试的重要部分。先读取两个源、再清理或覆盖目的，能够保证 vd 与任一源重合时仍计算原来的八对输入。RVV 序列也遵循相同的数据顺序：先生成宽化乘积，再清零目的并归约。若先清目的寄存器，别名场景中的源数据会被破坏，即使无别名样例全部通过也会留下语义缺口。

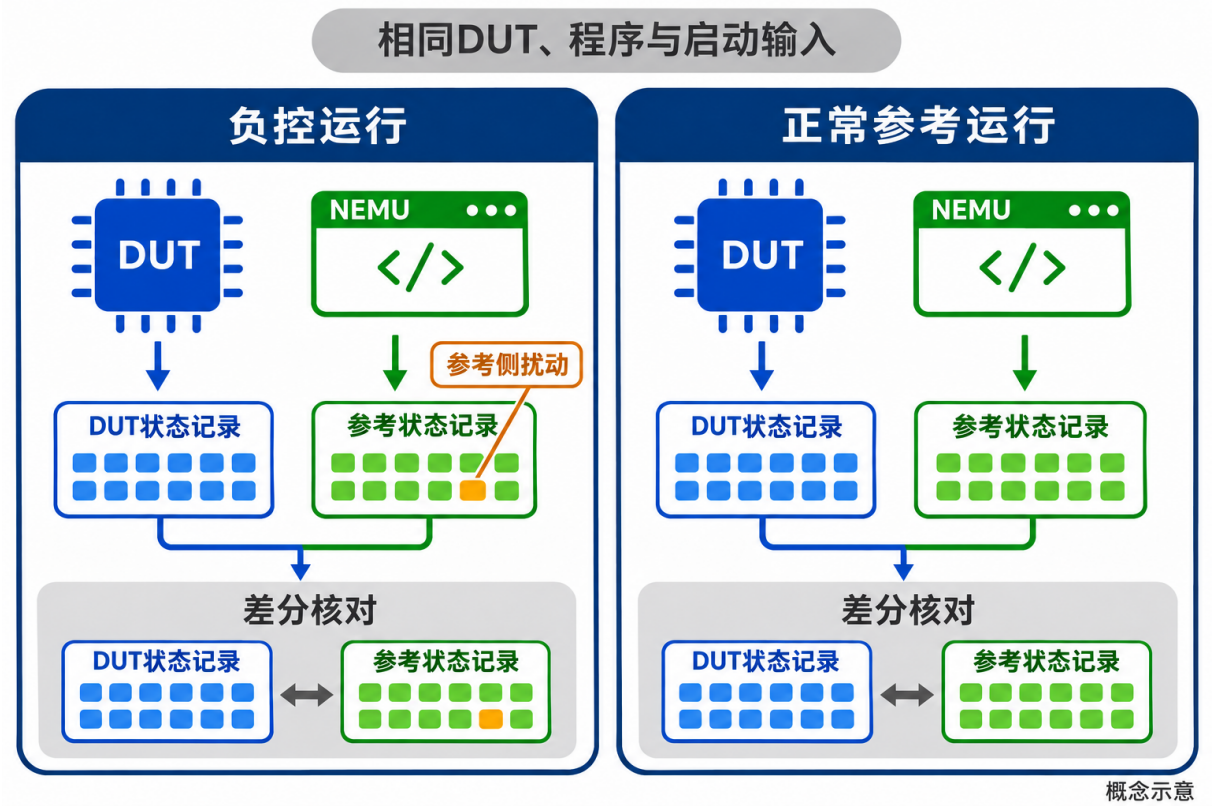
访存测试还检查输出槽和前后护栏。每个结果使用 16 字节槽保存完整向量，连续结果之间的步长与实际存储宽度一致。逐槽比较能够发现结果正确但写入地址错误、前一槽覆盖后一槽或高位残留等问题。输入高半区的非零哨兵则检查实现是否意外使用了规定输入范围之外的数据。

3.1.4 高位注错与正常参考对照

bit100 负控针对向量结果的高位比较。合法 VDot 结果的高 96 位均为零，选择完整向量的 bit100 注错，相当于改变高 64 位中的 bit36。低 32 位算术结果可以保持一致，而高位差异应被 DiffTest 检出，因此这项测试直接检查完整向量比较是否生效。

图中的两组记录来自 2026 年 10 月 24 日 core-01 实验。负控运行加载故障注入参考库，在 PC 0x80001bdc、指令字 0x0294050b、目的寄存器 v10 处出现首次失配，以 exit 1 结束；正常对照使用同一测试镜像和正常 reference，完成 15 个用例并以 GOOD TRAP、exit 0 结束。两次独立运行的镜像一致，使参考库成为对照中的关键变化项。

首次架构失配 PC 定位状态差异的起点，终止事件记录模拟器的退出位置。机器字、寄存器和高位差异共同对应注错目标；正常对照确认同一镜像、启动流程和数据能够完成整组测试。



概念示意：扰动检查与正常参考对照。技术依据见图 3-3。

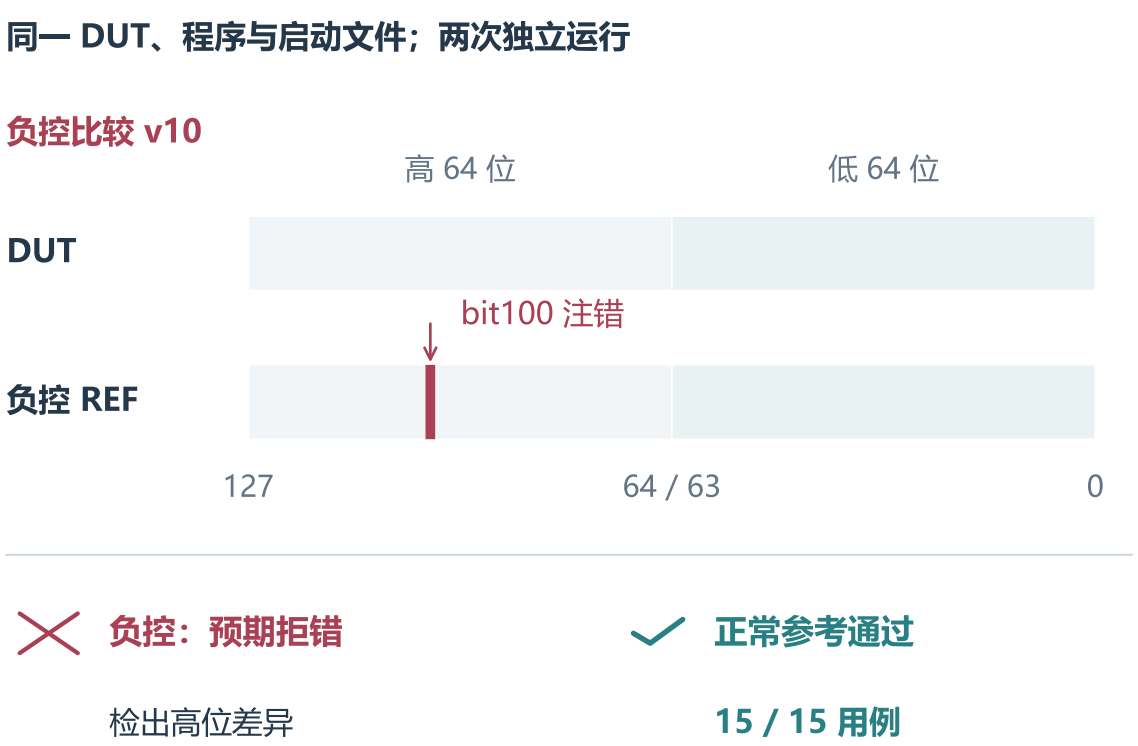


图 3-3　bit100 负控与正常参考对照

负控在 v10 的 bit100 检出差异，首次失配 PC 为 0x80001bdc；正常参考对照完成 15 个用例并以 GOOD TRAP、exit 0 结束。

3.2 RVV 对照与周期测量方法

3.2.1 等价 RVV 指令序列

两条路径均将低 8 个 signed int8 元素分别相乘并求和，目的向量低 32 位保存二补码结果，高 96 位清零。检查覆盖完整 128 位及寄存器副作用：vdot 保持其他向量寄存器，RVV 的中间操作使用预先声明的 scratch 集合。

表 3-1 等价 RVV 指令序列

RVV 路径	点积主体指令数	向量 scratch	入口与出口
已知配置单次	5	v2、v3	e8/m1/VL8、tu/mu；退出恢复相同配置
通用配置单次	7	v2、v3，另用 t3/t4 保存 CSR	保留原 VL/vtype，测试四种 tail/mask policy
独立四路批处理	14 / 4 个点积	v16–v23	e8/m1/VL8、tu/mu；退出恢复相同配置

已知配置序列：vwmul.vv → 设置 e16/m1/VL8 → vmv.v.i vd,0 → vwredsum.vs → 恢复 e8/m1/VL8。e16 下 8 个元素正好清零整个目的寄存器；宽化归约使用 tu，保留已经清零的高位。序列先乘后清目的寄存器，从而支持源与目的别名。通用序列增加 csrr vl/vtype，末尾用 vsetvl 恢复。四路批处理共享两次 vtype 设置，充分利用批量调用的配置复用。

每次调用保存并比较全部 32 个向量寄存器，按声明排除 RVV scratch；检查 VL、vtype、vstart、vxrm、vxsat、vcsr、实际循环次数以及缓冲区前后 canary。访存组逐个比较全部 128 位输出，并检查未使用输出空间。C 判定器使用真实源文件进行原生及 UBSan 位翻转测试。

3.2.2 计数窗口与实际执行边界

固定版本的 DecodeUnit.scala 为普通 CSR 设置 waitForward/blockBackward，而 Rename.scala 会解除大部分只读 CSR 的这些限制。初始观察中，cycle 与 instret 指令在 ROB 仍有旧指令时进入，相关标志为零。计时实验因此通过实际分派、采样和退休记录确定窗口顺序。

软件测量边界加入 fence rw,rw。该香山版本的 fence 通过等待旧指令、约束后续分派和 flush 形成边界，日志记录其与 CSR 及主体指令的执行关系。窗口定义依赖这套固定微架构实现，生产核的执行规则保持原样。

窗口结构如下（所有实验分支相同）：

fence; START; fence
read instret_begin; fence
read cycle_begin; fence
执行显式汇编主体（含循环控制）
fence; read cycle_end; fence
read instret_end; fence
STOP; fence
保存结果；完整状态检查；打印

观察器由 VDOT_T13_TRACE=1 启用，记录 CSR 实际采样、计数器入 ROB 及窗口内退休信息，并与 T12 观察器一起构建。它提供只读事件观察，数据通路、复位、忙闲接口和 NEMU 计时维持原实现。

窗口检查包含以下六项：

1. 精确匹配四条计数器指令的 PC、机器码、ROB 身份与顺序。

2. CSR 读值与对应退休写回数据相等，采样位于入队和退休之间。

3. 每条计数器的前一 fence 已退休，后一 fence 在计数器退休后才退休。

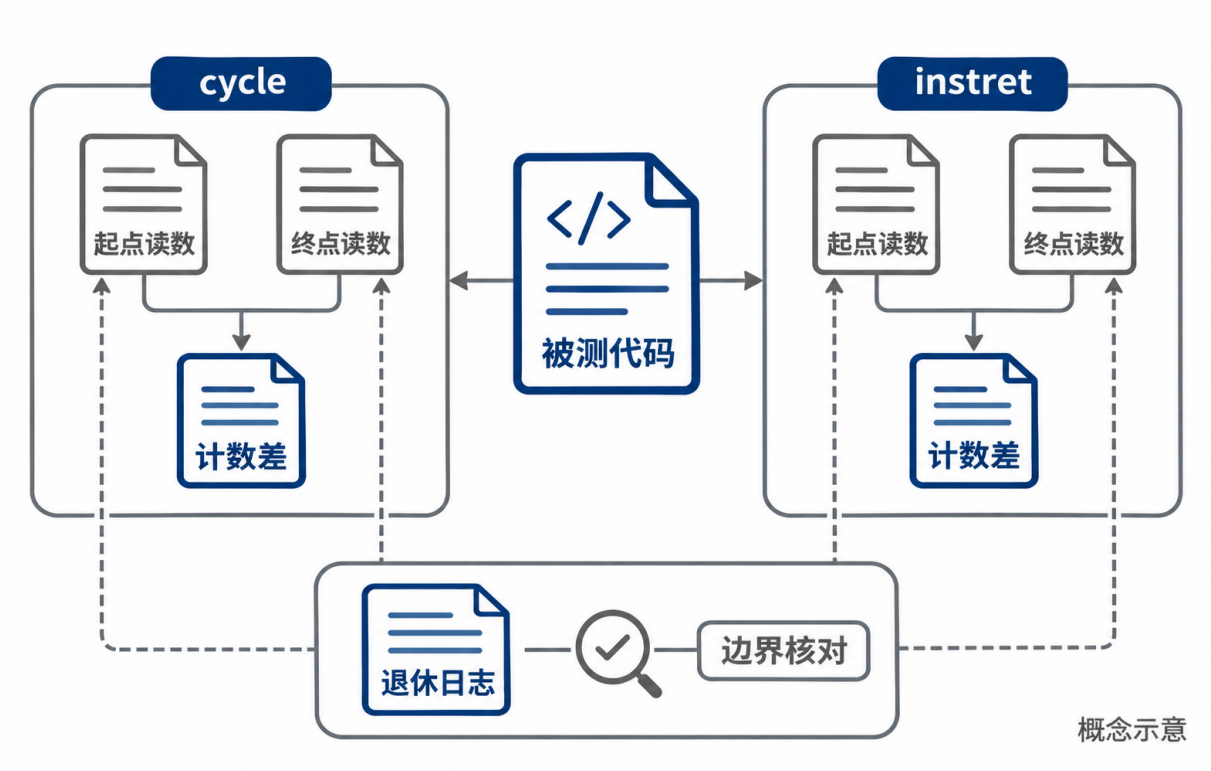
4. cycle 差值等于硬件全核采样 tick 差值。

5. instret 差值等于真实退休计数，也等于从 ELF 机器码展开有限循环得到的指令数。

6. 完整窗口及 UART 记录数量、顺序与 manifest 一致；真实 exit0、DiffTest 启用、完整成功 summary 后 GOOD TRAP。

绝对计数器可能大于 2^53，解析和差值计算全程使用 Python 整数。FU 子时钟受门控控制，局部事件先通过 T12 记录的父级 VFEX4 accept 逐事务关联，再与 ROB 事件比较；全核窗口周期使用统一的全局计数。

校准场景包括空窗口、8 条整数指令和 32 条依赖除法。正式结果保留完整的屏障、取指和循环开销；空窗口用于观察固定成本，小工作量场景则结合这些成本解释周期。



概念示意：测量窗口与执行边界。技术依据见图 3-4。

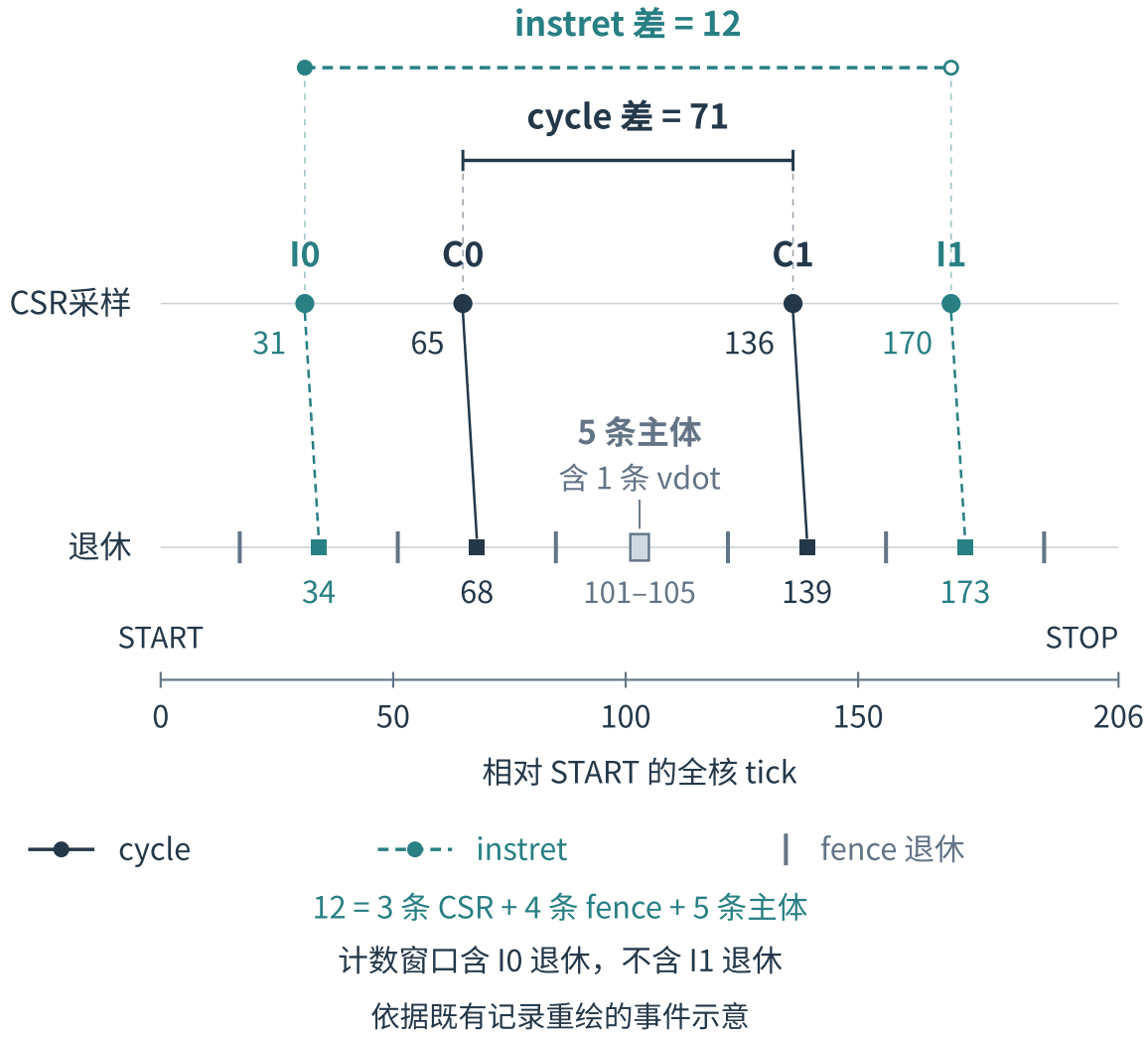


图 3-4　计时窗口 CSR 与退休边界

四条 CSR 分别记录 cycle 与 instret 起止值；图中标出采样和退休位置。固定 0f 的 after/dataset0/replay0/seq4 正式样本 repeat=2，cycle 差为 71、instret 差为 12。周期含隔离与循环开销，不扣空窗口；START/STOP 标记跨度为 206 个全核 tick。

3.2.3　工作负载、输入与重复测量

表 3-2　工作负载、输入与重复测量

场景	n	点积数	含义
single	1	1	已知配置完整调用窗口
chain	16、64、256	n	目的向量作为下一次源，观察依赖限制

表 3-2 工作负载、输入与重复测量（续）

场景	n	点积数	含义
batch	1、8、32	4n	每轮四个独立目的，比较共享配置的 RVV 批处理
stream	1、16、64	n	两个 128 位加载、点积、128 位存储以及指针更新
generic	1、16、64	n	原 vtype=192 的 CSR 保留对照

链式实验将上一结果的完整向量作为下一次源，数值可能逐步趋零，寄存器依赖关系则持续存在。批处理使用四个独立目的并共享输入和配置，用于观察独立请求吞吐。两者分别突出反馈等待与连续接收，对应不同的软件组织方式。

三组输入包括正极值、负极值和混合符号；高半区设置非零哨兵，访存组加入固定种子的随机数据。功能矩阵使用原始 22 组定向和前 32 组随机数据，轮换七种别名与四种 policy；54 对数据覆盖选定的别名和策略组合。另用固定 NEMU 对全部 10,022 组数据重新检查精简基线，明确与全核执行层分开计数。

每个配置在同一镜像内运行 6 轮配对，按执行轮次保留首次样本、预热和 4 轮正式样本；顺序按 AB/BA 交替，缓存状态随实际运行演进。每种配置单独分片，两个实现的入口均 64 字节对齐；代码长度、地址和寄存器压力的自然差异保留并记录。

每个性能镜像以 seed=1 在新的 EMU 进程中运行三次，按相同 seq 比较周期和退休指令数，完成确定性重放核查。重放记录与正式统计样本分别保存。原测量批次按最多 8 个宿主进程安排，各 DUT 均采用相同单核和 EMU_THREADS=1 配置；加速比使用 DUT 周期，宿主秒数用于记录执行成本。

3.2.4 加速比与统计方法

首次、预热和正式窗口完整保留。每个配置与数据集分别汇总原始周期、周期/点积、退休指令数及正式样本 min/median/max；配对加速比为 RVV cycles / vdot cycles，周期减少比例为 1 - vdot / RVV。配对保持场景、n 和数据集一致，同时列出加速和退化结果。

结果分为局部 FU 响应、完整序列窗口、独立批量吞吐和包含访存的循环四个层次。历史单在途 FU 的 E3/II4 描述局部接口行为；完整序列窗口则包含相同的隔离与循环开销。将各层指标对应到实际工作量，可以分别解释硬件执行和软件调用成本。

本节使用周期指标比较固定工作负载的执行效率。面积与静态时序由后续成本实验

独立测量，C8 软件收益则来自固定 GEMV 调用矩阵。三类结果分别呈现硬件周期、物理实现成本和应用调用效果。

正式加速比使用两种方法的总周期之比。设第 i 个配对窗口的 RVV 周期为 A_i、VDot 周期为 B_i，则汇总结果为 $\frac{\sum_i A_i}{\sum_i B_i}$。它等于各窗口加速比 $\frac{A_i}{B_i}$ 按 B_i 加权后的平均值。这样的定义对应完成整组固定工作量时两种实现各自付出的总周期，较长的窗口按其实际执行成本获得相应权重。

逐窗加速比的简单平均回答的是另一种问题：每个窗口无论长短均占相同权重。两者在样本周期不同的情况下可能给出不同数值，因此本报告沿用原测量合同的总周期比，并在样本文件中保留每一对原值。三组数据、每组四次正式窗口的数量一致，配对身份则保证统计中的工作量一一对应。

对于 0f 修复前后比较，96 对正式窗口的每对差值均为零。这比两组均值在显示精度内相同提供了更具体的信息：相同镜像和序号的窗口逐项保持周期，同时退休数也一致。面积与时序发生变化时，仍可据此把所观察到的代价定位到综合映射与路径，而不是该组软件窗口的执行周期。

3.3 点积与矩阵向量计算性能

3.3.1 四类点积微基准

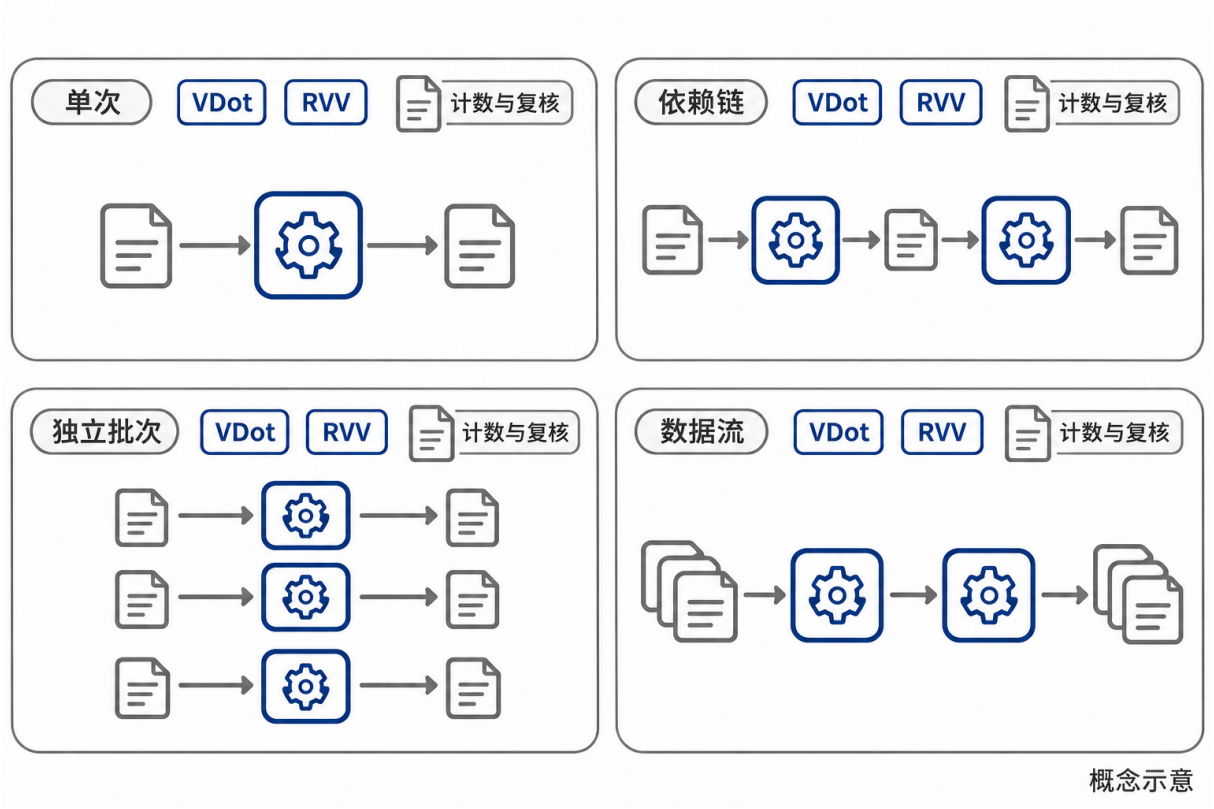
下表来自 0f 同镜像对照合同，均为正式窗口均值。

表 3-3 四类点积微基准

场景	vdot cycles	RVV cycles	加速比
单次点积	71.000	85.250	1.200704×
依赖链 256	1857.000	5440.500	2.929725×
四路独立 batch128	221.000	678.000	3.067873×
加载/计算/存储 stream64	1127.500	1133.000	1.004878×

独立批量可以利用 O2 的连续接收能力，依赖链受到前后数据依赖约束，stream 场景还承担访存及循环开销。结果显示收益随工作负载改变。

O1→O2 的独立同镜像比较中，batch128 由 362.5 降至 221 周期，减少 39.0345%。H1 及 0f 修复分别保持各自 96 对正式窗口周期相同。阶段间比较采用各自冻结合同。



概念示意：统一条件下的性能比较。技术依据见图 3-5。

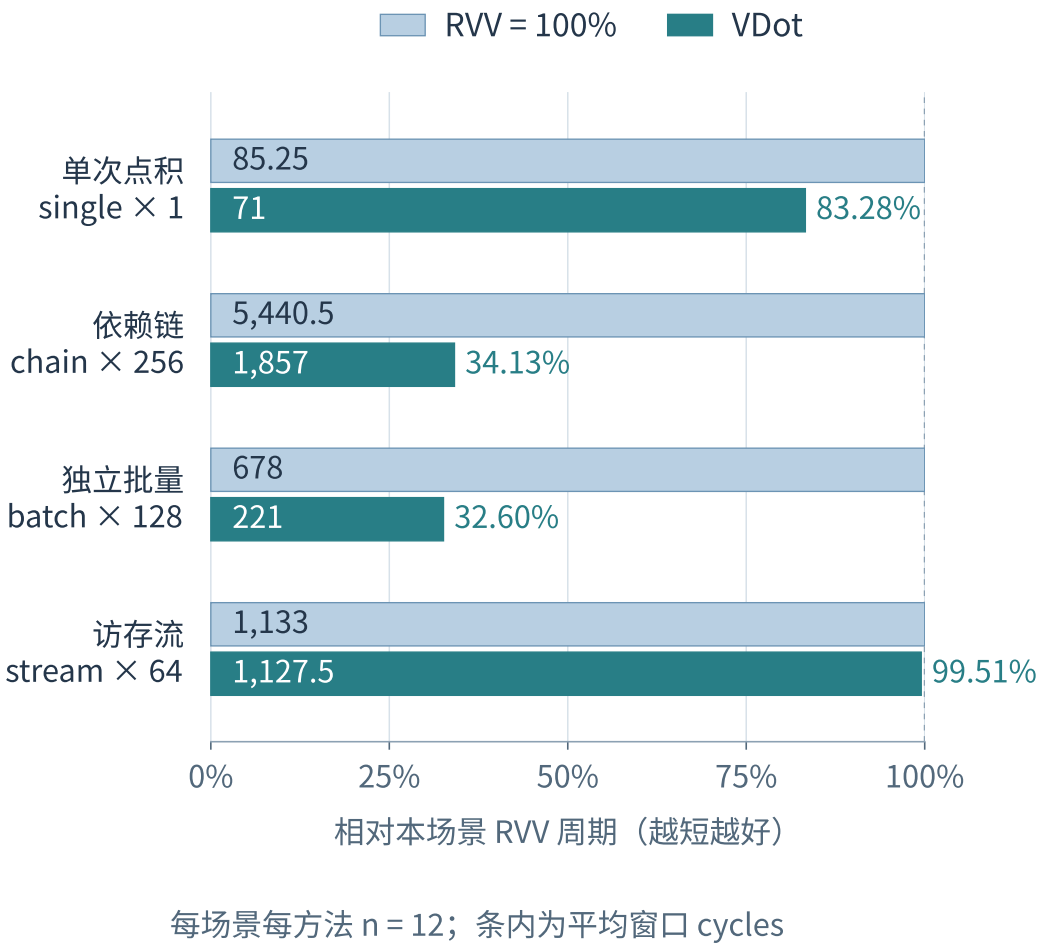


图 3-5　四类微基准的相对周期对照

取固定 after 记录，各场景、各方法均为 12 个正式样本。各场景按自身 RVV 平均窗口周期归一化为 100%，条内为实际平均窗口 cycles，条外为 VDot 周期占比，越短越好。single、chain、batch、stream 每窗口分别包含 1、256、128、64 次点积；batch 为四路独立任务。

3.3.2　单次调用与独立批量

单次点积中，VDot 为 71 周期，RVV 为 85.25 周期，加速比约 1.201。该窗口的点积工作量较小，配置、循环和测量边界占有明显比例。VDot 把多条向量算术操作收敛为一条自定义操作，节省的指令工作与共同固定开销一起形成了这一结果。

四路独立 batch128 的 VDot 均值为 221 周期，RVV 为 678 周期，加速比约 3.068。四个轮换目的为连续请求提供独立性，P 与 R 两槽可以在同一边沿完成旧 R 响应、旧 P 迁移和新请求接收。窗口中的 128 次点积与循环控制共同构成总工作量，持续接收能力在这一场景中得到更充分的利用。

O1 到 O2 的同镜像比较进一步隔离了流水组织的影响：batch128 由 362.5 降到 221 周期。O2 保留两槽状态并实现弹性推进，使前后事务能够重叠；当下游 ready 持续有效时，独立请求可逐拍接收。这个机制与实测周期变化相互对应，说明了优化为何对独立批量有效。

3.3.3 依赖链与流式循环

chain256 的 VDot 为 1857 周期，RVV 为 5440.5 周期，加速比约 2.930。上一条结果写回后，下一条才能获得所需源数据，因而该场景受到完整反馈路径的约束。发射等待、寄存器读取、功能单元执行和写回共同决定下一次请求何时可发出，局部两拍响应只是其中一段。

stream64 的 VDot 为 1127.5 周期，RVV 为 1133 周期，加速比约 1.005。两者差异较小，说明在这个加载、计算、存储和循环控制共同组成的窗口里，点积主体的优化只转化为有限的整窗收益。现有记录给出总体周期；进一步归因时，可以沿访存等待、数据依赖和单元占用逐项增加观察。

这四类结果共同描述 VDot 的使用特点：单次调用体现固定开销，独立批量体现连续接收，依赖链体现结果反馈，stream 体现数据搬运与计算的组合。应用选择可以据此关注自身工作组成，例如能否把多个点积组织为独立批量、配置是否能够跨调用共享，以及结果是否立即成为下一次输入。

3.3.4 每次点积的平均成本

在工作量明确时，总周期还可以换算为每次点积的窗口平均成本。single、chain256、batch128 和 stream64 分别完成 1、256、128 和 64 次点积，因而可以直接用对应均值除以点积数。这个指标把不同长度的循环放到相同工作单位下，便于观察批量组织和依赖关系的影响。

表 3-4 每次点积的平均成本

场景	点积次数	VDot 窗口周期/点积	RVV 窗口周期/点积
single	1	71.0000	85.2500
chain256	256	7.2539	21.2520
batch128	128	1.7266	5.2969
stream64	64	17.6172	17.7031

表中数值由前述窗口均值计算，保留四位小数。batch128 的 1.7266 周期/点积包含循环和计时边界，与功能单元逐拍接收的 II=1 共同描述系统的两个层次：接口可以每拍接受独立请求，软件窗口仍需要完成发射、循环及前后准备。两者之间的差异为进一步分析提供入口，也说明提高功能单元吞吐后还需考虑整条执行路径。

chain 与 batch 使用不同的依赖组织。chain 每次等待上一结果，batch 让四个目的独立，因此归一化后的差距体现完整反馈与连续工作方式的差异。stream 额外搬运输入和输出，其每次点积成本也包含访存与寻址。对实际软件进行选择时，先判断工作属于哪一种组织方式，再使用相应指标作为参考。

3.3.5 GEMV 调用与配置复用收益

firstdot 以首段点积初始化累加状态，后续段按约定求和。C8 在已验证 8×32 路径中复用配置，提供普通 C 显式入口和 RVV 回退。输入为对齐的 288 字节 A||x，输出为 8 个 16 字节完整槽。

表 3-5 GEMV 调用与配置复用收益

同合同 8×32 实现	完整调用 cycles
firstdot	282.25
原 C8 配置复用 vdot	273.00
本批最快合格 RVV	299.00

C8 相对 firstdot 减少 3.2772%，相对同批最快 RVV 加速 1.095238 倍。三数据集完整矩阵给出周期比较，普通 C 调用、软件包与最终镜像检查验证了 ABI、回退和接入方式。

原合格镜像、复制对照、SDK 接入版 BIN 均为 31440 字节，SHA 相同。PT_LOAD、48 条调用表项与 12 组输入数据一致，ELF 差异位于调试节。627 项输入与封存材料的身份核对，将 SDK 接入结果与原功能记录对应起来。

4×8 和 16×64 采用原 firstdot 入口，4×13 选择更快的 RVV。各尺寸沿用对应的已验证调用入口。

3.3.6 硬件与软件收益的对应关系

O2、H1 和 0f 分别解决不同问题。O2 调整两槽弹性控制，提高独立请求的重叠程度；H1 利用高 96 位恒零的语义，将结果状态压缩为低 32 位保存；0f 处理 DIV 取消后的关钟清理。把这三类改动分开，有助于分别观察吞吐、状态成本和父级控制正确性。

0f 修复前后的 192 个正式窗口组成 96 对，逐对周期差均为零，退休指令与工作量一致。四次额外 batch 重放也与对应记录相同。因此该修复在这些微基准中保持原有性能，同时解决取消后持续产生无效子时钟脉冲的问题。其面积和时序影响另由完整 VFEX4 父级综合结果评估。

firstdot 与 C8 属于软件调用层的优化。它们使用既定 VDot 语义，通过累加初始化、配置复用和调用组织减少软件开销。软件收益来自完整调用窗口的比较，硬件收益来自相应硬件版本的同镜像比较；这两条分析线最后汇合到选定的工程组合 0f ＋ firstdot V2 ＋原 C8。

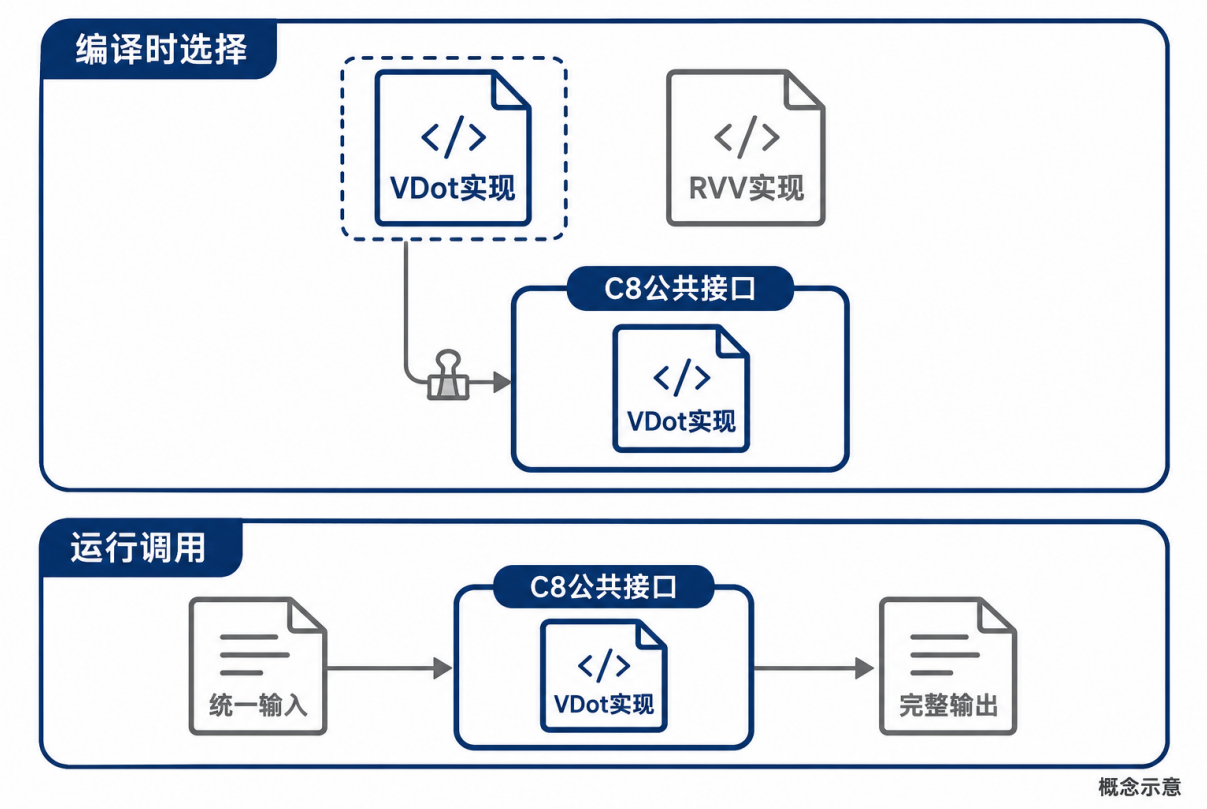
3.4 普通 C 接口与软件包接入

3.4.1 firstdot 与 C8 的调用组织

firstdot 用首段点积初始化累加值，后续段按约定归约，避免在首段之前额外构造等价的初始化工作。C8 针对 8×32 路径复用向量配置，把已验证的汇编函数接到普通 C 入口。两者均保留完整输出格式和相应的 RVV 对照。

在同批 8×32 完整调用窗口中，firstdot 为 282.25 周期，C8 为 273 周期，减少 3.2772%；最快合格 RVV 为 299 周期，C8 相对其为 1.095238 倍。这个比较同时包含函数执行和该合同规定的调用成本，反映了配置复用在固定 GEMV 尺寸上的效果。

不同尺寸采用各自的已验证入口。4×8 和 16×64 保持原 firstdot 路径，4×13 选用较快的 RVV，8×32 使用原 C8。这样的选择利用了实测差异：自定义点积与配置复用对某些形状有效，而较小或不规则的工作量可能更适合现有 RVV 序列。最终软件入口据此保留明确的选择与回退方式。



概念示意：公共接口与缓冲区组织。技术依据见图 3-6。

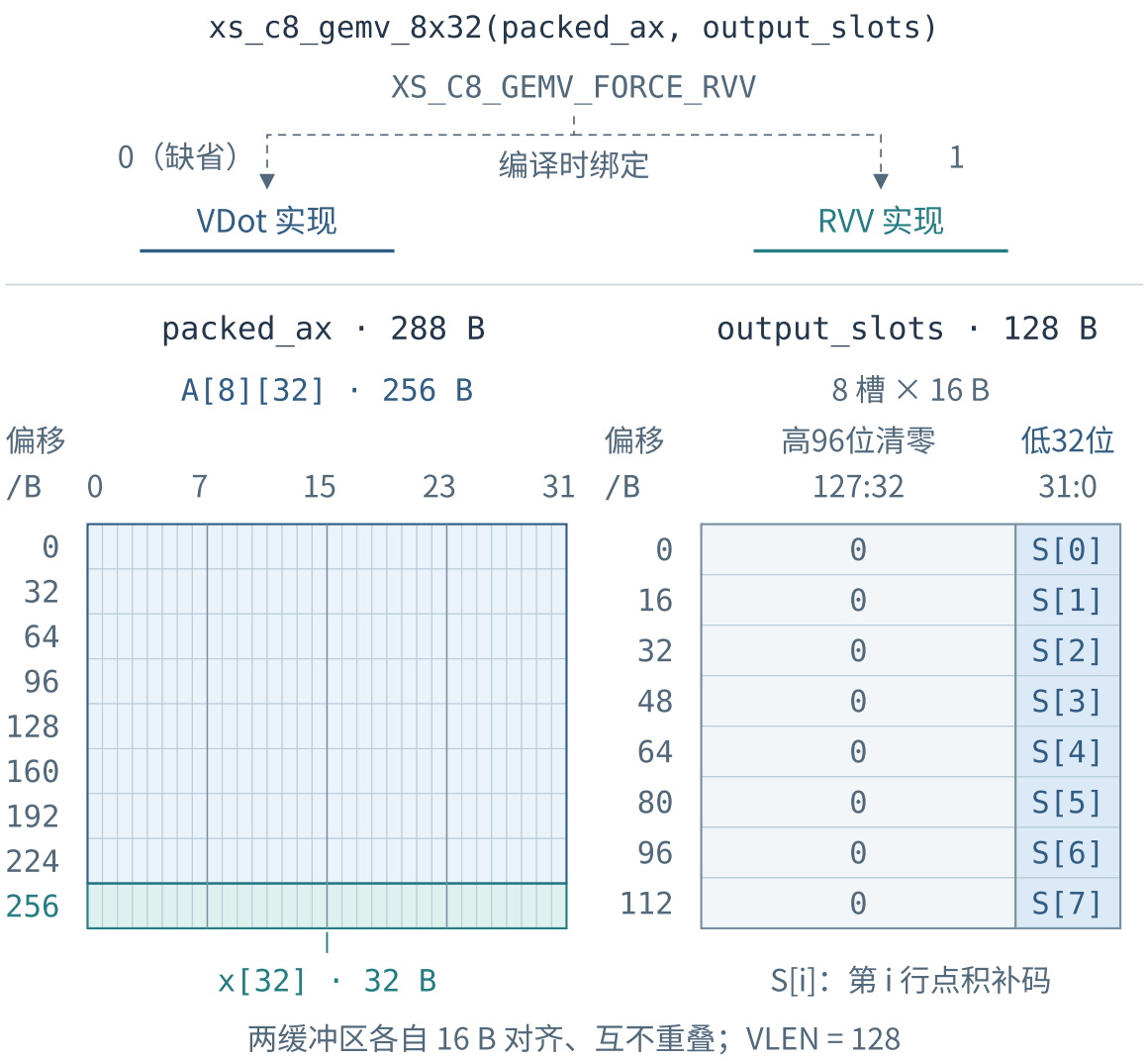


图 3-6　C8 公共调用、编译选择与输入输出布局

xs_c8_gemv_8x32 在编译时绑定 VDot 或 RVV 实现。packed_ax 包含行优先 A[8][32] 的 256 字节及其后的 x[32]，共 288 字节；output_slots 包含 8 个 16 字节槽，每槽高 96 位清零、低 32 位为点积补码。两缓冲区各自 16 字节对齐且互不重叠。横向结果位域为高位到低位示意，不表示字节端序；VLEN=128 及向量状态要求见正文。

3.4.2　输入输出布局与调用约定

函数原型为 void entry(const int8_t *packed_ax, void *output_slots)。已验证运行环境为当前香山昆明湖 V2/Nexus-AM 裸机 RV64、lp64d，**VLEN 恰好 128（vlenb=16）**，VS 已启用。默认路径要求该硬件实现项目的自定义 vdot 编码。调用环境须启用 VS，并确认目标硬件支持项目自定义指令；本次验证采用 Nexus-AM 裸机环境。

- 输入：有符号 INT8 行优先 A[8][32]，紧接 x[32]，共 288 字节。
- 输出：8 个 16 字节槽，共 128 字节。每槽低 32 位为 INT32 点积结果，高 96 位清零。槽步长为 16 字节，与完整向量存储一致。

- 两块缓冲区各自 16 字节对齐、有效且不重叠；输入不会被修改。调用者负责生命周期、数据打包及平台初始化。
- 沿用原 RV64 C ABI：栈及被调用者保存寄存器约束不变；向量寄存器和允许的调用者保存状态会改变。
- 返回状态为 vl=8、vtype=0（e8/m1/tu/mu）、vstart=0、vcsr=7，不保留原向量配置。

底层 vs1r.v 存储整个向量寄存器，输出行步长固定为 16 字节，因此接口要求 VLEN 恰好 128。VLEN256 的整寄存器存储会超出当前槽宽。强制 RVV 改变调用目标，完整静态库仍保留各函数，平台与缓冲区约定保持相同。

3.4.3 编译期选择与 RVV 回退

默认的 xs_c8_gemv_8x32 入口通过编译期别名连接配置复用 VDot 函数。设置 XS_C8_GEMV_FORCE_RVV=1 后连接相应 RVV 函数，显式 xs_c8_gemv_8x32_rvv 入口则始终选择 RVV。开关只接受字面量 0 或 1，错误配置在编译期被拒绝。

这一选择由头文件和匹配静态库共同形成。宏或 API 版本变化后，相关翻译单元一起重编，使调用点与库的选择一致。撤销 C8 时恢复原 firstdot 头文件和对应库即可回到原入口。编译期选择适用于当前明确的平台配置，调用方根据目标平台确认指令支持情况。

3.4.4 对象、装载镜像与调用表核对

比较对象依次为原合格 C8 镜像、原源码复制对照和 SDK 接入版。

表 3-6 对象、装载镜像与调用表核对

项目	结果
完整 BIN	三者全字节一致，均 31,440 字节
BIN SHA256	56805506eb98b68269c73a3645d8f75dff73b697e30dec5f93ccc4716089ad34
入口	均 0x80000000；ABI flags 及 ISA 属性一致
PT_LOAD	同地址、权限、对齐及完整内容；file30,024B，memory31,440B，零填充 1,416B
装载节	所有 SHF_ALLOC 节的位置、大小、权限、对齐及内容一致，包括 BSS
函数	16 个公共函数及启动/主程序等 5 个函数的地址、大小和机器码一致

表 3-6 对象、装载镜像与调用表核对（续）

项目	结果
测试表与数据	48 条 64 字节调用记录与原计划逐项匹配；12 组输入数组地址及内容一致
GNU 链接	无未解析符号；map 与实际子命令证明使用 SDK 库成员
完整 ELF 原字节	不同；差异仅在.debug_info、.debug_line、.debug_line_str
独立 GNU 复核	分别 objcopy --strip-debug 后，三份完整 ELF 全字节一致
去调试 ELF SHA256	7e26e295fb020a63334026f768ed4825372266465b4e6008973de966fefc04ab

调试节保存源文件/头文件路径及行号等信息，构建目录与头文件发生变化会改变这些记录。主检查器首先比较完整装载内容和调用计划，随后用去调试 ELF 作独立补充核对；三份原始 ELF 同时归档。

原 ELF SHA256：02159c7ef78cb31eb24cf706bf2974eadea313710563f235f2b11a7bb922de56。

复制对照 ELF SHA256：cfad14278ee4a3590de9bcbe7589b808fd91959156cef93ba92a78a90db0d99b。

SDK 接入 ELF SHA256：f0ddf1167c961101e70f58cfd012ae788d2965324ad057aa4ade5af1de5bc42e。

软件包接入按对象、链接和装载内容逐层核对。静态库只含完整 gemv.o，保留 16 个函数、原节和重定位；链接映射与 GNU 依赖文件确认调用程序实际使用 SDK 头和库成员。这样可以直接定位最终函数来自哪里，以及重定位和符号解析如何完成。

最终比较包含原合格镜像、原源码复制对照和 SDK 接入版。三者 BIN 均为 31440 字节且逐字节一致，入口为 0x80000000；PT_LOAD、BSS、函数地址与机器码、48 条调用表记录和 12 组输入数据也一致。装载内容相同，把接口改用 SDK 后的运行输入与原程序连接起来。

完整 ELF 的差异位于调试节。源文件路径、头文件位置和行号变化会改变这些信息，但装载段比较已经确认程序实际使用的字节相同。进一步用 GNU objcopy 去掉调试信息后，三份 ELF 也逐字节一致，形成与主检查不同路径的补充核对。

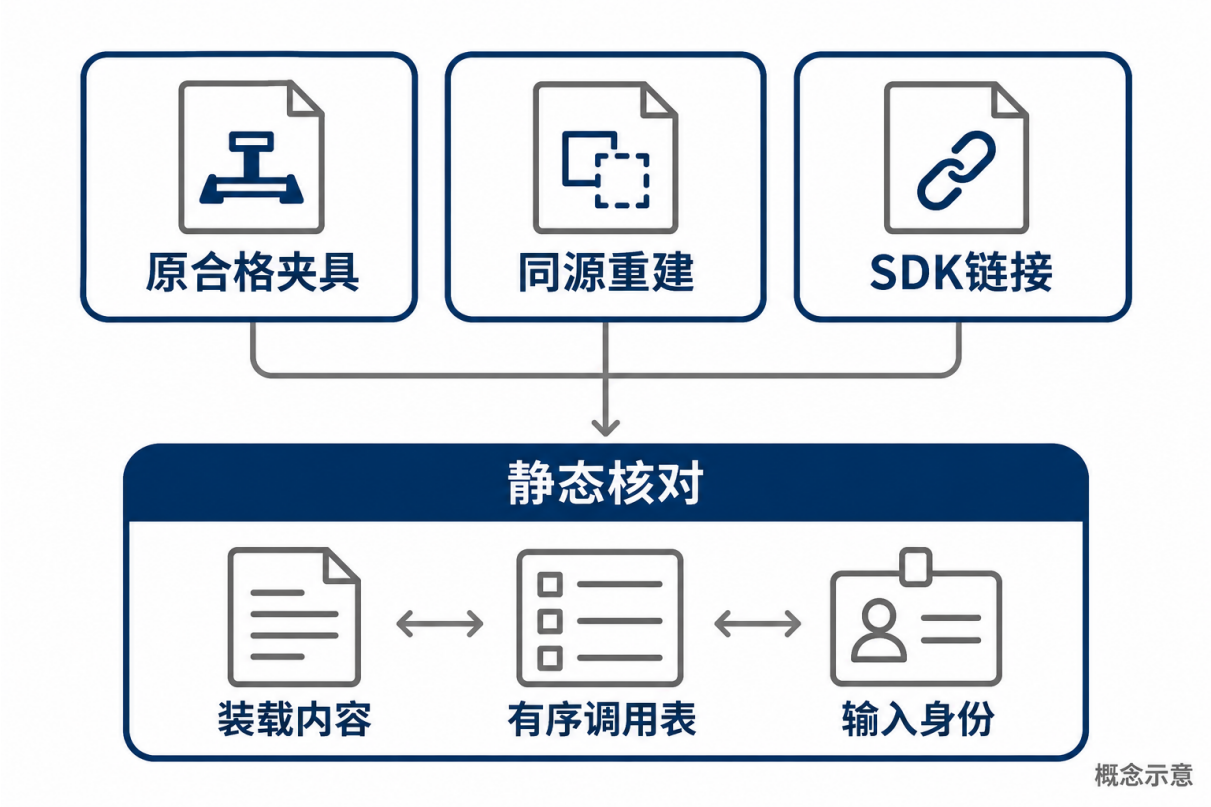
3.4.5 接入检查器的正负控制

7 项负控全部被拒绝：实际 ELF 代码字节改坏、调用表改坏、启动入口改变、BSS 大小改变、ISA 属性改变、输入计划改变，以及真实 GNU 链接错误空库。另有只改调试字节的正控，确认不会将纯调试变化判为行为改变。这些控制样例用于验证主机端镜像

检查器。

627 项输入及封存运行材料绑定 0f、EMU/NEMU 与 GNU 工具。原 33 个矩阵进程和 3 个 C 进程的记录关系及输出树保持，接入过程另行保存 GNU 构建和最终镜像等价回执，形成软件包与既有功能结果之间的连接。

成功流水线：2026-10-27T21:59:40.771873+08:00 至 2026-10-27T21:59:43.182911+08:00，exit0。两次 GNU 构建，8 份主流水线命令回执，另 3 份去调试 GNU 检查回执。流程首次通过，负控和原始文件全部保留。



概念示意：三种制品分别核对装载内容、有序调用表和输入身份。技术依据见图 3-7。

图 3-7 C8 三方静态装载内容与调用输入对照

原合格夹具、同源重建与 SDK 实际链接版的 BIN、PT_LOAD 和有序调用表、输入身份一致。PT_LOAD 按 30024 字节文件内容与 1416 字节零填充的真实比例绘制；零填充包含对齐尾部，不能全部称为 BSS。调用及输入网格表示逻辑记录，不表示物理地址排列。调试节差异位于非装载区；627 个受保护文件已核验。静态等价支持沿用既有夹具证据，SDK 接入未新增 EMU、NEMU 或全核运行。

3.4.6 接入结果与功能记录复用

C8 包完成对象、可重定位链接和接回原 C 程序的完整镜像等价检查。原 16 函数、48 例两轮 96 次正常调用的功能记录可据此复用，其中 8×32 占 12 例、24 次调用。

该接入实验采用静态构建与镜像比较，EMU/NEMU 运行次数为 0。273 cycles 与

1.095238 倍来自原第 25 条性能矩阵，描述原 C8 完整调用；这里的结果说明 SDK 接入保持了原测试程序的装载内容和调用方式。

软件包已完成从普通 C 入口到最终镜像的接入验证。面向新应用时，可固定新的输入布局、调用路径和最终镜像，使用同路径 firstdot/RVV 比较完整调用成本。现有其他尺寸继续采用原选择：4×8 与 16×64 使用 firstdot，4×13 使用 RVV。

3.5 面积、时序与实现成本

3.5.1 评估条件与主要结果

使用 Nangate45 Typical 1.10V/25°C，Yosys0.9、ABC1.01 及 OpenSTA2.0.17。主档周期 1ns、负载 5fF，采用固定线负载模型，另有 0.5ns、2ns 及 1fF、20fF 敏感性档。

表 3-7 评估条件与主要结果

指标	FU	完整 VFEX4 父级
标准单元面积 μm²	17,298.246	284,614.946
nominal r2r 到达 ns	1.163404	3.261218
主档 i2r setup 裕量 ns	−0.268520	−2.393998

父级面积包含 FU 及 DIV。负裕量表明 1ns 参考目标仍有时序违例，整核 Fmax、功耗和布线后时序为待测。H1 相对 O2 独立 FU 面积降低 2.33386%，父级降低 0.137349%，同时保留部分路径退化。0f 的取消关钟修复以正确性为依据，其周期不退化与成本变化分别留档。

3.5.2 寄存状态与综合面积

面积采用 Nangate45 Typical 库，在相同映射流程和约束下统计标准单元面积。VDot FU 为 17298.246 平方微米，完整 VFEX4 父级为 284614.946 平方微米。父级包含 FU、DIV 和共享控制，因此父级数值描述集成成本，FU 数值用于观察点积单元自身的结构变化。

0f 相对修复前的父级面积减少 264.936 平方微米，约 0.0930%，FU 面积不变。父级 DFF 数量增加一个，回溯到既有整数 DIV 的 Bypass_final_rem_plus_d_reg[2] 在映射中的保留差异；两版对应 DIV 生成 RTL 相同。综合会跨逻辑进行优化和裁剪，因此面积变化由完整映射结果给出，并非简单按某段新增逻辑逐门相加。

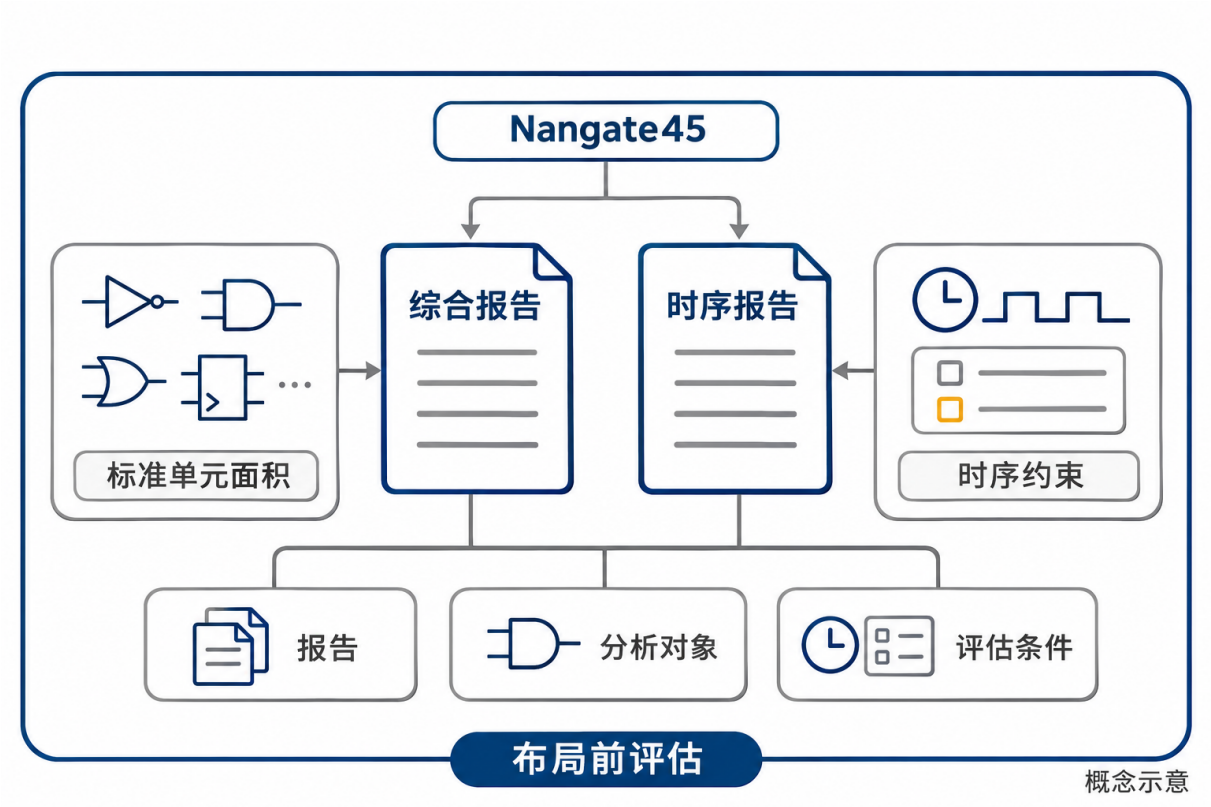
H1 的 32 位保存优化与 0f 的门控修复分别记录。前者直接利用固定结果布局减少状态，后者改变父级取消后的清理控制。评估它们时，既查看局部模块，也查看共享 ready、门控及父级输出路径，能够观察局部优化在集成环境中的整体代价。

3.5.3 时序约束与路径分类

静态时序使用 1ns 周期、5fF 负载作为名义档，并配置理想时钟、0.05ns 时钟不确定度与 transition、0.1ns 输入输出 delay、0.05ns 输入 transition 和固定线负载模型。另设 0.5ns、2ns 以及 1fF、20fF 负载档，在同一映射网表上观察约束敏感性。

路径按输入、寄存器和输出分类，记为 i2r、r2r、r2o、i2o，同时单列 ready、门控使能及父级中的 VDot 子集。到达时间描述数据传播，要求时间描述路径预算，裕量是两者的差。比较时把路径类别、端点和约束放在一起，便于区分组合路径变化与外部预算变化。

名义档下，FU 与父级所报 i2r 路径的 setup 裕量分别为 −0.268520ns 和 −2.393998ns，当前映射结果尚未满足 1ns 目标。五档共 70 项类别与条件对照中，到达时间增大 30 项、减小 15 项、不变 25 项。条形图展示 0f 在 1ns / 5fF 名义档下的 14 类时序裕量，负裕量位于零线左侧。五档完整 70 项数值、修复前后的到达时间差及实际起止点保留在图数据附件与完整对照表。



概念示意：面积与时序的不同约束。技术依据见图 3-8。

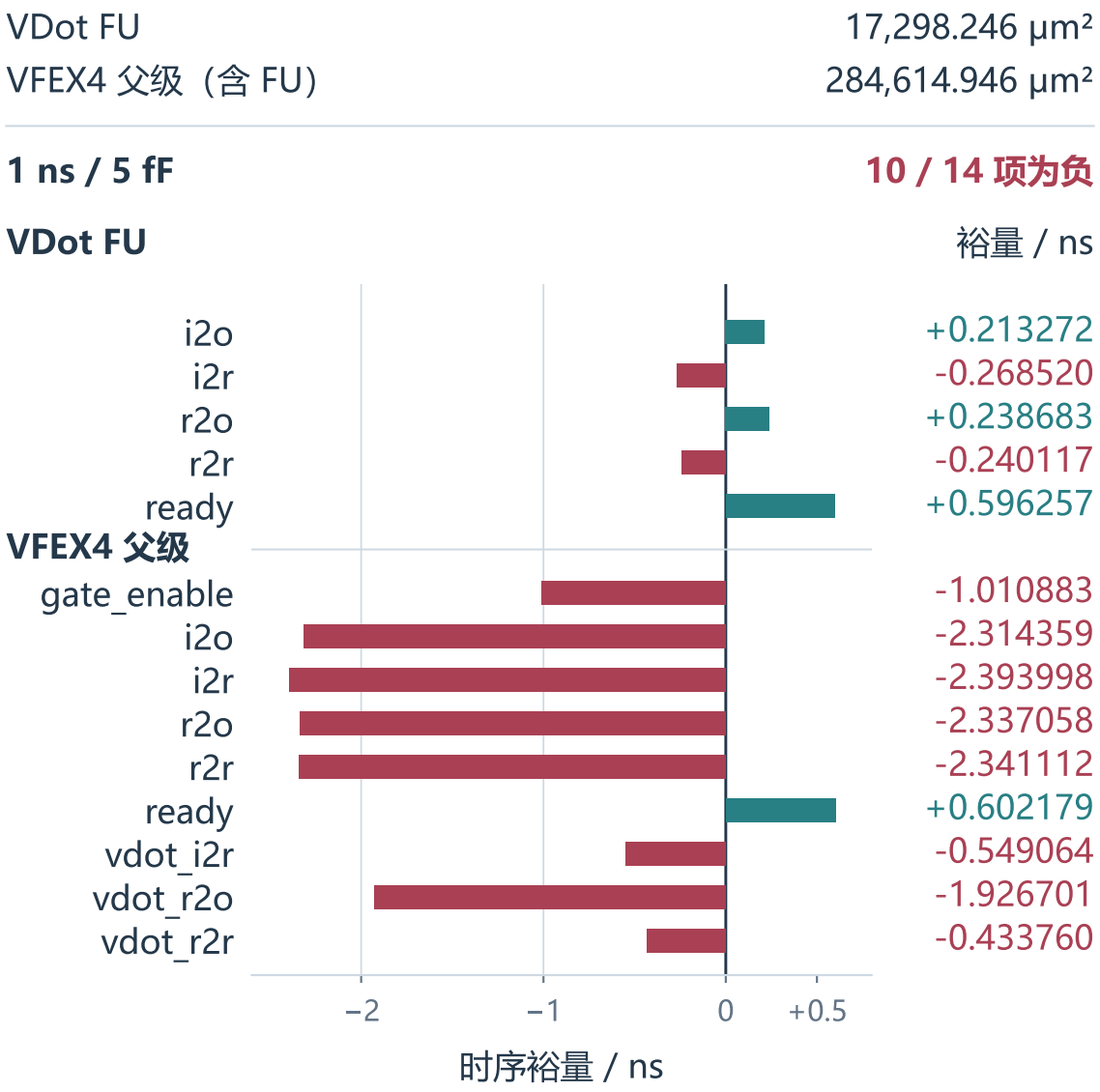


图 3-8　Nangate45 面积与时序裕量

Nangate45 Typical 1.10V、25°C 下的布局前评估。条形图展示 0f 在 1ns / 5fF 名义档的 14 类裕量，其中 10 类为负；五档 70 项完整数值保存在图数据附件。父级面积包含 FU。

3.5.4　父级关键路径与改进方向

0f 修复后的父级 ready 路径到达时间降低约 47.603ps，而 r2r、i2r 及 gate_enable 类别的最差到达分别增加约 94.304ps、119.132ps 和 32.965ps。部分路径改善与部分路径退化同时存在，说明完整父级中的控制、算术和门控网络对修改有不同响应。

端点回溯显示，两版 r2r 最差路径均位于 VFDiv，i2r 最差起点由 vsew 输入转为 src 输入，gate_enable 最差起点则在不同 VFDiv 实例间切换。类别中的最差路径可能随版本改变，因此优化时应检查实际起止点、逻辑深度与扇出，而不是仅根据类别名称判断同一条路径缩短了多少。

这些布局前结果给出了下一步时序工作的具体入口。可以优先分析完整父级中限制

名义档的路径，再以相同库和约束建立候选对照。布线寄生、多工艺角、整核 Fmax 和功耗属于后续物理评估内容；当前功能仿真、周期比较和静态时序分别描述已有工程的不同方面。

3.5.5 取消关钟与成本结果

DIV 取消后的清理需要经历明确的状态转移。父级识别匹配取消，子单元获得完成清理所需的边沿，随后撤销在途状态并回到空闲关钟。局部测试在 48 个取消窗口及其后 144 个采样拍中检查多余钟脉冲，观察目标是取消处理之后的时钟活动是否按预期停止。

门控输入使能与沿后采样的子时钟掩码代表不同位置的信号。使能参与 ClockGate 控制，掩码描述采样时哪些子时钟处于活动状态。结合占用、取消和清理事件阅读这些记录，能够解释为什么取消当拍仍需要一个有效清理边沿，以及何时可以重新接收其他功能单元的请求。

脉冲观察回答控制行为问题，面积和 STA 回答实现代价问题。动态功耗还取决于电容、电压、频率及各节点开关活动，需要相应模型与活动输入。当前评估据已有记录给出关钟行为、周期保持和同约束映射结果，后续功耗实验可以在相同功能场景上继续采集活动，保持比较对象一致。

3.5.6 门控修复的工程取舍

0f 解决了匹配 DIV 取消后的无效关钟保持。局部测试在 48 个取消窗口和其后 144 个采样拍中确认无多余钟脉冲，配套全核回归与周期比较进一步检查功能和执行代价。

修复的直接目标是恢复正确的取消与关钟行为。周期对照评估其执行代价，完整父级映射评估其面积与时序影响；动态功耗需要结合开关活动和功耗模型进一步测量。

最终工程组合采用 0f 修复：固定微基准周期保持一致，空闲关钟缺陷得到处理，父级面积小幅下降；名义档 r2r、i2r 及门控使能的到达时间分别增加约 94ps、119ps 和 33ps。后续时序优化可从这些实际路径入手，在相同库与约束下建立新候选对照。生产硬件对应提交为 0f12df675378556670afd0dd2153f47fcb330610。

3.6 验证汇总与设计结论

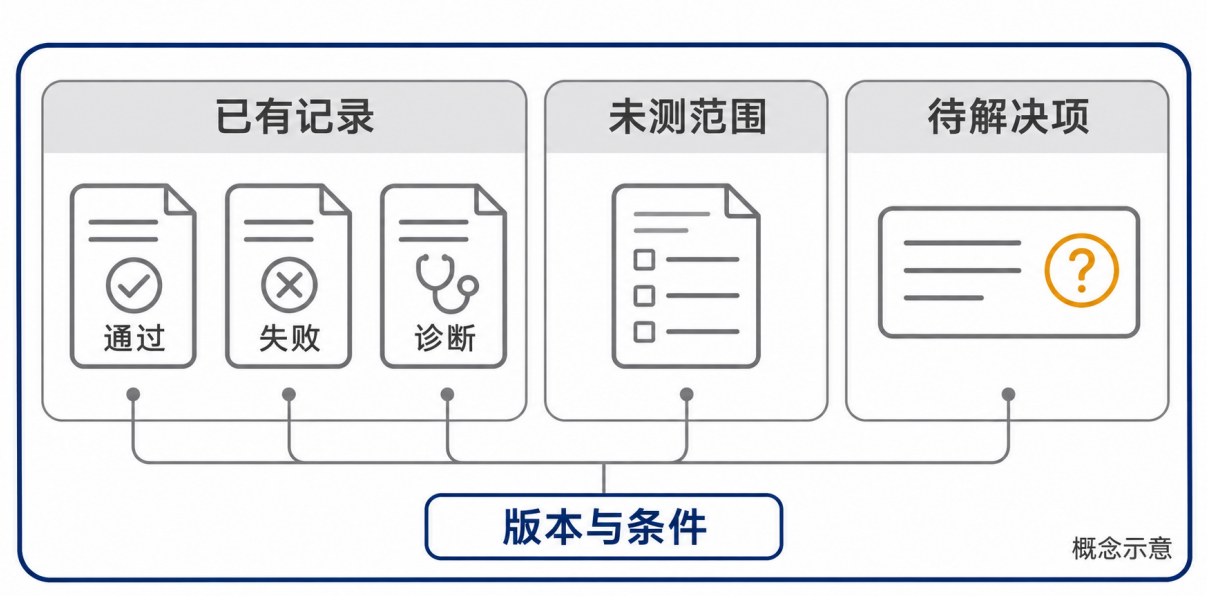
3.6.1 功能、复现与性能检查汇总

表 3-8 功能、复现与性能检查汇总

项目	验证结果	方法与结论
源码与输入身份	通过	0f 生产源码 398 项与 clean 副本及选定 Git 版本一致；完整工作树快照 24157 项覆盖源码和软件材料。
环境部署验证	通过，采用原记录	2026-10-09 hello/vadd 日志重新核对；阶段一 12 文件 SHA 及 220295598B 波形记录完成定位。
指令规格	内部合同已实现	SEW8/VLEN128，8 对 signed int8 点积，低 32 位结果、高 96 位清零；Q01–Q07 保留为待官方澄清项。
参考模型	主机检查通过	重新检查独立模型及确定性数据，覆盖 22 组定向与 10000 组随机输入。
功能单元与父级	原记录复算通过	0f 真实 FU 与 VFEX4 父级覆盖完整结果、反压、取消、身份、异常和严格 DIV 关钟；167 份原载荷及 4 ＋ 16 负控记录纳入复核。
整核协同仿真	组合验收通过	13 项原记录与 1 项新控制诊断，覆盖 117 个不同用例、124 次执行、O2 控制及负控恢复；宿主 UTC 问题按前述占用审计单列。
点积微基准	原周期记录复用	0f 修复前后 96 对正式窗周期一致；独立 batch128 为 221 对 678 cycles，加速比 3.067873 倍。
通用 GEMV 入口	输入复核与复现通过	四尺寸 768 窗口及 96 个普通 C 输出复现一致；33 个矩阵进程与 3 个普通 C 进程构成原运行集合，4×13 选择 RVV。
8×32 专用入口	最终镜像复核通过	627 项输入材料与原普通 C 镜像绑定，BIN、PT_LOAD、入口、48 表项和 12 数据组一致；原性能矩阵为 273 对 299 cycles。

表 3-8　功能、复现与性能检查汇总（续）

项目	验证结果	方法与结论
独立位宽扩展	独立单元验证通过	s16x8/r64 算术与包装扩展在原实验环境中复算；生产指令和 NEMU 采用 INT8。
面积与时序	成本复核通过，时序待收敛	0f 面积与五档 STA 的 1008 份文件回读；FU17298.246、父级 284614.946 平方微米，1ns 名义档存在负裕量。
独立构建复现	同机复现通过	独立源码、依赖、EMU/NEMU、AM/klib 和 25 个应用重建；复现使用同一主机，并复用工具及下载缓存。
entry-rollback	入口与回退明确	C8 静态库入口、RVV 回退、firstdot 原入口及 O1/B0 历史对照均保留。



概念示意：已知证据与尚未覆盖的边界。技术依据见图 3-9。

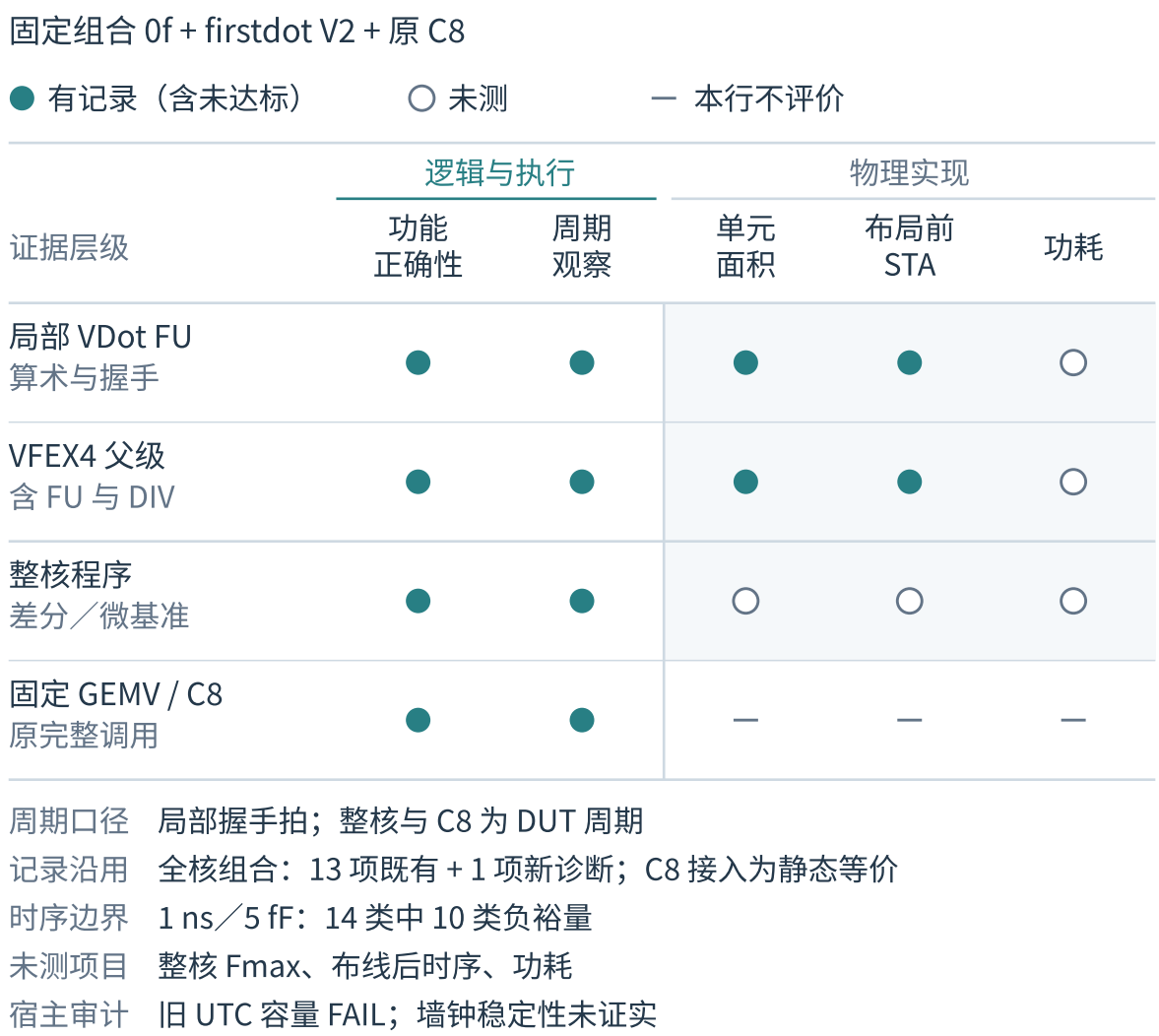


图 3-9 稳定组合的验收范围与保留边界

实心圆表示有相应记录，包含未达标结果，不等于 PASS；空心圆表示未测，横杠表示该软件证据行不评价物理实现。局部握手拍与整核、C8 的 DUT 周期分别记录。全核组合采用 13 项既有记录与 1 项新诊断，C8 接入为静态等价。旧 UTC 容量审计 FAIL、墙钟稳定性未证实均保留；1ns 名义档 14 类中 10 类负裕量，整核 Fmax、布线后时序和功耗未测。

3.6.2 优化候选的实测取舍

表 3-9 优化候选的实测取舍

探索	结果	最终处理
输入分离候选（P1/P1-R1）	对照窗口偏差超过预设 1% 限值	暂缓
单拍两槽候选（P4）	47 项成本检查中 6 项失败	未采用
单拍两槽修订（P4-R1）	47 项成本检查中 10 项失败	未采用

表 3-9 优化候选的实测取舍（续）

探索	结果	最终处理
C8 分段归约	实测 273.75→273 周期，一致性检查与 3% 收益目标未通过	未采用
FP32 到 QKV 探索（P3）	未取得完整计时窗口，尚未确认完整调用能否在 7200 秒预算内完成	暂缓

综合这些结果，工程选用 0f ＋ firstdot V2 ＋原 C8。应用评估包括点积微基准和固定 GEMV 调用；另有 INT16 独立单元扩展实验，生产指令保持 INT8。

3.6.3 阶段结论

本项目完成自定义指令硬件接入、独立 NEMU 扩展、分层功能验证及等价 RVV 周期分析。O2 提升独立吞吐，H1 降低结果状态成本，0f 处理 DIV 取消关钟，firstdot 与 C8 改善固定软件调用。固定源、镜像和样本支持结果复算，1ns 时序收敛及完整物理评估构成后续开发方向。

NEMU 补丁说明 • 测量合同 • 0f 性能与成本 • 版本数据对照

3.7 工程复现资料

3.7.1 固定版本与主要运行产物

测量软件以 T13 实现为基础，原文件及 SHA-256 保存在 team/references/zufan-t13/，对应提交为 f090665130107282be0990a7664aebf26e491b87。测量实验按自己的构建、镜像和运行记录建立样本集合。

该测量硬件源码提交为 4460300e9557ad20ac152fa67b1827c5d62802dd，工作负载的构建提交另行记录。

NEMU 以 a48a11f208ff81ad765591d83271d1b339fb3861 为基点，使用 vdot 补丁 1.1.0 及 xs-16ae9dd-vdot-runtime-tval-v1 平台配置。补丁绑定 7 个变化文件，SHA256 为 6c16 5bf5f94ef0bfb4f894a5f06eb6d620d4a7e0e12f3b1400fa72811f5d24db。

- 本章引用的历史资料发布提交为 eca41d7cd959f5f3c4f240e382919c492d1ccb96。
- 生产硬件提交为 0f12df675378556670afd0dd2153f47fcb330610。

- clean-02 EMU 的 SHA256 为 acf672eb39f836da4000b253849e3214089e9c8e5198060a57d2cc6bbd393da9。
- reference 的 SHA256 为 62b1707c52a4c323468481e0c1863b79f244d9a3fd54aeb7b7d2ecb5f01a35ca。
- C8 最终 BIN 的 SHA256 为 56805506eb98b68269c73a3645d8f75dff73b697e30dec5f93ccc4716089ad34。

以上标识对应本章采用的工程组合及主要运行产物，分项实验使用各自记录的构建与镜像身份。

3.7.2 两槽结构对照的样本记录

O1 与 O2 使用同一组 16 份冻结镜像，共执行 36 项（32 次首次镜像运行＋ 4 次代表 batch 镜像重放），478 个校验窗口，192 条正式性能窗口。每场景每模式每方法为 3 数据集 ×4measured ＝ 12 条样本；first/warmup 和重放完整保留，重放不增加独立样本数。周期降低按 $1 - \frac{\sum_i C_{\mathrm{O2,custom},i}}{\sum_i C_{\mathrm{O1,custom},i}}$，加速比按该模式本次实测的 $\frac{\sum_i C_{\mathrm{RVV},i}}{\sum_i C_{\mathrm{custom},i}}$。式中 C 为窗口周期数，i 为同一统计组内配对的正式窗口索引。每组逐 dataset/repeat/seq/镜像配对，机器码、工作量及退休指令一致；未扣循环/屏障或空窗开销。所有均值由该批正式窗口计算。single 对应完整调用，batch 对应四路同输入、独立目的的批量点积。

每个作业通过 DiffTest、完整 128 位输出、非 scratch 寄存器、CSR、哨兵和计数窗口检查，并满足各自 O1/O2 协议及有限排空条件。manifest 列出各内核使用的 scratch 集合，scratch_control 单独验证允许变化与受保护寄存器的划分。额外 4 次 batch 重放逐 seq 周期与退休数一致。30 项主机测试通过；全部原始 stdout/stderr、真实退出码、版本、输入和检查器摘要保留。

3.7.3 硬件对照与工具身份

表 3-10 硬件对照与工具身份

项目	O1	O2
实测 EMU SHA-256	6699225b76ceae479060741026683bb03de069c1e10a4a94e463304ba1e322d0	ac0123280bf6c742d27a7650505bc1d91db350d1519806edfd0be9c7ea208a57
构建源码提交	a239dace720b32216e83fb5a514930d334c69a26	b218cbc6e6b135e4ff219a902a9284a59d3c82c9
来源	从同一硬件输入重建	复用 G3 已验收固定 EMU

表 3-10 硬件对照与工具身份（续）

项目	O1	O2
O1_HANDOFF / O2_ELASTIC	1/0	0/1
O1_TRACE/O2_TRACE	1/0	0/1

429 项硬件源码与 21236 项依赖逐字节相同，MinimalConfig/单核/VLEN128/单线程 EMU；共同 FUSED_INPUT1、T12_TRACE0、T13_TRACE1。各自合法协议观察器只提供只读观察，共同 T13 提供全局周期/退休计数。

路径归一化后的实际 Verilator/C++ 命令、七类工具版本日志一致；实际 LLVM19.1.7 编译器由构建过程/proc 记录并核对。j2、12G JVM、O3/O0；最多两个单线程 EMU。新 O1 采用外置产品目录，原 G3 的 1687 份非对象生成文件已恢复并验证原始 SHA，旧 O2 EMU 不覆盖。构建记录分别列出硬件源码、实际工具和封存提交，便于对应实验输入与结果。

3.7.4 离线复算与模型数据

Windows 原生 Python 3.13.2 全量离线复算通过，覆盖 36 项作业、478 个窗口、192 条正式性能窗口及 4 项重放。见原生复算回执。复算直接读取封存的作业和窗口记录，Linux 构建来源与 Windows 复算回执相互对应。

交付时补齐了原冻结功能生成器依赖的两份模型语料，逐字节取自运行工具提交 63b95a5 并绑定 Git blob 及 SHA-256；原镜像、测量日志和 646 份主封存 payload 均未改变。原 30 项主机测试及新增 3 项语料绑定检查共 33 项通过；首次封装复算失败和修复后成功记录均保留。Windows 请用 Python UTF-8 模式执行：

```
python -X utf8 -B performance/portable.py --archive evidence/20261019-performance-ab --campaign
campaign-01 --output /absolute/new/portable-verification.json
```

上述相对路径适用于 Windows 交付目录；仓库内对应 team/optimizations/o2 下的路径。

3.7.5 取消关钟修复的对照输入

表 3-11 取消关钟修复的对照输入

对象	修复前	修复后
实现	O2 两槽＋ H1 低 32 位保存	O2 ＋ H1 ＋ DIV 取消关钟修复

表 3-11 取消关钟修复的对照输入（续）

对象	修复前	修复后
硬件源码	8b5024fcbd96986871cc40ce8c9fb71a67c495eb	0f12df675378556670afd0dd2153f47fcb330610
EMU SHA256	e37b3c267cae29036bda0b6952a17db2523ca0b3a573d45281341d4207df8747	2d1db77cc407486a2387a4a69da2498b68d44980239025eebcc02370c02826e8

硬件 429 份 Git 源码、21236 份依赖、每版 1672 份生成产物、实际编译命令和七类工具版本均经核验。生产源码仅 ExeUnit.scala 不同。

同一 MinimalConfig/单核单线程/O2 与 T13 观察配置、128MB、seed=1、正常 NEMU 和 flash；208 份冻结软件载荷不变。

正常 NEMU SHA256：efbed0774addc27eac38af1a19ab5d9c9dd5f179b5015f25ae7b80b966185bd4。

生成物差异包括 Git 标识、只读诊断日志、顶层注释及标量 FDiv 公共子表达式内联；这些差异与生产源码变化一起纳入生成物审查。

3.7.6 修复前后逐窗周期核对

表 3-12 修复前后逐窗周期核对

场景	修复前 vdot 均值	修复后 vdot 均值	两版 RVV 均值	两版 RVV/vdot
单次点积 1	71.000	71.000	85.250	1.200704×
依赖链 256	1857.000	1857.000	5440.500	2.929725×
四路独立批量 128	221.000	221.000	678.000	3.067873×
加载/计算/存储 64	1127.500	1127.500	1133.000	1.004878×

单位均为 DUT 全局 cycles。每场景三组数据 × 四轮，vdot 和 RVV 分别配对；192 正式窗口形成 96 对，每对差值均为 0，退休指令与工作量相同。

共 478 个校验窗口，保留 first/warmup；四次额外 batch 重放完全一致，不加入正式样本。36 项实际 EMU 均通过完整 128 位结果、DiffTest、GOOD TRAP、窗口计数和 O2 生命周期/排空检查。

各场景加速比按 $\frac{\sum_i C_{\mathrm{RVV},i}}{\sum_i C_{\mathrm{vdot},i}}$ 计算，其中 C 为窗口周期数，i 为该场景中配对的正式窗

口索引；保留共同计时开销。修复前后逐窗周期一致，表明 0f 在这组固定微基准中保持了原有性能。

详见逐窗差值、全部样本、复算回执。

3.7.7 综合面积与完整接口

修复前采用 H1 封存的 205 份生成、综合和 STA 文件，经逐字节核验后作为基线；修复后重新生成 RTL 并完成综合与 STA，形成同条件对照。

两边使用相同完整生产接口、相同未修改 O4 脚本、Yosys0.9/ABC1.01/OpenSTA2.0.17 及其动态库、Nangate45 typical 25°C/1.1V 库。

ABC 目标 1000ps、BUF_X1 驱动、5fF 映射负载。面积是 Liberty 单元面积之和，单位 μm²，非芯片布局面积。

表 3-13 综合面积与完整接口

边界	修复前面积	修复后面积	增量	增量比例	DFF 数量	ICG 数量
fu	17298.246	17298.246	+0.000	+0.000000%	1742→1742	0→0
parent	284879.882	284614.946	-264.936	-0.092999%	17171→17172	3→3

父级映射 DFF 增加 1 个，已回溯到既有整数 DIV 的 Bypass_final_rem_plus_d_reg[2]：旧网表该位无驱动和消费者，新网表保留 DFF_X1。两版对应 DIV 生成 RTL 相同，没有新增 RTL 寄存器声明或 VDot 状态位；该差异来自整体综合优化与裁剪，面积因此按完整映射结果统计。详情在归档 dff-review.json。

两版 VDot 均保存 32 位结果，并在公开 128 位输出处补齐高 96 位零；算术与控制/perf 寄存器分组保持一致。父级 DIV 使用自己的完整结果格式。

FU 生成 5 份文件完全相同。父级 58 份生成文件仅 ExeUnit.sv 和 LogPerfEndpoint.sv 不同；后者只改变打印顺序，并被实际 SYNTHESIS 定义排除。

3.7.8 五档时序详细结果

同一映射的五档条件为 1ns/5fF、0.5ns/5fF、2ns/5fF、1ns/1fF、1ns/20fF。理想时钟、0.05ns uncertainty 与 transition、0.1ns 输入/输出 delay、0.05ns 输入 transition、5K_hvratio_1_1 线负载。parent testEnable=0，不额外排除 ready/reset/flush 路径。

每版 70 条类别/档位路径对照，父级网表规范化经逐 bit 拓扑绑定；未知单元和动态未约束端点拒收。以下 nominal 到达时间包含相应外部输入延迟；表格按路径类别比较，各版实际最差端点另行列出。

表 3-14 五档时序详细结果

边界/路径	修复前 arrival(ns)	修复后 arrival(ns)	差值 (ps)	修复后 slack(ns)
fu/i2o	0.636728	0.636728	+0.000	0.213272
fu/i2r	1.190323	1.190323	+0.000	-0.268520
fu/r2o	0.611317	0.611317	+0.000	0.238683
fu/r2r	1.163404	1.163404	+0.000	-0.240117
fu/ready	0.253743	0.253743	+0.000	0.596257
parent/gate_enable	1.855397	1.888362	+32.965	-1.010883
parent/i2o	3.128214	3.164359	+36.145	-2.314359
parent/i2r	3.194971	3.314103	+119.132	-2.393998
parent/r2o	3.185769	3.187058	+1.289	-2.337058
parent/r2r	3.166914	3.261218	+94.304	-2.341112
parent/ready	0.295424	0.247821	-47.603	0.602179
parent/vdot_i2r	1.511395	1.474949	-36.446	-0.549064
parent/vdot_r2o	2.832115	2.776701	-55.414	-1.926701
parent/vdot_r2r	1.345889	1.356106	+10.217	-0.433760

70 条类别/档位比较中，到达时间变大 30 条、变小 15 条、相同 25 条。全部差值和端点见完整时序 CSV。

nominal 所报路径中，修复前有 10 类负 slack，修复后有 10 类。两版在 1ns 目标下均未收敛；整核 Fmax、布线后时序与功耗列为后续评估内容。

最差端点回溯已保存 path-endpoint-review.json：前后 r2r 均是 VFDiv 的 fsm_q[0] 到 quo_m1_iter_q_f64_0[1]，i2r 最差起点从 vsew 输入变为 src 输入；gate_enable 最差起点从 vfdivs_1 切到 vfdivs_0，终点仍是浮点 DIV 的 ICG。这些端点属于完整父级的时序分析对象，端点变化与到达时间变化共同用于解释取舍。

3.7.9 复算入口与实验归档

主机检查共 62 项，包括周期配对 30 项、验收身份 6 项、成本审计 20 项和归档拒错 6 项，均通过。预检中曾发现验收 JSON 字段适配和生成范围遗漏日志模块两项问题，修订后完成复核；对应失败回执与成功结果一并保存。

原始周期根：/home/riscv/riscv-lab/o2-r3-div-gate-performance-20261025。

原始综合根：/home/riscv/riscv-lab/o2-r3-div-gate-cost-20261025。

原 H1 完整 build：/home/riscv/riscv-lab/o2-h1-build-before-r3-div-gate-20261024。

原始 stdout/stderr、真实退出码、命令、源/依赖/镜像/工具/库哈希及所有路径报告随归档保存；EMU/NEMU 和 EDA 可执行文件保留在本地，用 SHA 绑定，不放进日常 Git 提交。

周期和成本复算入口分别为 performance/recheck.py 与 cost/compare.py，使用独立输出路径保存新回执。package_evaluation.py --verify ARCHIVE --receipt NEW_JSON 用于核对归档字节。完整归档保存源、工具、库、镜像和路径报告的对应关系，可据此复算统计或重新安排相同输入的实验。

3.7.10 C 接口与编译开关

表 3-15 C 接口与编译开关

入口	目标
xs_c8_gemv_8x32，开关缺省或 0	config_reuse_gemv_8x32_reuse_vdot
xs_c8_gemv_8x32，XS_C8_GEMV_FORCE_RVV=1	config_reuse_gemv_8x32_reuse_rvv
xs_c8_gemv_8x32_rvv	始终 config_reuse_gemv_8x32_reuse_rvv

XS_C8_GEMV_FORCE_RVV 最终展开只能是字面量 0 或 1；空值、表达式和未知标识不受支持，编译时拒绝。独立 guard 为 XS_C8_SELECTED_GEMV_H，API 版本 XS_C8_GEMV_API_VERSION=1。XS_C8_GEMV_PROFILE 为静态配置标签，CPU 能力由调用环境确认。修改宏或 API 版本后，相关翻译单元一同重编，以保持调用点与库选择一致。

C8 使用独立的开关、guard 和入口别名，与原 XS_GEMV_FORCE_RVV 及 xs_gemv_* 接口并存。应用选择 C8 或 firstdot 时，分别使用对应的头文件与静态库。

3.7.11 对象构建与接入命令

独立 GNU 汇编参数固定为 -march=rv64gcv -mabi=lp64d -mno-relax -c。完整 gemv.o 为 27,160 字节，SHA256 为 f8bc13daeead98a414f313600a5778c96ac95a66b5bcb5c714ea3b98e0d25ab3。确定性 libxs_gemv_c8.a 只含一个 gemv.o，保留 16 个函数、原节和重定位；config_reuse_gemv.h 从合格 gemv.h 原字节复制，仅改产物文件名。

软件包绑定源基点 c823a8786e052c6e26f6915563c42b6d730dd8e3、选中 RTL 0f12df

675378556670afd0dd2153f47fcb330610 和 clean-02 配套身份。完整输入、工具身份及验收结果以 INPUTS.json、验证器输出和版本数据档案记录为准。验收同时检查对象内容、链接关系与运行输入身份。

包级静态检查覆盖完整对象、单库成员、缺省及 0/1 配置下两别名的 GNU 调用对象、可重定位链接等价、符号解析、错误配置拒绝和 SDK 搬移。example.c 提供编译与链接示例。随后的小节给出接回原 C 程序的最终镜像等价结果，将包级检查与原矩阵、普通 C 调用的功能记录衔接起来。

接入新应用时，根据 wrapper、分派、CSR 保存、对象组合和最终布局的变化安排目标运行；性能比较同时计入调用者实际承担的工作。叶函数字节核对用于确认计算主体，完整镜像和调用计划则用于确认最终运行方式。

本包内设 XS_C8_GEMV_FORCE_RVV=1 或显式调用 xs_c8_gemv_8x32_rvv 后完整重编。撤销 C8 选用时，恢复原 firstdot 头和匹配库并完整重编。C8 稳定别名对应 8×32，其余尺寸沿用各自选定的 firstdot 或 RVV 入口。

3.7.12 链接对象与检查入口

- 原主程序只把底层头的包含改为 selected_gemv_c8.h。
- 8×32 reuse vdot/RVV × 三数据集，共 6 个函数表项改用 xs_c8_gemv_8x32 / xs_c8_gemv_8x32_rvv。其余 42 项、数据、期望值、顺序和校验不变，测试向量沿用原计划。
- 真正从 libxs_gemv_c8.a(gemv.o) 链入完整合格对象；SDK 程序源目录没有 gemv.S，对象目录没有绕过库的 gemv.o。GNU 依赖文件和 link map 确认实际包含 SDK 头、实际抽取库成员。
- 在全新输出中分别构建原源码复制对照和 SDK 接入程序，保留原 AM/klib 和链接次序；只有输出路径和记录链接来源的选项不同。

接入仅调整头文件与六个 8×32 函数表项，其余调用计划和数据保持一致。

3.7.13 宿主运行记录与并发审计

原 14 项全核记录的 UTC 容量检查出现 80.119ms 表观四路重叠，状态为 FAIL/WALL_CLOCK_LIMITED。后续按既定合同重新执行 o2-control-diagnostic，其余 13 项从原始日志重新解析，形成组合验收。

最终组合 14 项通过功能、NEMU 差分、O2 覆盖和负控后同镜像恢复。**组合采用 13 项复用记录与 1 项新执行记录**。选定记录的严格 UTC 并发峰值为 3（上限 3）；新单进程以同 boot/PID 的单调时钟包络验证，峰值 1，包络 1497.000 秒。该耗时是宿主审计数据，不用于 DUT 加速比。

新控制诊断检查 131072 字节寄存器结果，退出 0；独立离线审计再次通过。原 13 项的占用记录采用 UTC，新控制诊断另有单调时钟包络。

本次 UTC 间隔与单调包络仍相差-62.888356 秒（UTC 更短）。因此墙钟稳定性仍未证实；新占用审计依赖同 boot 的单调包络，而非容差修正 UTC。原 UTC 容量失败及两种时间基准的差异均保存在审计记录中。

0f真实FU与VFEX4父级；完整128位、反压/取消/身份/异常、严格DIV关钟及4+16负控；复用0f原167载荷；未新执行RTL仿真，不转授C1/C2/C3
固定原13项重新解析+新o2-control-diagnostic；117不同/124次功能例，O2覆盖及负控恢复；组合验收不是fresh14；原14项UTC容量FAIL保留；本次UTC间隔比单调时钟短62.888356秒，系统墙钟稳定性未证实，新占用审计采用单调包络

(a) FU、父级与全核验证记录

0f同约束面积/五档STA资料1008文件回读；FU17298.246、父级284614.946微米平方；不是新综合；1ns负slack保留，整核Fmax/布线后时序/功耗NOT_TESTED

(b) 面积与时序记录

图 3-10 R5 功能与成本验收摘要

功能条目记录局部与全核验证的采用情况；成本条目记录面积结果和 1ns 目标下的负裕量。

3.7.14 工程使用与回退资料

当前 1ns STA 尚未收敛，功耗、整核 Fmax 和布线后时序待评估。INT16 扩展已有独立单元结果，生产全核继续采用 INT8。4×8 和 16×64 的配置复用控制差异保留为后续分析方向，4×13 采用 RVV。内部规格 Q01–Q07 仍待官方澄清。

选定的 0f ＋ firstdot V2 ＋原 C8 作为稳定工程组合，保留可使用入口与回退路径。后续候选围绕父级关键时序、软件调用配置及实际应用工作量开展比较，沿用已经明确的输入、镜像和样本组织。

冻结文件 24157 项，生产源码 398 项与 0f 及 clean 副本一致。快照在 /home/riscv/riscv-lab/software-import-20261026/r5-g5-acceptance-01/frozen-workspace.tar.gz，已逐项回读，含工作树文件但不含 Git 历史和外部依赖。使用/回退见 README，逐项身份见数据记录，原始新增证据见 manifest。失败尝试与原 28 条数据档案共同保存在工程记录中。

3.7.15 技术资料索引

周期测量边界与统计合同；O1 与 O2 同镜像验证过程；0f 硬件性能与五档成本详细记录；C8 调用 ABI、对象及回退合同；C8 最终链接和镜像等价验证；R5 组合验收和证据复用。